B1 Il livello 3: Rete

Login

Il 29 ottobre 1969 alle 22.30, nello Stanford Research Institute di Menlo Park (California), il programmatore Bill Duvall riceve sul suo calcolatore (uno Scientific Data System 940), la stringa «**lo**».

A inviarla è stato Charles Kline, suo collega dell'Università della California di Los Angeles.

Il sistema di comunicazione, infatti, dopo aver inoltrato correttamente la «**l**» e la «**o**», va in crash e non riesce a spedire «**gin**».

Questa è stata la prima parola «pronunciata» dalla rete Internet.

La supervisione del progetto fu a cura di **Leonard Kleinrock**, che impiegò il primo rudimentale router denominato **IMP** (*Interface Message Processor*).

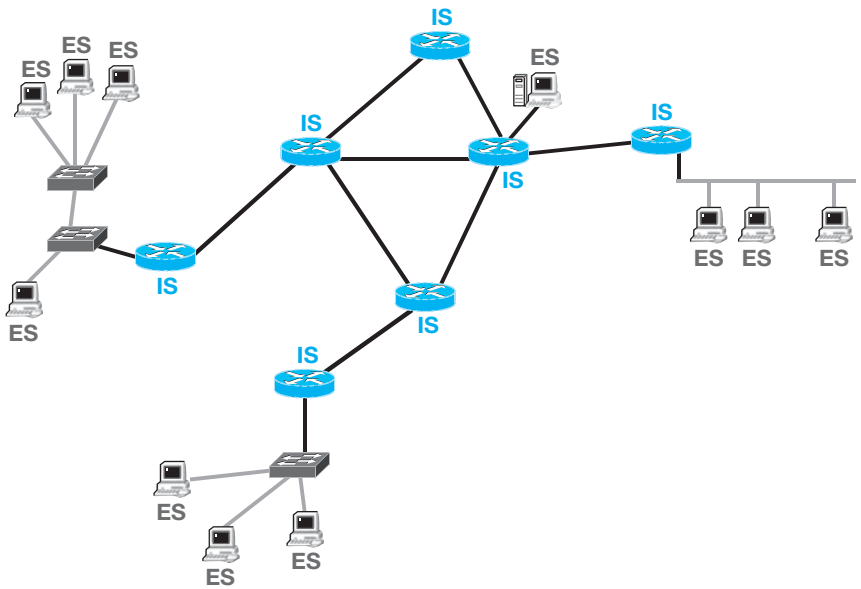
A partire dal 5 dicembre dello stesso anno si collegarono alla rete anche l'Università della California di Santa Barbara e l'Università dello Utah, come stabilito nel progetto originario.

In una rete con topologia a maglia come una WAN, si pone il problema di far raggiungere la destinazione ai pacchetti dei messaggi, destinazione che non è direttamente connessa al mittente come nei modelli LAN.

In altri termini la rete magliata deve fornire un meccanismo per individuare un cammino valido da mittente a destinatario per qualsiasi coppia di stazioni **host** (o **ES**, *End System*) presenti sulla rete.

Questo è il compito del **livello 3 Rete** (*Network Layer*).

Il cammino deve coinvolgere quindi stazioni intermedie, denominate **router** (o **IS**, *Intermediate System*), che rappresentano i nodi intermedi della rete e che posseggono più di una interfaccia fisica per almeno due connessioni su reti differenti.



I nodi intermedi devono quindi possedere una serie di capacità che consentano di instradare i pacchetti da una interfaccia di provenienza a una interfaccia di destinazione.

Elementi base dei protocolli di livello 3 Rete (Network) sono quindi:

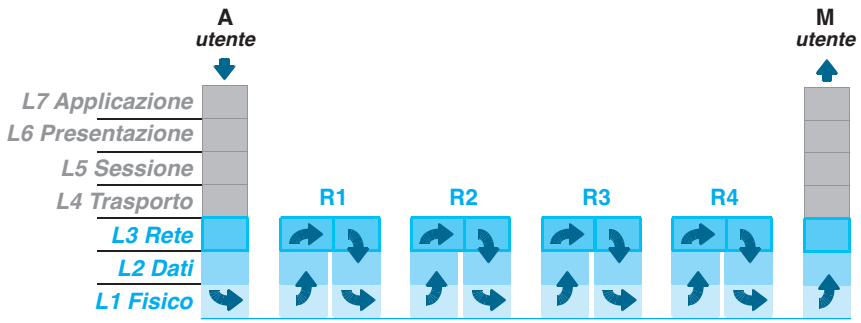
1. l'apparato **router** e le sue porte di inoltro;
2. gli **indirizzi** contenuti nei pacchetti di livello 3 Rete;
3. la **tabella di instradamento** (*TdI*), cioè i dati per decidere i cammini;
4. l'**algoritmo di instradamento**, le regole per creare la TdI.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Su un nodo intermedio IS deve essere implementato un algoritmo di instradamento (procedure) a sua volta basato su una tabella di instradamento (struttura dati). In sintesi:

Sulla ricezione di un pacchetto, il livello 3 Rete operante sugli IS apre la PDU (il pacchetto) di livello 3 per estrarne gli indirizzi di livello 3, quindi avvia l'algoritmo di instradamento che aggiorna la tabella di instradamento da cui decidere la porta di inoltro.

Per quanto esposto, la problematica dell'instradamento affrontata al livello 3 di rete viene quasi del tutto affrontata e risolta dai nodi intermedi IS, mentre i nodi ES non implementano in modo sostanziale il livello 3. D'altra parte i nodi IS non hanno spesso ragione di occuparsi delle problematiche dei livelli superiori al terzo, pertanto uno schema a livelli delle stazioni su una rete WAN che descrive una circolazione dall'host **A** all'host **M** risulta essere (vedi anche la figura del paragrafo ***Routing CNLS: by network address***):



Si può anche affermare che il livello 3 Network **isola i dettagli dell'hardware di rete** e li nasconde ai livelli superiori: gran parte del livello 3 e tutti i livelli superiori al 3 sono esclusivamente software.

1 Routing

La tabella di instradamento è una struttura dati di semplice concezione, costituita da un certo numero di righe che contengono le **regole** con cui decidere su che porta dell'IS un determinato pacchetto deve essere inoltrato. Questa operazione è detta **routing**.

Ogni riga è costituita da alcuni campi, tra cui *Indirizzo* e *Porta*.

Quando riceve un pacchetto di livello 3, l'IS ne estrae l'indirizzo destinazione, consulta le righe della TdI alla ricerca di tale indirizzo (nella colonna *Indirizzo*) e, individuata la relativa riga, determina la porta ove instradare dal campo *Porta* corrispondente.

Il campo *Indirizzo* può prevedere anche gruppi di indirizzi, in modo tale che non sia necessario avere una riga per ogni singolo nodo della rete.

Inoltre è possibile che esistano più regole che contengono lo stesso indirizzo (ma differente numero di porta), affinché si possano avere alternative di instradamento in caso di porte (o linee) guaste.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Gli altri campi delle righe di una TdI riguardano le cosiddette **metriche**, ovvero una metrica di **costo**, valutata sia in base ai costi economici delle varie linee, sia in base all'efficienza o velocità delle linee; e una metrica di salto (**hops**), che segnala quanti nodi intermedi devono essere attraversati prima della destinazione.

I campi di metrica servono per stabilire l'instradamento in caso di alternative di linea (più regole per la stessa destinazione) e, generalmente, la consuetudine suggerisce di privilegiare prima le linee con minor costo, quindi quelle con minor hops.

Gran parte dell'efficienza e delle caratteristiche degli algoritmi di routing si basa sui modi in cui la TdI viene creata e mantenuta aggiornata dagli IS.

La TdI ideale dovrebbe contenere tante righe quanti sono gli host di una rete e dovrebbe essere disponibile per ogni IS della rete.

Inoltre, a ogni modifica della topologia, ovvero a causa del succedersi di linee interrotte, nuove linee, nuovi IS e IS eliminati, nuovi ES ed ES eliminati, la TdI dovrebbe essere aggiornata immediatamente su tutti gli host.

Tutto ciò non è possibile su reti magliate anche di piccole dimensioni, pertanto gli algoritmi di routing in dotazione agli IS non possono operare in senso deterministico ma devono adeguarsi a criteri sostanziali di funzionamento, tra i quali:

- *Robustezza e versatilità*, per adattarsi senza particolari oneri alle variazioni della topologia.
- *Stabilità*: l'algoritmo deve convergere, cioè deve trovare un cammino in tempi ragionevoli; se non ci sono cambiamenti sulla rete, l'instradamento deve essere immediato usando la TdI.
- *Equità*: l'algoritmo nel calcolare la TdI deve evitare di danneggiare particolari nodi, anche se per questo deve utilizzare qualche cammino non ottimale.

Inoltre si possono stabilire degli obiettivi che costituiscono l'ottimalità di un algoritmo di instradamento, anche se spesso sono in contrasto fra loro:

- Minimizzazione del ritardo medio di ogni pacchetto (dal punto di vista dell'utente di rete).
- Massimizzazione dell'utilizzo della rete (dal punto di vista del gestore di rete).
- Possedere un certo grado di tolleranza ai guasti (*fault tolerance*).

Esistono sostanzialmente due approcci al problema dell'instradamento che, pur essendo simili per quanto riguarda il problema della ricerca del cammino, in seguito adottano tecniche di ritrasmissione completamente differenti; i due approcci riflettono le due categorie di reti che abbiamo già descritto più volte: reti non orientate alla connessione (*connectionless* o **CNLS**) e reti orientate alla connessione (*connection oriented* o **CONS**).

PROGRAMMA

Hops

Dato lo schema nella figura del paragrafo ***Routing CNSL: by network address****, scrivere un programma in linguaggio C che, digitati due numeri indicanti il router di partenza e il router di arrivo, stampi il percorso dell'ipotetico pacchetto.*

OUTPUT

```
Digitare router sorgente (1..4): 3
Digitare router destinazione (1..4): 1
Tratta: R3 R2
Tratta: R3 R2 R1  ok
```

Per rappresentare i collegamenti dei router R1, R2 R3 e R4 della figura si può usare una serie di triplette del genere:

`R1,R2,2`, cioè "sulla porta P2 di R1 è connesso il router R2",
`R2,R1,4`, cioè "sulla porta P4 di R2 è connesso il router R1", ecc…

In tutto risultano **6** triplette.

Il codice in linguaggio C potrebbe essere il seguente:

```
00 #include <stdio.h>
01 #include <conio.h>
02 #include <string.h>
03
04 typedef struct {
05   char ra[3];       // Il router ra è connesso...
06   char rb[3];       // ...al router rb ...
07   unsigned char p;  // ...sulla porta p (di ra)
08 } LINK;
09 LINK linkrouter[] =
   {{"R1","R2",2},{"R2","R1",4},{"R2","R3",2},{"R3","R2",5},{"R3","R4",3},{"R4","R3",1}};
10
11 int main (void)
12 {
13   char chS,chD, szHops[100], *pc;
14   int i,rm, rd, r, ok, n = 0;
15
16   printf("Digitare router sorgente (1..4): ");
17   chS = getche();
18   printf("\nDigitare router destinazione (1..4): ");
19   chD = getche();
20
21   rm = chS - '0';
22   rd = chD - '0';
23   r = -1;
24   szHops[0] = 0;
25   pc = szHops;
26   n = n + sprintf(pc+n,"R%c ",rm + '0');
27
28   while ((rm!=rd) && (n<70))
29   {
30       ok = -1;
31       for (i=0; i<sizeof(linkrouter)/sizeof(LINK); i++)
32       {
33         if (linkrouter[i].ra[1] == rm + '0')
34         {
35           ok = i;
36           if (linkrouter[i].rb[1] != r + '0')
37           {
38             ok = -2;
39             r = rm;
40             rm = linkrouter[i].rb[1] - '0';
41             break;
42           }
43         }
44       }
45       if (ok>=0)
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

PROGRAMMA

```
46        {
47          r = rm;
48          rm = linkrouter[ok].rb[1] - '0';
49        }
50        if (ok==-1)
51        {
52          printf("\nImpossibile instradare");
53          return 0;
54        }
55
56        n = n + sprintf(pc+n,"R%c ",rm + '0');
57        printf("\nTratta: %s",szHops);
58    }
59
60    (rm==rd) ? printf(" ok") : printf("\nErrore TTL!");
61    return 0;
62  }
```

Le triplette vengono controllate individuando la prima che consente al pacchetto di uscire dal router ma non sulla porta da cui è entrato (**righe 33÷44**).

Se nessuna tripletta soddisfa la condizione, si prende la prima tripletta d'uscita dal router, anche se è quella di provenienza (**righe 45÷49**).

Si nota che:

a se il mittente non è R1 e la destinazione sta alla sua "destra" (per esempio da R2 a R4), il programma "torna indietro" e poi risale verso la destinazione;

b se il mittente non è R4 e il destinatario sta alla sua "sinistra" (per esempio da R3 a R1), il programma agisce razionalmente;

c se invece si propone un input illecito tipo da R1 (1) a R9 (9), il programma continua a cercare il percorso indefinitamente (salvo segnalare la situazione con il messaggio "Errore TTL").

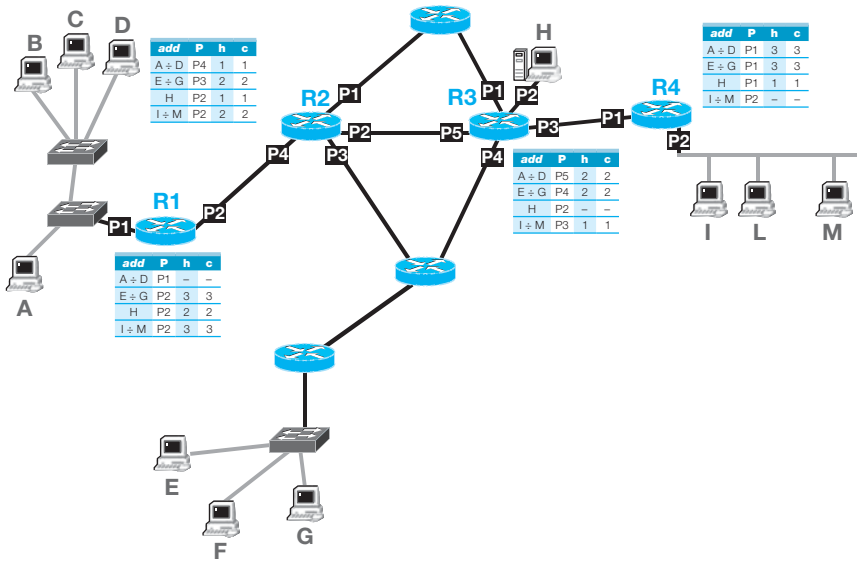
Questa situazione è realistica, come si vedrà più oltre (cfr. ***TTL in Pacchetto IP***).

2 Routing CNLS: by network address

Tramite il routing **by network address** il mittente e il destinatario dei pacchetti sono specificati apponendo in ogni pacchetto di L3 Rete i relativi indirizzi. Il pacchetto di L3 Rete rimane immutato per tutto il percorso, mentre la trama L2 Datalink viene modificata per ogni singola tratta attraversata adattandosi allo strato fisico della nuova linea: cambiano quindi gli indirizzi di livello 2. Inoltre l'IS (che nelle reti CNLS viene anche detto semplicemente **router**) deve prendere la decisione di instradamento per ogni pacchetto, consultandone l'indirizzo L3 di destinazione: gli IS usano l'indirizzo del destinatario del pacchetto come chiave per la propria TdI per determinare la porta su cui effettuare il successivo instradamento.

Questa operazione viene ripetuta a ogni transizione per un IS e per ogni pacchetto, perciò implica un rallentamento nel cammino. I pacchetti di un singolo mittente possono prendere direzioni anche differenti e arrivare alla stessa destinazione attraverso cammini diversi. Per due pacchetti destinati allo stesso host, la TdI dell'IS potrebbe essere cambiata in funzione di nuovi calcoli dovuti alla modifica della topologia (per esempio guasti o nuovi nodi), e quindi il cammino dei due pacchetti potrebbe essere diverso.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*



ESEMPIO

Routing CNLS

Dato lo schema di rete della figura sopra, mostrare, per ogni tratta, l'intestazione di livello 2 e 3 di un pacchetto che parte dall'host A e giunge all'host M.

Il pacchetto in partenza dall'host **A** circola inizialmente su una LAN e deve raggiungere il router **R1**. Il suo formato sarà quindi `r1 a A M`, dove la parte chiara indica l'intestazione di livello 2 Datalink (per esempio Ethernet) con gli indirizzi MAC destinazione e mittente (r1 e a), mentre la parte grigia è il pacchetto di livello 3 Network, con l'intestazione che riporta gli indirizzi mittente e destinazione (A e M).

Giunto su **R1**, il router elimina la busta di livello 2, legge l'indirizzo di livello 3 destinazione, consulta la TdI riportata in figura e inoltra sulla porta **P2** il pacchetto imbustandolo nuovamente con gli indirizzi di livello 2 adeguati: `r2 r1 A M`.

Seguendo lo stesso procedimento, la sequenza dei pacchetti sulle rimanenti tratte risulta:

- `r3 r2 A M` (tratta **R2-R3**, attraverso la porta **P2** di **R2**);
- `r4 r3 A M` (tratta **R3-R4**, attraverso la porta **P3** di **R3**);
- `m r4 A M` (sulla LAN di destinazione, attraverso la porta **P2** di **R4**).

Si nota che le TdI riportate non contengono nessuna regola alternativa per gli host, cioè in nessuna colonna *Indirizzo* (***add***) delle TdI in figura compare un host più di una volta.

Se tutto ciò diminuisce le prestazioni complessive (possibilità di pacchetti duplicati, pacchetti che arrivano in ritardo rispetto all'ordine di trasmissione, ecc.), il procedimento ottiene tuttavia lo scopo di limitare i problemi dovuti a eventuali interruzioni, malfunzionamenti e congestioni rilevabili sui cammini.

Si può affermare che il routing *by network address* supporta un alto tasso di fault tolerance e, in negativo, un alto tasso di congestione di rete.

Questa tecnica è in uso presso architetture di rete non orientate alla connessione (CNLS), dato che privilegiare le alternative di cammino significa non fidarsi dell'affidabilità delle infrastrutture di rete. Per questo motivo i protocolli di L3 di reti CNLS non effettuano il controllo degli errori a questo livello, poiché esso risiederebbe all'interno degli IS, cioè nelle stazioni che possiedono le interfacce di rete inaffidabili, in genere possedute dagli enti gestori delle reti. Il controllo dell'errore viene demandato al livello 4 (Trasporto), cioè alle macchine finali (ES), cioè quelle possedute da chi ha instaurato ed è proprietario della comunicazione.

È il modo classico di routing CNLS adottato dalla rete **TCP/IP**.

PROGRAMMA

Routing CNLS e next hop

Date le TdI per tre router R1, R2 e R3, acquisire in input gli indirizzi mittente e destinario di un pacchetto trasmesso dall'host A (direttamente connesso al router R1), quindi mostrare in output il percorso del pacchetto.

(In Appendice la versione per Java)

OUTPUT

```
Digitare indirizzo sorgente (a,b,c): a
Digitare indirizzo destinazione (a,b,c): c
Percorso: a R1 R2 R3 c
```

Per rappresentare i collegamenti dei router R1, R2, R3 e R4 si usa la stessa tecnica del ***PROGRAMMA*** precedente.

Le tre TdI dei router R1, R2 e R3 possono essere rappresentate con una struttura dati che riflette le componenti standard di una TdI: Indirizzo, porta, hop e costo (il costo non sarà tuttavia analizzato).

Il programma, dà per scontato che il router di partenza sia R1 (**riga 34**).

Tra gli host previsti dal programma (**a**, **b**, **c**), si nota che l'unico direttamente connesso a R1 è **a** (infatti gli hops nella riga della TdI di R1 relativa ad **a** vale 0).

Il codice in linguaggio C potrebbe essere il seguente:

```
00 #include <stdio.h>
01 #include <conio.h>
02 #include <string.h>
03 #include <stdlib.h>
04
05 typedef struct {
06   char add;          // indirizzo
07   unsigned char p;   // porta
08   unsigned char h;   // hop
09   unsigned char c;   // costo
10 } TDI;
11 TDI tdiR1[] = {{'a',1,0,0},{'b',2,1,5},{'c',2,2,2}};
12 TDI tdiR2[] = {{'a',3,1,0},{'b',7,0,5},{'c',2,1,2}};
13 TDI tdiR3[] = {{'a',9,2,0},{'b',8,1,5},{'c',9,0,0}};
14 TDI* tdi;
15
16 typedef struct {
17   char ra[3];        // Il router ra è connesso...
18   char rb[3];        // ...al router rb...
19   unsigned char p;   // ...sulla porta p (di ra)
20 } LINK;
21 LINK linkrouter[] = {{"R1","R2",2},{"R2","R1",4},{"R2","R3",2},{"R3","R2",5}};
22
23 int main (void)
24 {
25   char chS,chD;
26   char szHops[100], *pc;
27   int i,j,h,r,p,n = 0;
28
29   printf("Digitare indirizzo sorgente (a,b,c): ");
30   chS = getche();
31   printf("\nDigitare indirizzo destinazione (a,b,c): ");
32   chD = getche();
33
34   h = -1;  r = 1;
35   szHops[0] = 0;
36   pc = szHops;
37   n = n + sprintf(pc+n,"%c ",chS);
38   while (h)
39   {
40     p = -1; h = -1;
41     n = n + sprintf(pc+n,"R%c ",r + '0');
42
43     if (tdi) free(tdi);
44     switch (r)
45     {
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

PROGRAMMA

```
46      case 1: tdi = malloc(sizeof(tdiR1)); memcpy(tdi,tdiR1,sizeof(tdiR1)); break;
47      case 2: tdi = malloc(sizeof(tdiR2)); memcpy(tdi,tdiR2,sizeof(tdiR2)); break;
48      case 3: tdi = malloc(sizeof(tdiR3)); memcpy(tdi,tdiR3,sizeof(tdiR3)); break;
49      }
50
51      for (i=0; i<sizeof(tdi); i++)
52      {
53        if (chD==tdi[i].add)
54        {
55          h = tdi[i].h; p = tdi[i].p;
56          break;
57        }
58      }
59
60      if (h>0)
61      {
62        for (j=0; j<sizeof(linkrouter)/sizeof(LINK); j++)
63        {
64          if ((linkrouter[j].ra[1]==r+'0') && (linkrouter[j].p==p))
65          {
66            r = linkrouter[j].rb[1]-'0';
67            break;
68          }
69        }
70      }
71      else
72      {
73        if (h==0) n = n + sprintf(pc+n,"%c",chD);
74        else
75        {
76          printf("\nImpossibile instradare");
77          return 0;
78        }
79      }
80    }
81
82    printf("\nPercorso: %s",szHops);
83    return 0;
84  }
```

Per ogni salto (detto anche **next hop**), l'algoritmo di inoltro è realizzato tramite:

1. Caricamento della TdI del router (**righe 44÷49**).
2. L'individuazione della riga di inoltro (*regola*) sulla TdI del router (**righe 51÷58**).
3. L'individuazione del router *next hop* (**righe 62÷69**) o controllo dell'arrivo a destinazione (basato sul controllo del valore hops della riga, dato che hops=0 significa destinazione raggiunta, **righe 73÷79**).

Per prevenire loop ed errori si usa la sentinella h = – 1.

3 Routing CONS: label swapping

Tramite il **label swapping** il mittente e il destinatario aprono preventivamente una **connessione** decidendo il cammino attraverso gli IS.

La ricerca del cammino tra mittente e destinatario avviene attraverso gli stessi principi operanti sulle reti CNLS, sfruttando tabelle e algoritmi di instradamento attraverso gli indirizzi univoci di livello 3 assegnati alle stazioni.

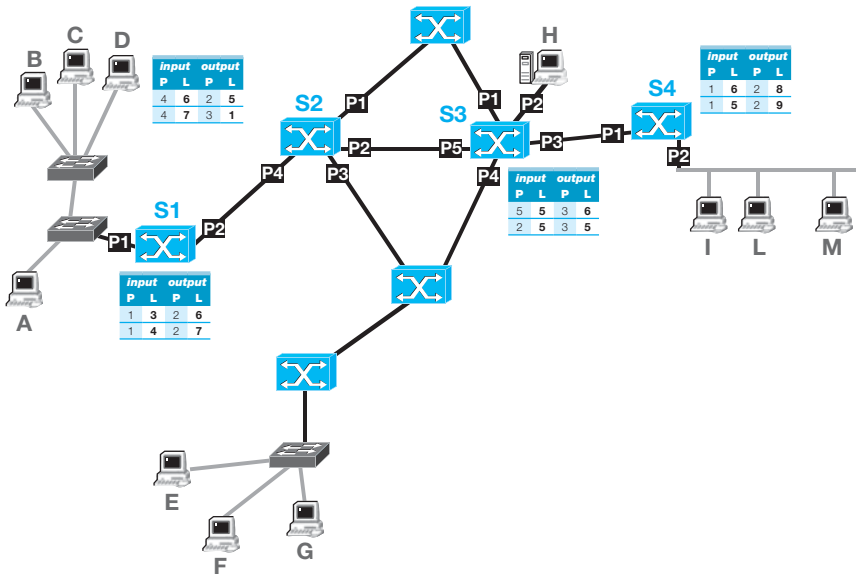
Una volta stabilito il percorso, però, i pacchetti successivi circolano sempre sullo stesso cammino e non contengono più gli indirizzi mittente/destinatario, ma solo un identificativo della connessione, in modo tale che gli

IS (che nelle reti CONS sono anche detti **switch**) li possano instradare nel cammino già calcolato, denominato **circuito virtuale**.

Per ottenere questo effetto è necessario che ogni IS sia a conoscenza del fatto che per determinati pacchetti in entrata, la porta in uscita sia sempre la stessa, quella scelta dall'algoritmo di routing nel momento della connessione. Per questo motivo i protocolli label swapping operanti negli IS agiscono su un campo speciale (**label**) dell'intestazione del pacchetto nel quale inseriscono l'identificativo di riconoscimento del circuito virtuale.

Il procedimento funziona solo se ogni IS, quando riceve i pacchetti, associa la label del pacchetto in entrata da una determinata porta a una nuova label creata localmente e associata alla porta d'uscita stabilita durante la connessione: il pacchetto in entrata subirà, al passaggio su ogni IS, la modifica del campo label (label swapping). Questa associazione viene mantenuta in una speciale tabella dell'IS detta **look-up table**.

L'operazione di label swapping è un'operazione decisamente veloce e che non richiede nessun calcolo se non un accesso indicizzato alla look-up table; addirittura l'operazione può essere eseguita in hardware. La differenza con un accesso a una TdI tradizionale è forte: la coppia (porta ingresso, label sul pacchetto) costituisce già l'indice esatto d'accesso alla look-up table per recuperare la coppia d'uscita (porta d'uscita, nuova label); inoltre la dimensione della tabella è uguale al numero di circuiti virtuali attivi in quel momento su quell'IS, cioè relativamente pochi (mentre una classica TdI deve prevedere, in linea di principio, una riga per ogni possibile destinazione, anche quelle magari non utilizzate da tempo).



Evidentemente questo sistema ha problemi se un circuito virtuale incappa in una tratta difettosa o che cade durante la sessione: il label swapping presume che le connessioni di livello 2 Datalink siano estremamente affidabili e non prevede tolleranza ai guasti di sessione.

Il label swapping è la tecnica standard per le comunicazioni CONS ed è adottata su reti come **ATM** (*Asynchronous Transfer Mode*) e **MPLS** (*Multi Protocol Label Switching*).

ESEMPIO

Routing CONS

Data la rete della figura nella pagina precedente, mostrare il circuito virtuale di un pacchetto «sessione» emesso dall'host A e diretto all'host M.

Osservando la look-up table del label-switch **S1**, si nota che **A** potrebbe aver creato il circuito virtuale di label **L=3** o di label **L=4**.

Entrambi i circuiti virtuali escono sulla porta **P2** di **S1**, ma una volta giunti a **S2**, il secondo circuito virtuale viene smistato sulla porta **P3** di **S2** (come risulta dalla look-up table di **S2**): questo non è il circuito virtuale richiesto dal testo, dato che non raggiungerà (presumibilmente) l'host **M**.

Il primo circuito virtuale, invece, viene inoltrato da **S2** sulla porta **P2** con label **L=5** (look-up table di **S2**).

Giunto a **S3**, viene inoltrato sulla porta **P3** con label **L=6**, quindi giunto su **S4** viene inviato a destinazione (**M**) tramite la porta **P2** e con label **L=8**.

In definitiva la sequenza delle label del circuito virtuale che collega l'host **A** all'host **M** è: **3-6-5-6-8**.

Si nota che la figura descrive anche un circuito virtuale completo dall'host **H** a un host sulla rete LAN di **I**, **L**, **M**.

4 Algoritmi di instradamento

A prescindere dalla modalità di comunicazione, connessa o datagramma (un altro modo per indicare le comunicazioni disconnesse), è possibile elencare e discutere tutta una serie di algoritmi di instradamento che generalmente sono adottati da numerosi IS, sia in modo CONS sia in modo CNLS.

In definitiva si possono catalogare due classi di algoritmi:

- **non adattativi** (o statici), che usano criteri di instradamento costanti, in genere stabiliti manualmente da un responsabile di rete o basati su algoritmi generali;
- **adattativi** (o dinamici), che usano criteri di instradamento calcolati automaticamente in funzione della topologia della rete, dello stato dei collegamenti e del carico. Monitorizzano in tempo reale, o quasi, lo stato della rete, e si adattano ai diversi scenari che possono presentarsi nella topologia.

In molti casi è necessario ricordare come uno stack di rete utilizzi all'interno del proprio livello 3 Network uno o più algoritmi di instradamento, utilizzati in diversi contesti magari in base alle dislocazioni degli IS all'interno della rete oppure come supporto l'uno dell'altro.

4.1 FDR

FDR (*Fixed Directory Routing*), algoritmo non adattativo in cui ogni nodo IS ha una TdI scritta manualmente con regole *Indirizzo/Porta* costituite da indirizzo di destinazione e linea su cui trasmettere. In qualche caso è possibile associare più linee per un destinatario, da utilizzarsi in caso di guasto sulla prima, per gestire un pò di fault tolerance. Le regole delle TdI con questa caratteristica di fissità sono dette anche **route statiche**.

Il gestore della rete (*network administrator* o *sysadmin*) ha il totale controllo dei flussi di carico e può modificare le TdI allorché si sia in presenza di guasti o per ottimizzare il traffico eventualmente congestionato su qualche nodo.

4.2 Flooding

Adottando il **flooding**, algoritmo non adattativo, i pacchetti vengono ritrasmessi dagli IS sempre su tutte le linee possibili, tranne quella da cui sono arrivati. Metodo proposto in campo militare per l'ovvia considerazione che il pacchetto arriva sempre a destinazione (a prescindere dal prezzo pagato dalla rete), si tratta di un algoritmo garantito convergente.

Su configurazioni a maglia genera numerosi **loop** (circoli viziosi), cioè pacchetti che transitano più volte sugli stessi IS causando notevole congestione; inoltre non ottimizza l'uso delle linee; per questo motivo sono spesso introdotti criteri per evitare i loop e i pacchetti duplicati, come particolari campi nei pacchetti L3 di tipo **age counter** che vengono aggiornati a ogni transito sui nodi intermedi per consentire l'eliminazione di pacchetti recidivi.

Per questi motivi in realtà non viene quasi mai utilizzato come algoritmo di instradamento, ma come metodo di supporto per altri algoritmi.

4.3 Routing centralizzato

Adottando il **routing centralizzato**, algoritmo adattativo, uno speciale IS nella rete detto **RCC** (*Routing Control Center*), si incarica di:

- ricevere informazioni sullo stato della rete dai nodi host connessi a ogni altro router;
- usare tali informazioni per calcolare le TdI per ogni router della rete;
- distribuire le TdI da lui calcolate a tutti gli altri router presenti in rete.

Il metodo centralizzato ottimizza le prestazioni ma è poco robusto, dato che riunisce tutti gli aspetti negativi di una gestione centralizzata: in caso di guasti dell'RCC o di qualche linea a lui connessa, l'intera rete subisce un collasso. Inoltre accumula molto carico sull'RCC che, per questo motivo, deve essere una stazione molto potente sia in prestazioni sia in connessioni.

4.4 Routing isolato

Adottando il **routing isolato**, modo adattativo, ogni IS decide la propria TdI autonomamente e senza consultare altri router. Ogni IS determina la propria TdI esclusivamente in base al traffico che lo attraversa. Questa gestione è molto simile alla gestione del routing sui Bridge 802.1D, in questo caso effettuata tramite un algoritmo detto di **backward learning**, integrato anch'esso con uno speciale algoritmo di Spanning Tree. La gestione del routing in questo caso è eminentemente locale e quindi si adatta per un traffico di questo tipo. I pacchetti "stranieri" sono instradati con difficoltà.

Il backward learning si basa su un concetto semplice: *se da una porta P giunge un pacchetto proveniente dall'host A, allora tramite quella porta P si può raggiungere l'host A*. In questo caso la regola (address=A; porta=P) viene automaticamente aggiunta alla TdI. Anche il backward learning viene usato spesso come supporto per altri algoritmi.

In questo caso gli IS risolvono il problema dell'instradamento – e della costruzione delle relative TdI – cooperando tra loro scambiandosi informazioni di servizio periodicamente, utili per la conoscenza reciproca della topologia della rete e quindi dei cammini possibili.

Si tratta della metodologia attuata più frequentemente soprattutto su reti ampie, e integra le caratteristiche positive dei due sistemi precedenti, cioè l'ottimizzazione delle prestazioni (per esempio routing centralizzato) con la risoluzione efficiente dell'instradamento locale (per esempio routing isolato).

Ogni IS calcola la propria TdI «dialogando» con altri IS e con gli ES, utilizzando una serie di algoritmi e procedure di tipo **neighbor greetings**, anch'essi localizzabili a livello L3 Network.

Esistono due approcci tipici per il routing distribuito:

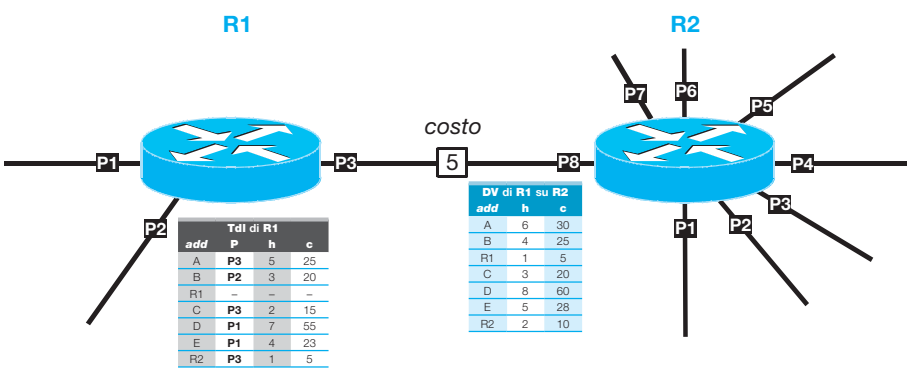
- algoritmi di tipo **distance vector**, relativamente semplici e usati per reti o sottoreti medio-piccole (fino a 1000 nodi);
- algoritmi di tipo **link state packet**, relativamente complessi e usati per reti ampie (migliaia di nodi).

5 Algoritmi distance vector

Algoritmi anche detti di Bellmann-Ford. In questo caso i router adiacenti si scambiano periodicamente dei pacchetti di servizio detti **DV** (*Distance Vector*).

Il DV è un insieme di triple di informazioni di tipo *indirizzo-hops-costo*, derivato dalla TdI e mancante del numero di linea. Le TdI sono costruite in genere con algoritmi di backward learning basati sul traffico locale, mentre l'inoltro di DV serve per aggiornare la topologia dei router dinamicamente: cammini migliori, eliminazione di nodi e relativo autoadattamento, aggiunta di nodi e relativo autoadattamento.

Quando un router modifica la propria TdI in base a qualche modifica della topologia o a qualche ricalcolo ottimizzato, invia solo ai *router adiacenti* un DV.



TdI di R1

add	P	h	c
A	P3	5	25
B	P2	3	20
R1	–	–	–
C	P3	2	15
D	P1	7	55
E	P1	4	23
R2	P3	1	5

DV di R1 su R2

add	h	c
A	6	30
B	4	25
R1	1	5
C	3	20
D	8	60
E	5	28
R2	2	10

Ogni router memorizza per ogni sua porta l'ultimo distance vector ricevuto su quella porta. A questo punto un router ricalcola la sua TdI se:

- cade una linea attiva;
- scopre un nuovo IS;

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

- viene eliminato un IS;
- riceve un DV diverso da quello memorizzato.

Se due DV sulla stessa porta contengono la stessa destinazione, e spesso accade, viene scelta la regola in base al costo e, a ugual costo, in base al minor numero di hops. Pur non riportando la porta, la stessa è ricavabile dal DV semplicemente annotando quella da cui è stato ricevuto.

A questo punto quasi certamente la TdI del router ricevente sarà modificata e quindi sarà costretto a inviare un DV per ogni suo router adiacente.

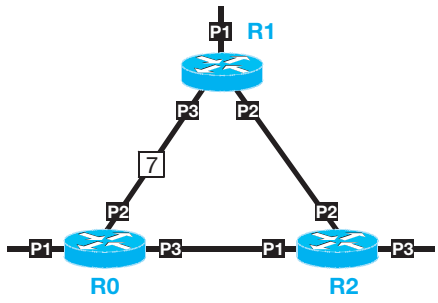
Quando cambiano una o più TdI sui router di una rete, avviene una propagazione di vari DV che ingenera normalmente altre modifiche di TdI e quindi altre propagazioni di DV; ben presto però il traffico di DV diminuirà fino a scomparire, cioè le TdI tenderanno a stabilizzarsi velocemente ottenendo una nuova configurazione di rete. In generale possono presentarsi loop, e in genere l'algoritmo converge in funzione del router più lento (o quello più trafficato o connesso con i link più lenti).

Si tratta di un algoritmo non deterministico e quindi è difficile capirne o predirne il funzionamento. Nessun router contiene la mappa complessiva della rete.

ESEMPIO

Distance vector

Nella figura sono rappresentati tre router di una rete che adottano distance vector.



Le TdI di R0 e R2 valgono:

R0	add	P	h	c
	A	1	5	10
	B	2	3	15
	C	3	6	28
	R1	*2*	*0*	*7*
	R2	*3*	*0*	**?**

R2	add	P	h	c
	A	1	6	17
	B	2	3	12
	C	3	5	21
	R0	*1*	*0*	**?**
	R1	*2*	*0*	**?**

a) Determinare la TdI di R1 e completare le TdI di R0 e R2.

*b) Se giunge a R0 sulla porta P1 il pacchetto di livello 3 Rete: **Z;C;3;15;T-PDU** mostrare come agisce distance vector.*

a) Osservando la terza riga delle due TdI assegnate, si desume che il costo del link R0-R2 vale 28–21=**7**. Infatti per raggiungere C con costo 28 il router R0 attraversa R2 e R2 a sua volta raggiunge C con costo 21.

Osservando la seconda riga delle TdI si osserva che l'host B viene raggiunto da R0 con costo 15 attraversando R1; analogamente per R2, ma con costo 12. L'host B deve essere raggiunto dalla porta P1 di R1 (altrimenti i costi e i salti non tornano: se B fosse raggiungibile da R0 attraverso R2 con 2 hops, B dovrebbe essere direttamente connesso a R2, ma così non è). Quindi il costo del link sulla porta P1 di R1 vale 15–7=**8**.

Ne deriva che il costo del link R2-R1 vale 12–8=**4**, desunto dalla seconda riga della TdI di R2.

Ora è possibile completare le tabelle e ipotizzare quella di R1:

R0	add	P	h	c
	A	1	5	10
	B	2	3	15
	C	3	6	28
	R1	2	0	7
	R2	3	0	**7**

R2	add	P	h	c
	A	1	6	17
	B	2	3	12
	C	3	5	21
	R0	1	0	**7**
	R1	2	0	**4**

R1	add	P	h	c
	A	**3**	**6**	**17**
	B	**1**	**1**	**8**
	C	**2**	**6**	**24**
	R0	**3**	**0**	**7**
	R2	**2**	**0**	**4**

b) Se nel pacchetto di livello 3 Rete **Z;C;3,15;PDU** giunto su P1 di R0 si interpreta Z come indirizzo mittente, C come destinatario e 3 e 15 rispettivamente come hops e costo, allora R0 impara una nuova regola da inserire nella sua TdI: **Z,1,3,15**, inoltra il pacchetto sulla sua porta P3 (regola sulla terza riga della TdI) e avvia un aggiornamento dei DV inviando il **DV=Z,3,15** sia a R2 sia a R1.

Ben presto l'algoritmo convergerà e le TdI di R1 e R2 avranno aggiunto, rispettivamente, le regole **Z,3,4,22** e **Z,1,4,22**.

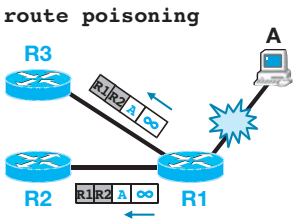
Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

5.1 Loop di routing

Gli algoritmi basati su distance vector hanno uno spiacevole effetto collaterale, i cosiddetti **loop di routing**. Si tratta di particolari condizioni per le quali le TdI non sono aggiornate in modo corretto e quindi continuano a instradare cammini non più validi. I router devono quindi adottare alcuni criteri generali per prevenire queste situazioni.

5.2 Route poisoning

Classica situazione per cui un router R1, una volta stabilito che la tratta su una sua porta è guasta (*link down*), deve segnalare la situazione ai router adiacenti. La segnalazione di R1 avviene inviando un'informazione di routing circa la linea guasta, ma con metrica infinita (**route poisoning**). Ricevuta l'informazione, R2 e R3 inibiranno l'eventuale regola di routing contenuta nelle proprie tabelle.



5.3 Split horizon

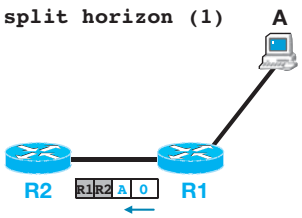
Purtroppo gli aggiornamenti collaborativi delle TdI da parte di router adiacenti hanno un inconveniente: essendo periodici, se avvengono quasi contemporaneamente possono ingenerare loop di routing.

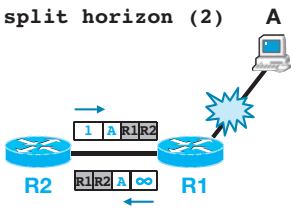
Per esempio, se R1 e R2 sono stabili, R1 ha informato R2 che attraverso di lui si possono raggiungere determinati host. Ora R1 scopre che una sua porta è guasta e proprio quegli host non sono più raggiungibili; invierà l'informazione «poisoned» a R2 con metrica infinita; ma R2, che non ha ancora ricevuto questo messaggio, sta inviando a R1 la regola per cui attraverso di lui si possono raggiungere proprio quegli host.

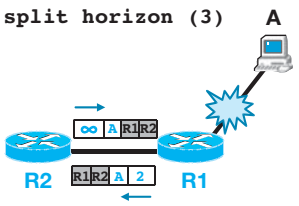
Ora R2 ha ricevuto l'irraggiungibilità di quegli host, ma R1 invece li ritiene raggiungibili tramite R2, benché con un hop in più. La situazione si ripete periodicamente a ogni scambio periodico di informazioni, generando un **count to infinity** di salti nelle tabelle di R1 e R2.

Per evitare il count to infinity si adotta **split horizon**, una regola abbastanza banale: se R2 riceve una regola da R1 che riguarda una sua linea diretta (hop=0), allora R2 non ritrasmette quella regola a R1.

Una variante migliore è **split horizon** con **poisoning reverse**: se ricevuta una regola «poisoned» (indicante una linea guasta), si ritrasmette in flooding l'informazione a tutti i router adiacenti, affinché l'informazione sul guasto si diffonda più velocemente.







5.4 Path holddown

La situazione di count to infinity diventa più grave quando si estende a più di due router interconnessi (per esempio R1 ↔ R2, R2 ↔ R3, R3 ↔ R1). Ora lo split horizon non funziona più: R1 in presenza di guasto invia a R2

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

e R3 l'informazione con metrica infinita; ma contemporaneamente R2 informa R3 che proprio quella linea è ancora attiva (non ha ricevuto ancora l'informazione da R1). Lo split horizon non funziona per R3: il guasto non riguarda una sua linea diretta, quindi invia regolarmente gli aggiornamenti a R2.

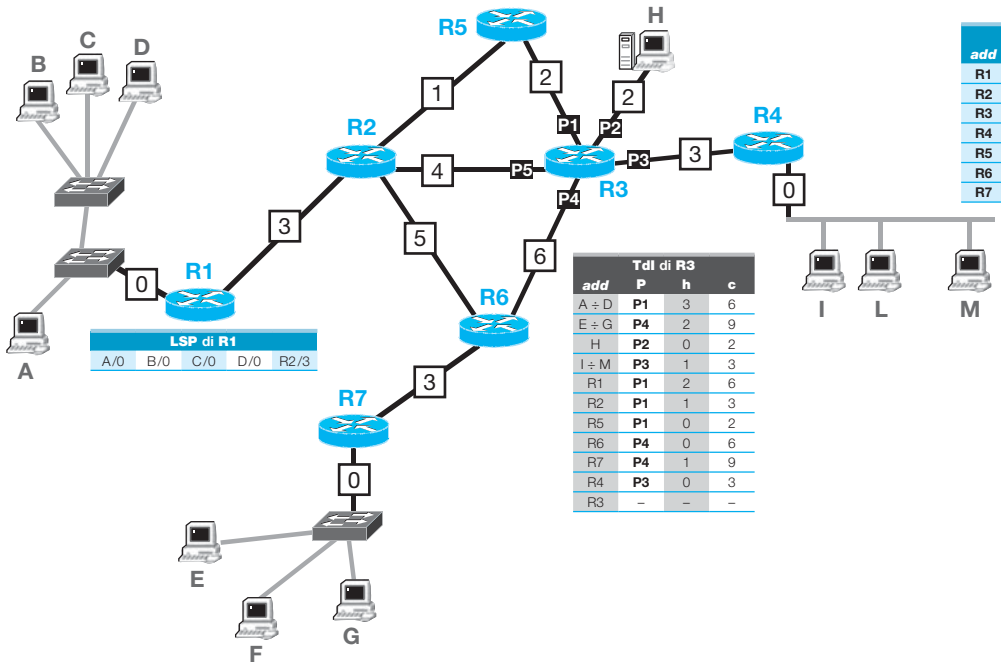
In questi casi si adotta il **path holddown**: una volta ricevuta una regola con metrica infinita (poisoned, cioè linea guasta), bisogna ignorare per un determinato intervallo di tempo altre regole ricevute successivamente circa quella linea.

6 Algoritmi link state packet

Anche in questo caso ogni router «impara» il proprio ambito locale con il backward learning (linee e nodi adiacenti), ma fa in modo che la sua conoscenza si propaghi a tutti i router sulla rete, non solo a quelli adiacenti. Trasmette in flooding queste informazioni tramite un pacchetto di servizio detto **LSP** (*Link State Packet*) in modo che tutti gli altri router, tramite i LSP ricevuti, possano ricostruirsi una mappa globale della rete. In questo modo tutti i router di una rete possiederanno le informazioni dell'intera topologia della rete, a differenza del metodo distance vector.

Quando un router modifica la sua «conoscenza» locale inizia una fase di *update process*. Pertanto genera un LSP che contiene:

- identità di ogni vicino connesso;
- costo del collegamento (*link*) relativo;
- stato del link;
- numero di sequenza;
- tempo di validità;
- age counter.



LSP di R1

A/0	B/0	C/0	D/0	R2/3

Tdl di R3

add	P	h	c
A ÷ D	P1	3	6
E ÷ G	P4	2	9
H	P2	0	2
I ÷ M	P3	1	3
R1	P1	2	6
R2	P1	1	3
R5	P1	0	2
R6	P4	0	6
R7	P4	1	9
R4	P3	0	3
R3	–	–	–

LSPdbase della rete

add	adiacenze				
R1	A/0	B/0	C/0	D/0	R2/3
R2	R1/3	R5/1	R3/4	R6/5	
R3	R2/4	R5/2	H/2	R4/3	R6/6
R4	R3/3	I/0	L/0	M/0	
R5	R2/1	R3/2			
R6	R7/3	R2/5	R3/6		
R7	E/0	F/0	G/0	R6/3	

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Il LSP è quindi trasmesso in flooding ottimizzato, per raggiungere tutti i router della rete.

L'insieme dei LSP che giacciono su un router – e che tende a essere identico su ogni altro router della rete – è detto **LSPdbase** (*LSP database*) e rappresenta la matrice delle adiacenze del grafo pesato (in base al costo) della rete.

Tramite il LSPdbase memorizzato su ogni router, dato un indirizzo mittente (quello del router) e un indirizzo destinazione (quello del pacchetto da inoltrare) è sempre possibile calcolare la porta d'uscita tramite l'**algoritmo di Dijkstra** o **SPF** (*Shortest Path First*). Un router quindi applica SPF ogni qualvolta non possiede la regola per un indirizzo destinazione, creando così la propria TdI.

Il metodo link state packet garantisce la convergenza, a differenza del metodo distance vector, e la raggiunge in tempi inferiori purché il router sia sufficientemente potente in termini di memoria e prestazioni di CPU.

Inoltre esso non ingenera i loop di routing, basando il calcolo dell'instradamento su una TdI realizzata in modo deterministico.

Per contro, l'uso del flooding per la diffusione dei LSP determina un considerevole traffico di servizio, con relativa perdita di banda.

Edsger Wybe Dijkstra

È il nome di uno degli studiosi più influenti della storia dell'informatica (1930-2002). Tra i suoi risultati più noti si ricorda il concetto di **semaforo** (fondamentale per la programmazione concorrente), l'algoritmo **Shortest Path** (ricerca del cammino minimo in un grafo pesato) e il biasimo assoluto verso l'istruzione **go to**.

Detti N il numero dei nodi di un grafo ed E i suoi collegamenti, l'algoritmo Shortest Path ha complessità $O(N^2)$, mentre l'algoritmo di Bellman-Ford una complessità pari a $O(N \cdot E)$.

L'algoritmo per determinare se un numero è pari o dispari ha complessità O(1), mentre individuare un elemento in una lista non ordinata di N elementi ha complessità O(N).

ESEMPIO

Link state packet

Data una rete come in figura, calcolare il LSP del router R2 e la TdI di R2.

Il LSP di R2 è la serie di (add/costo) di tutti gli host adiacenti, perciò:

LSP(R2) = **E/0**, **F/0**, **H/0**, **R1/2**, **R0/4**.

Per calcolare la TdI di R2 serve il LSPdbase della rete, che è la «somma» di tutti i LSP.

Si calcolano quindi gli altri due LSP, relativi a R0 e R1:

LSP(R0) = **A/0**, **B/0**, **C/0**, **R1/1**, **R2/4**;
LSP(R1) = **D/3**, **R0/1**, **R2/2**.

Il LSPdbase risulta:

R0: A/0, B/0, C/0, R1/1, R2/4;
R1: D/3, R0/1, R2/2;
R2: E/0, F/0, H/0, R1/2, R0/4.

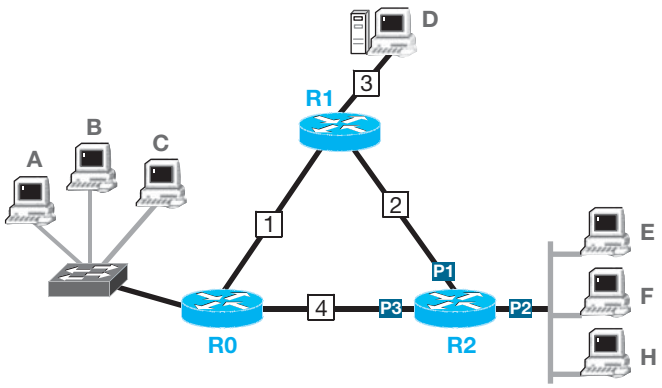
A partire dal LSPdbase, R2 calcola le porte di inoltro di ogni host e router che vi compaiono, calcolando il cammino minimo:

TdI di R2

add	P	h	c
A÷C	1	2	3
D	1	1	5
E÷H	2	0	0
R0	1	1	3
R1	1	0	2

Si nota che potrebbero essere utilizzate alcune regole alternative:

A÷C	3	0	4
D	3	1	8
R0	3	0	4
R1	3	1	5



Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

ESERCIZI PER LA VERIFICA ORALE

Saper rispondere ai **requisiti fondamentali** dà una sufficiente garanzia per sentirsi pronti all'interrogazione. Saper anche rispondere ai **requisiti avanzati** dimostra una padronanza eccellente degli argomenti del capitolo.

Requisiti fondamentali

1. Ricordare e commentare gli elementi base dei protocolli di livello 3 Rete.
2. Spiegare cosa si intende per regola di routing.
3. Commentare il significato di metrica, hop e costo per le regole di routing.
4. Elencare i due approcci base del routing e commentarne le caratteristiche.
5. Chiarire la nozione di next hop.
6. Illustrare la nozione di circuito virtuale.
7. Descrivere i modi di routing FDR e flooding.
8. Elencare i principali metodi di routing illustrandone le caratteristiche base.
9. Introdurre il concetto di algoritmo di routing distribuito.
10. Spiegare la tecnica di routing Backward Learning e mostrarne qualche esempio.
11. Descrivere l'algoritmo Distance Vector.
12. Illustrare la nozione di loop di routing ed elencare le tecniche per prevenirne gli effetti.
13. Descrivere l'algoritmo Link State Packet.

Requisiti avanzati

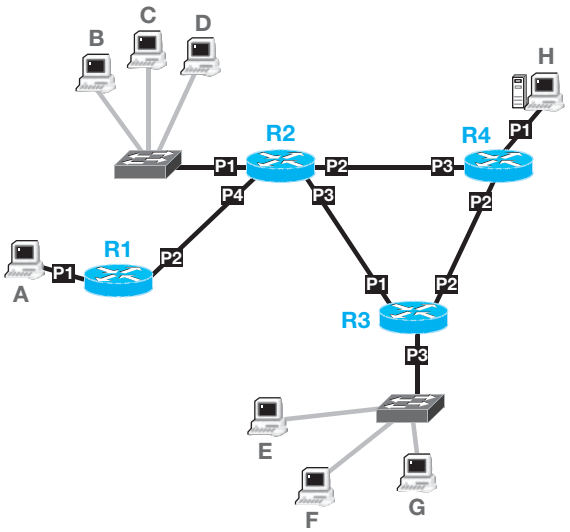
1. Mostrare su un foglio il tragitto «end to end» (da mittente a destinatario) di un pacchetto su WAN in riferimento ai livelli OSI dei nodi attraversati.
2. Elencare i criteri generali adottati e gli obiettivi perseguiti da un protocollo di routing.
3. Osservando la figura del Routing CNLS, spiegare ogni tratta attraversata da un pacchetto che va da C a H.
4. Spiegare i campi di una look-up table.
5. Associare i metodi di routing CNLS e CONS ai principali stack di rete.
6. Riportare la tassonomia degli algoritmi di instradamento e discuterli.
7. Definire la nozione di «route statica» e provare a indicarne un possibile impiego.
8. Ricordare quali modi di instradamento di supporto utilizzano Distance Vector e Link State Packet.
9. Descrivere il comportamento di Distance Vector alla modifica della topologia.
10. Discutere la tecnica di split horizon.
11. Discutere la tecnica di path holddown.
12. Confrontare i modi di distribuzione della struttura dati distance vector e della struttura dati link state packet.
13. Disegnare su un foglio lo schema di una rete magliata con qualche IS e ES, assegnare i costi ai rami e individuare i cammini migliori per qualche coppia di ES.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

ESERCIZI PER LA VERIFICA SCRITTA

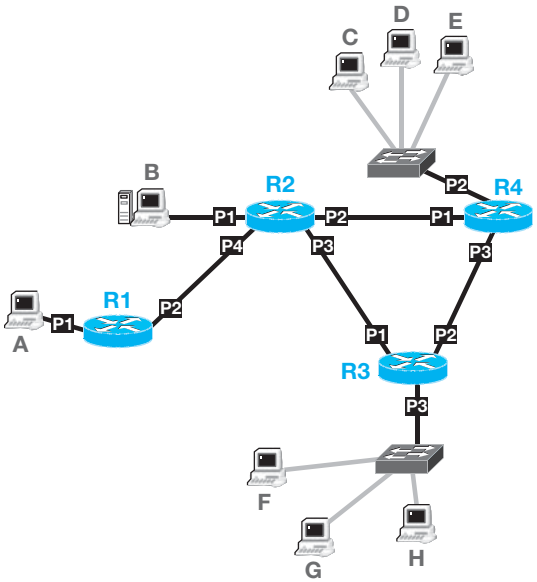
Instradamento

1 Dato il seguente schema di rete, determinare le TdI dei router R1, R2, R3 e R4.



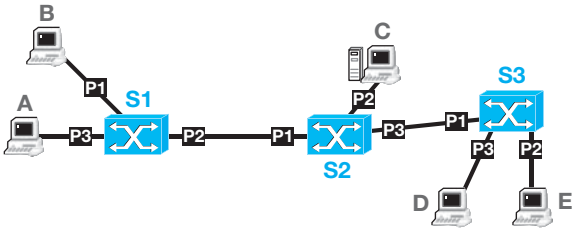
2 Dato lo schema di rete precedente, mostrare, per ogni tratta, l'intestazione di livello 2 e 3 di un pacchetto che parte dall'host A e giunge all'host G.

3 Dato il seguente schema di rete, determinare le TdI dei router R1, R2, R3 e R4.

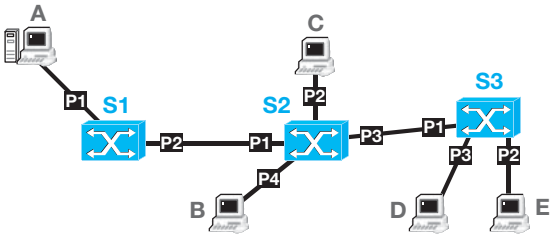


4 Dato lo schema di rete precedente, mostrare, per ogni tratta, l'intestazione di livello 2 e 3 di un pacchetto che parte dall'host E e giunge all'host A.

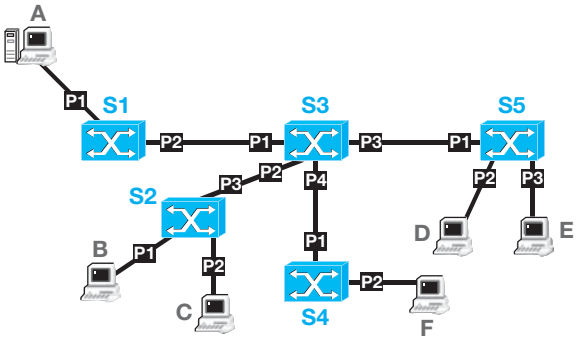
5 Dato il seguente schema di rete, determinare le look-up table degli switch in modo che siano definiti i seguenti circuiti virtuali: A-C, A-E, B-C, B-D, C-E.



6 Dato il seguente schema di rete, determinare le look-up table degli switch in modo che siano definiti i seguenti circuiti virtuali: A-D, C-E, D-E, A-B, B-A.



7 Dato il seguente schema di rete e le look-up table degli switch, indicare quali circuiti virtuali sono descritti.



Switch S2

input		output	
P	L	P	L
1	1	3	2
2	1	3	3
1	2	3	1

Switch S3

input		output	
P	L	P	L
2	3	3	4
2	1	3	3
2	2	3	2

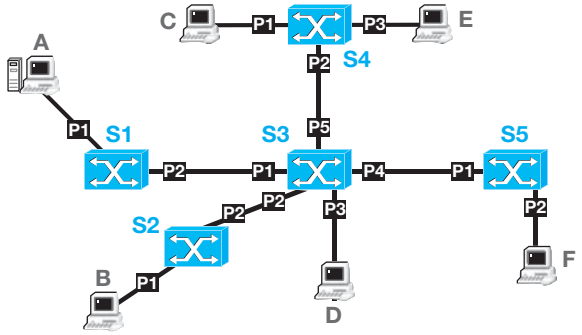
Switch S5

input		output	
P	L	P	L
1	2	3	4
1	4	2	1
1	3	2	4

8 Dato lo schema di rete precedente, aggiornare le look-up table per i seguenti collegamenti: C-E, F-A, D-E, B-F, A-F.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

9 Dato il seguente schema di rete e le look-up table degli switch, indicare quali circuiti virtuali sono descritti.



Switch S1

input		output	
P	L	P	L
1	**1**	2	**1**
2	**2**	1	**2**
2	**3**	1	**3**

Switch S3

input		output	
P	L	P	L
1	**1**	5	**1**
5	**2**	1	**2**
3	**3**	1	**3**

Switch S4

input		output	
P	L	P	L
2	**1**	1	**1**
3	**2**	2	**2**

10 Dato lo schema di rete precedente, aggiornare le look-up table per i seguenti collegamenti: A-D, E-D, A-F, B-D, C-E.

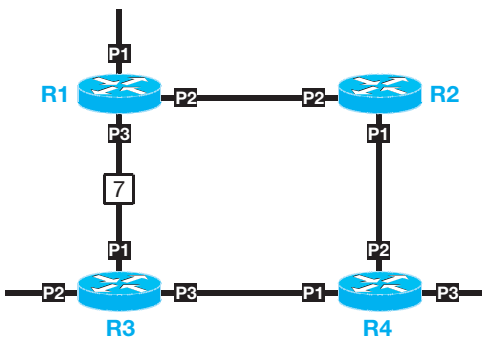
11 Dato lo schema di rete dell'Esercizio 1, determinare le TdI dei router dopo aver instradato il seguente traffico in backward learning (le TdI sono inizialmente vuote):

A → B, A → H, B → A, H → D, G → A, G → D, A → G.

12 Dato lo schema di rete dell'Esercizio 3, determinare le TdI dei router dopo aver instradato il seguente traffico in backward learning (le TdI sono inizialmente vuote):

A → B, A → H, B → A, H → D, G → A, G → D, A → G.

13 Dato lo schema di rete seguente e le TdI relative, determinare i costi mancanti delle linee.



TdI R1

Add	P	h	c
A	3	1	7
B	1	0	0
C	1	0	0

TdI R2

Add	P	h	c
A	1	2	7
B	2	1	6

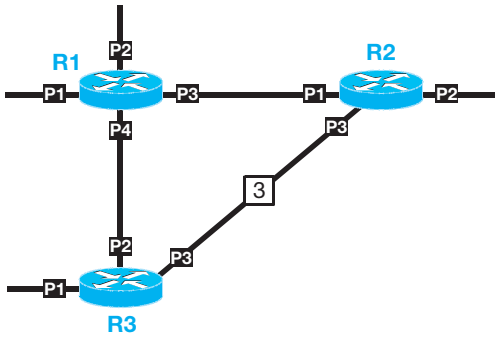
TdI R4

Add	P	h	c
B	1	2	10
C	2	2	10

14 Dato lo schema di rete dell'esempio precedente, riportare le TdI aggiornate in base al traffico (Mitt, Dest, h, c):

a) su P2 di R3 giunge (X,A,0,0);
b) su P3 di R4 giunge (Y,B,1,9);
c) su P1 di R1 giunge (Z,X,2,3).

15 Dato lo schema di rete seguente e le TdI relative, determinare i costi mancanti delle linee.



TdI R1

Add	P	h	c
B	4	1	6
C	2	0	0

TdI R2

Add	P	h	c
A	3	1	3
B	1	2	10

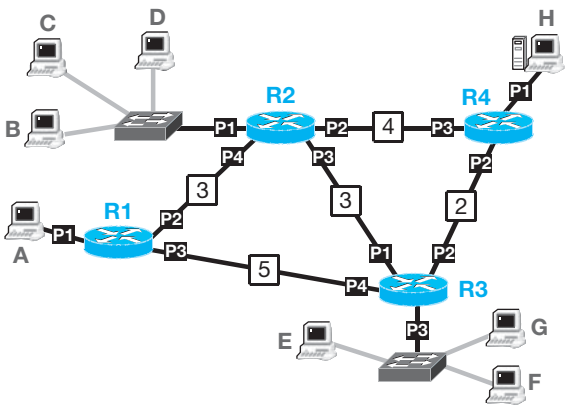
TdI R3

Add	P	h	c
A	1	0	0
B	1	0	0

16 Dato lo schema di rete dell'esempio precedente, riportare le TdI aggiornate in base al traffico (Mitt, Dest, h, c):

a) su P2 di R2 giunge (X,A,0,0);
b su P2 di R1 giunge (Y,B,1,9);
c) su P1 di R3 giunge (Z,C,2,3).

17 Dato il seguente schema di rete (le linee senza costo hanno costo 0), determinare il LSP di ogni router, le TdI di ogni router e il LSPdbase della rete.



Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

ESERCIZI PER LA VERIFICA DI LABORATORIO

Per compilare gli esercizi proposti è stato usato l'ambiente gratuito multipiattaforma Windows/Linux ***Code::Blocks 10.5*** con **gcc 4.4.1** e ***Eclipse SDK 4.2.0*** con **Jdk 7** per **Java 1.7**.

I testi degli esercizi presentano uno o più **layout** di input/output richiesti; in questo modo sono indicate, a volte, alcune specifiche del programma come l'output, il controllo dell'input o i valori ammissibili.

Instradamento

1 Scrivere un programma che implementa l'algoritmo FDR con route statiche scritte in un file di testo `(Add;P;h;c)`.

Layout:

OUTPUT

```
FDR su router con 3 interfacce. TdI:
Add P h c
A   1 3 9
B   2 2 4
C   3 2 10
D   1 1 5
Pacchetto da instradare (Mitt,Dest): Z A
Su porta 1

Pacchetto da instradare (Mitt,Dest): Z X
Non instradabile

Pacchetto da instradare (Mitt,Dest): 0
```

2 Scrivere un programma che implementa l'algoritmo FDR con route statiche scritte in un file di testo `(Add;P;h;c)` e `flooding`.

Layout:

OUTPUT

```
FDR su router con 3 interfacce. TdI:
Add P h c
A   1 3 9
B   2 2 4
C   3 2 10
D   1 1 5
Pacchetto da instradare (P,Mitt,Dest):2 Z A
Su porta 1

Pacchetto da instradare (P,Mitt,Dest):1 Z X
Su porte 2,3 (flooding)

Pacchetto da instradare (P,Mitt,Dest): 0
```

3 Scrivere un programma che implementa l'algoritmo backward learning su un router a tre porte.

Layout:

OUTPUT

```
TdI vuota.
Traffico 1) (P,Mitt,Dest,h,c): 1 A C 2 10
Flooding
Traffico 2) (P,Mitt,Dest,h,c): 2 B C 1 3
Flooding
Traffico 3) (P,Mitt,Dest,h,c): 2 B A 1 3
Su P1
Traffico 4) (P,Mitt,Dest,h,c): 3 C E 3 7
Flooding
Traffico 5) (P,Mitt,Dest,h,c): 1 A C 2 10
Su P3
Traffico 6) (P,Mitt,Dest,h,c): 0
```

4 Scrivere un programma che, date le look-up table di 4 switch, instradi i pacchetti forniti dall'utente nella forma (`Mitt`, `Label`).

Layout:

OUTPUT

```
Inserire Mittente e Label: A 3
Circuito virtuale: 3-6-5-6-8

Inserire Mittente e Label: H 5
Circuito virtuale: 5-5-9

Inserire Mittente e Label: 0
```

*Nota: ispirarsi al programma **Routing CNLS e next hop** presente nel testo. Il layout del programma agisce sullo schema di rete connessa riportata nel disegno del paragrafo **Routing CONS: label swapping**.*

5 Scrivere un programma che implementa l'algoritmo FDR con route statiche scritte in un file di testo `(Add;P;h;c)` e `flooding` ma con il formato di indirizzi IP.

Layout:

OUTPUT

```
FDR su router con 3 interfacce. TdI:
Add           P h c
192.168.0.1   1 3 9
192.168.1.1   2 2 4
192.168.2.1   3 2 10
192.168.3.1   1 1 5
Pacchetto da instradare (P,Mitt,Dest):
2 172.16.13.1 192.168.0.1; su porta 1

Pacchetto da instradare (P,Mitt,Dest):
1 172.16.13.1 192.168.5.1; flooding

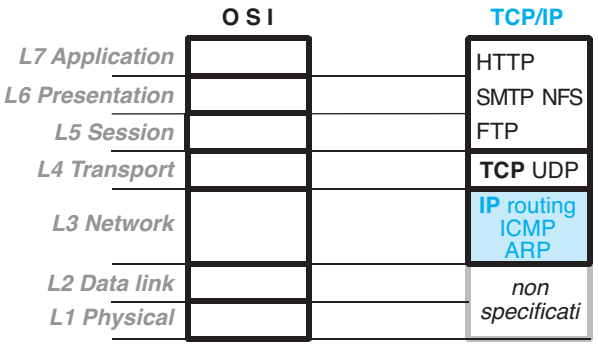
Pacchetto da instradare (P,Mitt,Dest):
0
```

B2 Protocollo IP

Il protocollo **IP** (*Internet Protocol*), meglio noto come **IPv4** (*IP version 4*), è il protocollo di livello 3 Network più famoso e utilizzato al mondo.

La prima formulazione ufficiale è stata redatta nel 1981 con l'RFC 791.

Assieme a TCP (*Transmission Control Protocol*) costituisce l'ossatura della rete comunemente nota come *TCP/IP* o *Internet Protocol Suite*.



La figura mostra l'architettura dell'Internet Protocol Suite e la paragona con il modello di riferimento ISO/OSI, riportando i protocolli più famosi operanti nei livelli della suite TCP/IP.

Siccome IP è un protocollo **disconnesso**, esso determina la natura complessivamente disconnessa dell'intera rete TCP/IP.

La parte **connessa** di TCP/IP, comunque sempre necessaria in una rete, è offerta dal protocollo di trasporto TCP, che è un protocollo connesso.

In molti casi e soprattutto quando i livelli inferiori sono ritenuti affidabili, TCP/IP può usare come livello di trasporto UDP (*User Datagram Protocol*) al posto di TCP, offrendo in questo caso un esempio di rete quasi totalmente disconnessa dato che UDP è un protocollo disconnesso.

I livelli 5 Sessione e 6 Presentazione non esistono in TCP/IP e sono demandati ai protocolli operanti al livello 7 Applicazione, pertanto non esiste uno standard univoco per gestire le problematiche di livello Sessione e di livello Presentazione.

L'architettura di rete TCP/IP non specifica i livelli 1 e 2 della rete, ma utilizza quelli normalmente disponibili sugli host e conformi agli standard.

Per esempio, nell'ambito di rete locale Ethernet, IP opera sul protocollo LLC o direttamente sulle trame Ethernet; oppure, nell'ambito delle reti geografiche, IP opera sul protocollo PPP, ma anche integrato con tecnologie

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

connesse come ATM e MPLS (cfr.). Esistono anche realizzazioni per reti fisiche molto strane, spesso diffuse solo all'interno di certe comunità o utilizzate in ambito industriale.

1 Protocollo ARP

Il protocollo **ARP** (*Address Resolution Protocol*, RFC 826) serve per ottenere l'indirizzo di livello 2 (per esempio un MAC address) di una stazione di cui si conosce solo l'indirizzo di livello 3 (per esempio l'indirizzo IP).

Quando un host A (mittente) su una LAN si ritrova a dover spedire una informazione a un host B (destinazione) di cui conosce *solo* l'indirizzo IP, affinché i pacchetti che contengono l'informazione possano raggiungere l'host B sulla LAN, quei pacchetti devono contenere l'indirizzo MAC destinatario dell'host B.

Per recuperare tale indirizzo, l'host A deve servirsi di ARP: invia un pacchetto **ARP Request** in broadcast sulla rete, contenente l'indirizzo IP di B e attende che B, se presente nel dominio di broadcast e regolarmente acceso, risponda con un pacchetto **ARP Reply** di tipo unicast, contenente il suo indirizzo MAC.

Solo ora l'host A può inviare l'informazione all'host B, incapsulandola in una trama Ethernet corredata dall'indirizzo MAC destinazione di B.

Il protocollo ARP deve essere implementato *su tutti gli host di una rete*, sia la parte client (*ARP Request*), sia la parte server (*ARP Reply*).

Il protocollo ARP memorizza tramite il sistema operativo una tabella che contiene le associazioni (*Indirizzo IP*, *MAC address*) in modo da evitare di inviare un ARP Request se l'indirizzo IP dell'host ricercato è già stato «risolto» e il suo indirizzo MAC si trova nella tabella. Questa tabella è detta **ARP cache** ed è consultabile e gestibile con il programma omonimo **arp** (cfr. ***Sistemi e reti, vol. 1, Applicazioni, arp***). Le voci nell'ARP cache vengono periodicamente eliminate e reinserite, per evitare che un'informazione troppo vecchia fornisca associazioni errate.

Come si è cercato di rappresentare nella figura precedente, il protocollo ARP non è rigorosamente un protocollo di livello 3 Network (come indica la letteratura), piuttosto un protocollo di livello "2,5", notazione che indica una diretta dipendenza dal livello 2 Data link su cui ARP si ritrova a operare.

Proxy ARP

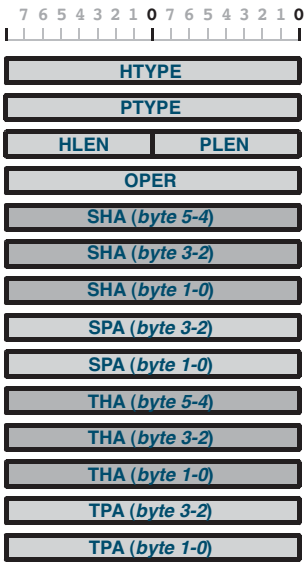
Siccome ARP agisce in broadcast, la sua azione non si estende tra le subnet di una rete privata: gli host di una sottorete diversa da quella del mittente *ARP Request* non risponderanno, dato che i router bloccano il traffico MAC broadcast.

In questi casi si può attivare sui router di una rete privata il protocollo **proxy ARP** (RFC 1027): ora la richiesta ARP verrà inoltrata dal router alla sottorete opportuna, e l'host «distante» risponderà al router con il proprio indirizzo MAC. Il router con proxy ARP quindi renderà la risposta al richiedente originale.

1.1 Pacchetto ARP

Il pacchetto ARP ha dimensione fissa 28 ottetti. Il significato dei campi è il seguente (tra parentesi la dimensione in bit):

- **HTYPE** (16, *Hardware Type*): codifica del tipo di livello fisico (Ethernet=1).
- **PTYPE** (16, *Protocol Type*): codifica del protocollo richiedente. Come Ethertype (per esempio IPv4=0800h).



Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

- **HLEN** (8, *Hardware Length*): dimensione in byte dell'indirizzo fisico richiesto.
- **PLEN** (8, *Protocol Length*): dimensione in byte dell'indirizzo «richiedente».
- **OPER** (16, *Operation*): codifica per ARP Request (1) e Reply (2).
- **SHA** (16, *Sender Hardware Address*): se *Request*, indirizzo MAC del richiedente; se Reply, indirizzo MAC **richiesto**.
- **SPA** (16, *Sender Protocol Address*): se *Request*, indirizzo IP del richiedente; se *Reply*, indirizzo IP richiesto.
- **THA** (16, *Target Hardware Address*): se *Request*, non usato (tutti a 0); se *Reply*, indirizzo MAC del richiedente.
- **TPA** (16, *Target Protocol Address*): se *Request*, indirizzo IP «**di cui scoprire l'equivalente fisico**»; se *Reply*, indirizzo IP del richiedente.

ESEMPIO

Frame ARP

Data la seguente sequenza esadecimale di byte che rappresenta un pacchetto ARP, decodificarne l'intestazione.

```
00 01 08 00 06 04 00 02 00 13 46 0b 22 ba c0 a8 00 01 00 16 ce
6e 8b 24 c0 a8 00 72
```

HTYPE (byte 0-1 = 0001h): codifica 1, cioè hardware di tipo Ethernet.
PTYPE (byte 2-3 = 0800h): richiedente con codifica Ethertype, cioè IP.
HLEN (byte 4 = 06h): lunghezza indirizzo richiesto 6 byte (MAC Address).
PLEN (byte 5 = 04h): lunghezza indirizzo del richiedente 4 byte (IP address).
OPER (byte 6-7 = 00h 02h): pacchetto ARP di tipo **ARP Reply**.
SHA (byte 8-13 = 00h 13h 46h 0bh 22h bah): **MAC richiesto**.
SPA (byte 14-17 = c0h a8h 00h 01h): IP host richiesto, 192.168.0.1.
THA (byte 18-23 = 00h 16h ceh 6eh 8bh 24h): MAC del richiedente.
TPA (byte 24-27 = c0h a8h 00h 72h): IP del richiedente (192.168.0.114).

Quando un host A vuole l'indirizzo MAC di un host B di cui conosce solo l'indirizzo IP, prima di tutto consulta la sua ARP cache per verificare se esiste l'indirizzo IP di B. Infatti se esiste l'indirizzo IP di B in ARP cache, allora si può prelevare immediatamente il suo indirizzo MAC e non è necessario fare una richiesta in rete.

Se invece l'indirizzo IP di B non compare in ARP cache, allora va fatta una richiesta ARP in rete. Si prepara quindi un pacchetto ARP (*ARP Request*) con i seguenti campi impostati:

HTYPE = 1, hardware di tipo Ethernet.
PTYPE = 0800h, richiedente di tipo IP (Ethertype = 0800h).
HLEN = 6, dimensione indirizzo richiesto, i 6 byte di un indirizzo MAC.
PLEN = 4, dimensione indirizzo del richiedente, i 4 byte di un indirizzo IP.
OPER = 1, pacchetto ARP di tipo *ARP Request*.
SHA = **MAC(A)**, l'indirizzo MAC del richiedente.
SPA = **IP(A)**, l'indirizzo IP del richiedente.
THA = 00h 00h 00h 00h 00h 00h (tutti a zero, non usato in *ARP Request*).
TPA = **IP(B)**, l'indirizzo IP dell'host di cui si vuole l'indirizzo MAC.

RARP

Reverse Address Resolution Protocol, RFC 903, è il nome del protocollo «inverso» ad ARP: indicato un indirizzo di livello 2 (per esempio il MAC address) di un host, restituisce l'indirizzo di livello 3 (per esempio l'indirizzo IP) di quell'host.

Come per ARP, la *RARP Request* è inviata in broadcast, ma a differenza di ARP la parte server del protocollo (cioè chi risponde con *RARP Reply*) è realizzata solo su alcune macchine, in modo da centralizzare la distribuzione degli indirizzi IP. Il formato del pacchetto è identico, anche se l'uso dei campi è ovviamente diverso. OPER vale 3 per *RARP Request* e 4 per *RARP Reply*.

RARP ha un problema: i pacchetti broadcast vengono 'fermati' dai router, perciò il servizio di distribuzione degli indirizzi IP non è ottimizzato.

Per questo motivo RARP è reso obsoleto da protocolli di livello superiore come **BOOTP** e **DHCP** (cfr. ***Sistemi e reti, vol. 3, DHCP***).

Quindi il pacchetto viene incapsulato in un frame **Ethernet broadcast** con *Ethertype* = 0806h (la codifica di ARP) e trasmesso su Ethernet.

Il pacchetto *ARP Reply* corrispondente sarà di tipo unicast, e conterrà, nel campo SHA, il MAC dell'host B ricercato (**MAC(B)**).

Infine A, ricevendo risposta, inserirà l'associazione (**IP(B)**, **MAC(B)**) nell'ARP cache.



PROGRAMMA

Leggere un frame da file di testo

Dato un file di testo (per esempio frameeth.txt) contenente una serie di coppie di caratteri esadecimali rappresentanti altrettanti byte, scrivere un programma in linguaggio C per ricavare l'equivalente array di byte in memoria.

(In Appendice la versione per Java)

OUTPUT

```
Array frame:
00 0d 2b 11 f1 01 00 50 fc 23 4b 9c 00 81 f0 f0 da 3a 0e 00 ff ef 16 00 00 00 00
00 00 00 c6 10 a1 31 00 50 fc 23 4b 9c 05 dc aa aa 03 00 00 00 08 00 45 00 00 3d
5e d1 00 ff fa 0e 65 21 00 11 43 51 fd 99 08 00 45 00 00 3d 5e d1 00 00 80 11 b8
16 c2 02
```

Per analizzare convenientemente i pacchetti circolanti su una rete è utile rappresentare la loro sequenza di byte in esadecimale, magari in un file di testo.

Quindi è molto comodo poter disporre di quella sequenza in un vettore di byte in modo da poter accedere agli elementi con un indice.

Un esempio di programma in linguaggio C potrebbe essere:

```
00 #include <stdio.h>
01 #include <string.h>
02 #include <stdlib.h>
03
04 unsigned char* getBytesFromTextFile(char* szFileName, int* n);
05
06 int main (void)
07 {
08   unsigned char* pb;
09   int i,n;
10
11   pb = getBytesFromTextFile("frameeth.txt",&n);
12   printf("Array frame:\n");
13   for (i=0; i<n; i++)
14   {
15     printf("%02x ",pb[i]);
16   }
17   return 0;
18 }
19
20 unsigned char* getBytesFromTextFile(char* szFileName, int* n)
21 {
22 #define MAXCHARxRIGA (1000)
23 #define MAXRIGHE     (1000)
24
25   FILE *fp;
26   char buffer[MAXCHARxRIGA]; char szBuffer[MAXRIGHE*MAXCHARxRIGA];
27   unsigned char* pbyte; char ch;
28   unsigned char byt=0;
29   int l=-1, blnAncora=1, sts=0, i=0, j=0;
30
31   *n = -1; szBuffer[0]=0;
32   fp = fopen (szFileName, "r");
33   while (blnAncora)
34   {
35     blnAncora=1;
36     if (feof(fp) || fgets(buffer, MAXCHARxRIGA, fp)==NULL) blnAncora=0;
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

PROGRAMMA

```
37      else
38      {
39        buffer[strlen(buffer)-1] = '\0';
40        strcat(szBuffer,buffer);
41      }
42    }
43    fclose(fp); //printf("FILE:\n%s\n\bytes letti\n",szBuffer);
44
45    l = strlen(szBuffer);
46    while (i<l)
47    {
48        ch = szBuffer[i];
49        if (ch>='A' && ch<='Z') ch=ch+32;
50        switch (sts)
51        {
52          case 0:
53            if ((ch>='0' && ch<='9') || (ch>='a' && ch<='f'))
54            {
55              if (ch >= '0' && ch <= '9') byt = 16*( ch - '0');
56              else byt = 16*(ch - 'a' + 10);
57              sts = 1;
58            }
59          break;
60
61          case 1:
62            if ((ch>='0' && ch<='9') || (ch>='a' && ch<='f'))
63            {
64              if (ch >= '0' && ch <= '9') byt = byt + ch - '0';
65              else byt = byt + ch - 'a' + 10;
66              if (j==0) pbyte = malloc(1); else pbyte=realloc(pbyte,j+1);
67              *(pbyte+j) = byt;
68              j++;
69              sts = 0;
70            }
71          break;
72        }
73        i++;
74    }
75    *n = j;
76    return pbyte;
77  }
```

La funzione **getBytesFromTextFile()** ha il compito di leggere la sequenza Ascii delle coppie di caratteri esadecimali su un file di testo e restituire il vettore che poi viene stampato nel main().

La funzione accetta il nome del file fisico su disco e restituisce il vettore sottoforma di puntatore e la sua dimensione.

La sequenza su disco può essere scritta su una o più righe, tipicamente con uno spazio che separa le coppie di caratteri esadecimali.

I caratteri esadecimali possono essere scritti indifferentemente in minuscolo o maiuscolo (**riga 49**). Il breve algoritmo di conversione (**righe 48÷73**) usa un semplice automa a stati finiti con due stati codificati sulla variabile **sts**.

2 Pacchetto IP

Il compito fondamentale di IP è quindi di instradare i messaggi sulla rete tramite tabelle di instradamento e algoritmi di routing, adottando un preciso schema di indirizzamento per gli indirizzi di livello 3.

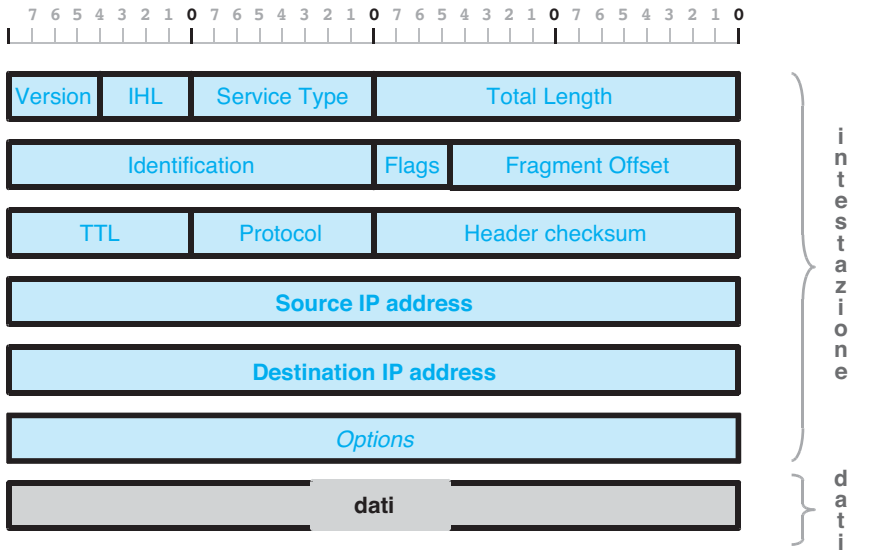
Inoltre ha anche funzioni di frammentazione e riassemblaggio dei messaggi, e di rilevazione (ma *non* correzione) degli errori.

Un singolo pacchetto IP, detto anche **datagramma IP**, contiene tipicamente una PDU di livello superiore (per esempio TCP), che può essere ampia fino a 64 kB. Anche se IP è in grado di gestire pacchetti di tale di-

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

mensione, di fatto quasi sempre li frammenta in pacchettini più piccoli, di solito 1500 byte.

Il formato del pacchetto IP è mostrato in figura:



Sia i campi, sia i byte nei campi, sia i bit all'interno dei byte sono inseriti in modo big endian: il primo byte a essere spedito (e ricevuto) è *Version/IHL*, all'interno del quale, per esempio, il valore 45h significa *Version=4* e *IHL=5*.

L'ampiezza minima dell'header IP vale **20** byte, ma ha una lunghezza variabile a causa del campo facoltativo *Options*. In ogni caso non supera i **60** byte. L'header IP è seguito dal campo **dati** (o payload) che contiene la PDU del protocollo di livello superiore (per esempio TCP).

Il significato dei campi dell'header del pacchetto IP è il seguente (tra parentesi la dimensione in bit):

- **Version** (4): è il numero di versione del protocollo IP che ha generato il pacchetto. Per IPv4 vale 4.
- **IHL** (4, *Internet Header Length*): è la lunghezza dell'header IP, variabile in funzione del campo **Options**, espressa come numero di parole da 32 bit (4 byte). Il minimo *IHL* vale 5 (5 · 4=20 byte se il campo *Options* è lungo 0 byte), il massimo vale 15 (15 · 4=60 byte, se il campo *Options* è lungo 40 byte). Serve per «raggiungere» la parte *dati* e calcolarne la dimensione assieme al campo *Total Length*.
- **Service Type** (8): campo a bit che specifica come un protocollo di livello superiore vuole che il pacchetto sia trattato; dopo varie ridefinizioni oggi i trattamenti sono codificati in RFC 3168, una codifica delle quali riguarda lo streaming in tempo reale della voce VoIP (Voice over IP).
- **Total Length** (16): è la lunghezza del pacchetto IP (header + dati) espresso in byte; il pacchetto IP più ampio occupa quindi 2^{16}=65536 byte (64 kB), che può definirsi l'MTU di IP. Assieme a *IHL* serve per calcolare la dimensione dalla parte *dati* del datagramma IP (=Total Length – IHL · 4).
- **Identification** (16): contiene un numero intero che identifica il pacchetto frammentato da IP; è usato per permettere il riassemblaggio di

RFC 3514: il bit del male

Network Working Group
S. Bellovin
Request for Comments: 3514
AT&T Labs Research
Category: Informational
1 April 2003

The Security Flag in the IPv4 Header

Abstract

Firewalls, packet filters, intrusion detection systems, and the like often have difficulty distinguishing between packets that have malicious intent and those that are merely unusual. We define a security flag in the IPv4 header as a means of distinguishing the two cases.

1. Introduction

Firewalls, packet filters, intrusion detection systems, and the like often have difficulty distinguishing between packets that have malicious intent and those that are merely unusual. The problem is that making such determinations is hard. To solve this problem, we define a security flag, known as the **"evil" bit**, in the IPv4 [RFC791] header.

Benign packets have **this bit set to 0**; those that are used for an attack will have **the bit set to 1**. [...]

Osservare la data di questa RFC.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

un pacchetto frammentato, dato che tutti i frammenti di uno stesso pacchetto originale hanno questo campo impostato con lo stesso valore.

- **Flags** (3): campo di bit che specifica se un pacchetto può essere frammentato o meno (bit1, DF, *don't fragment*) e se si tratta dell'ultimo frammento di un pacchetto o del primo e/o unico frammento (bit2, MF, *more fragment*). Siccome il destinatario non sa il numero totale dei frammenti di un pacchetto e i frammenti possono anche arrivare fuori ordine, il ricevente riconosce l'ultimo frammento solo quando analizza questo bit.

  Se un datagramma IP che presenta DF=1 è più ampio dell'MTU della rete fisica su cui deve essere inoltrato (per esempio da un router), il pacchetto viene scartato e il router notifica al mittente l'errore tramite il protocollo ICMP (*Internet Control Message Protocol*).
- **Fragment Offset** (13): è il numero di sequenza di un frammento (in multipli di 8 byte). Dato che il campo occupa 13 bit, può numerare al massimo 2^{13}=8192 pacchettini (da 0 a 8191) da 8 byte l'uno, che è la dimensione minima di un pacchetto frammentato (in questo particolare caso i pacchettini devono essere numerati al massimo fino a 8188).

  Chi deve riassemblare il pacchetto originale fa scattare un timer alla ricezione del primo frammento: se il timer scade prima di aver ricevuto tutti i frammenti va segnalato al mittente con un messaggio ICMP.

  Si deve tenere presente che se il frammento di un pacchetto originale si presenta su un router che deve inoltrarlo su una rete il cui MTU è inferiore all'ampiezza della dimensione del frammento, il frammento viene a sua volta frammentato, e così via.
- **TTL** (4, *Time to live*): è un campo che viene decrementato a ogni passaggio (hop) del pacchetto su un router; quando il contatore arriva a zero il pacchetto viene scartato e il router notifica lo scarto al mittente con il protocollo ICMP. È uno dei pochi campi dell'header che viene modificato durante il cammino e ciò causa anche il ricalcolo del campo *Header Checksum*.
- **Protocol** (8): codifica che identifica il protocollo di routing o di livello superiore contenuto nel campo *dati* del pacchetto. La codifica valida a livello mondiale è gestita dallo IANA (*Internet Assigned Numbers Authority*).
- **Header checksum** (16): è un campo utilizzato per controllare che l'header IP sia corretto. Si nota che la somma di controllo riguarda solo l'intestazione del pacchetto, dato che IP demanda il controllo del pacchetto allo strato superiore (tipicamente TCP). Il calcolo è effettuato con il modo *Internet checksum* (cfr. ***Sistemi e reti, vol. 1, Controllo dell'errore***). È uno dei pochi campi dell'header che può essere modificato durante il cammino, per esempio a causa di un decremento di TTL.
- **Source address** (32) e **Destination address** (32): sono gli **indirizzi IP** di mittente e destinatario, entrambi su 4 byte (32 bit);
- **Options**: è un campo facoltativo (presente solo se *HIL*>5), usato per fornire varie informazioni, tipo sicurezza e routing.

La lista dei protocolli più importanti trasportati da un datagramma IP nel campo *Protocol*:

IP checksum

L'*Header checksum* di un pacchetto IPv4 comprende le prime 10 parole da 16 bit dell'header IP, escludendo la parola riservata all'*Header checksum*: in tutto 9 word (a 16 bit).
Un modo efficace di effettuare il calcolo consiste nell'eseguire la somma semplice delle 9 word, quindi se il risultato eccede i 16 bit considerare la parte troncata: a questa si somma il valore eccedente (carry).
Infine si inverte il valore ottenuto con un NOT.
Il ricevente calcola agevolmente l'integrità dell'header IP eseguendo il calcolo di checksum su *tutte* le 10 parole dell'header IP (compreso l'*Header checksum*): il risultato del calcolo dà 0 se l'header è integro; risulta diverso da 0 in caso di errore.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Valore	Sigla	Protocollo
01h	ICMP	*Internet Control Message Protocol, RFC 792*
02h	IGMP	*Internet Group Management Protocol, RFC 1112*
06h	TCP	*Transmission Control Protocol, RFC 793*
08h	EGP	*Exterior Gateway Protocol, RFC 888*
09h	IGP	*Interior Gateway Protocol (variabile in base a quello adottato)*
11h	UDP	*User Datagram Protocol, RFC 768*
58h	EIGRP	*Enhanced Interior Gateway Routing Protocol (Cisco)*
59h	OSPF	*Open Shortest Path First, RFC 1583*
7Ch	IS-IS	*Intermediate System To Intermediate System, RFC 1142*
89h	MPLS	*Multiprotocol Label Switching, RFC 4023*

ESEMPIO

Header IP

Data la seguente sequenza esadecimale di byte che rappresenta un pacchetto IP, decodificarne l'intestazione.

45 00 00 28 0f 47 40 00 80 06 91 ed 91 fe a0 ed 41 d0 e4 df 0d 2c 00 50 38 af ff f3 11 4c 6c 54 50 10 25 bc 6c bd 00 00

Versione (byte 0 = 45h): 4, Ipv4.
HIL(byte 0 = 45h): 5, no campo Options, lunghezza header=5 · 4=20 byte.
Total length (byte 2-3 = 0028h): lunghezza del pacchetto = 40 byte (28h).
Identification (byte 4-5 = 0f47h): 0f47h.
Flags (byte 6 = 40h): 40h = **010**00000b, cioè Flags=02h, DF=1.
Il pacchetto non deve essere frammentato.
Fragment offset (byte 6-7 = 4000h): 4000h = 010**0000000000000b**, cioè Offset = 0.
TTL (byte 8 = 80h): conteggio impostato a 128 (80h).
Protocol (byte 9 = 06h): in base alla codifica IANA, 06h = TCP.
Il datagramma IP trasporta un pacchetto TCP.
Header checksum (byte 10-11 = 91edh): l'IP Internet checksum vale 37357 (91edh).
Source IP address (byte 12-15 =91fea0edh): 145.254.160.237 (91h.feh.a0h.edh).
Destination IP address (byte 16-19 = 41d0e4dfh): 65.208.228.223 (41h.d0h.e4h.dfh).
Dati (byte 20-39): PDU di TCP.

PROGRAMMA

Header IP

Data una sequenza di byte rappresentante un pacchetto IP, decodificare il contenuto del suo header con un programma in linguaggio C.

OUTPUT

```
Ip frame:
45 00 00 28 0f 47 40 00 80 06 91 ed 91 fe a0 ed 41 d0 e4 df 0d 2c 00 50 38 af ff
f3 11 4c 6c 54 50 10 25 bc 6c bd 00 00
Version: 4
HIL: 5
Total Length: 40
Identification: 3911
Flags: 2; MF=0; DF=1
Fragment offset: 0
TTL: 128
Protocol: 06h
Header checksum: 37357
Source Ip: 145.254.160.237
Destination Ip: 65.208.228.223
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

PROGRAMMA

La sequenza di byte del pacchetto IP è memorizzata in un file di testo (ip.txt), pertanto viene usata la funzione *getBytesFromTextFile()* implementata nel ***PROGRAMMA*** precedente (e qui inclusa).

Un esempio di programma in linguaggio C potrebbe essere:

```
00 #include <stdio.h>
01 #include <string.h>
02 #include <stdlib.h>
03 #include "getBytesFromTextFile.c"
04
05 int main (void)
06 {
07   unsigned char* pb;
08   int i,n;
09   unsigned char byte;
10   unsigned int word;
11
12   pb = getBytesFromTextFile("ip.txt",&n);
13   printf("Ip frame:\n");
14   for (i=0; i<n; i++)
15   {
16     printf("%02x ",pb[i]);
17   }
18
19   byte = pb[0] >> 4;
20   printf("\nVersion: %d",byte);
21
22   byte = pb[0] & 0x0f;
23   printf("\nHIL: %d",byte);
24
25   word = pb[2]*256 + pb[3];
26   printf("\nTotal Length: %d",word);
27
28   word = pb[4]*256 + pb[5];
29   printf("\nIdentification: %d",word);
30
31   byte = pb[6] >> 5;
32   printf("\nFlags: %d; MF=%d; DF=%d",byte, byte>>2, byte>>1);
33
34   word = pb[6]*256 + pb[7];
35   printf("\nFragment offset: %d",word & 0x1FFF);
36
37   byte = pb[8];
38   printf("\nTTL: %d",byte);
39
40   byte = pb[9];
41   printf("\nProtocol: %02xh",byte);
42
43   word = pb[10]*256 + pb[11];
44   printf("\nHeader checksum: %d",word);
45
46   printf("\nSource Ip: %d.%d.%d.%d",pb[12],pb[13],pb[14],pb[15]);
47   printf("\nDestination Ip: %d.%d.%d.%d",pb[16],pb[17],pb[18],pb[19]);
48
49   return 0;
50 }
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

ESEMPIO

Frammentazione IP

Un router riceve un datagramma IP ampio 5000 byte (Total Length = 5000) e lo deve inoltrare su una rete Ethernet con MTU pari a 1500 byte. Elencare i frammenti creati con i valori dei campi significativi per la frammentazione.
Si ipotizza IHL=5 (pacchetto senza Options, cioè con header ampio 20 byte).

La dimensione massima del pacchetto IP circolante sulla rete Ethernet vale 1500, di cui 1500 – 20 = **1480** utilizzabili per il campo *dati* (payload).

Siccome 1480 è un multiplo di 8, la dimensione del payload del frammento è accettabile: 1480/8 = **185** (altrimenti si sarebbe dovuto scegliere un'ampiezza inferiore multipla di 8).

I 5000 byte originali devono essere ripartiti in frammenti ampi 1480 byte, per un totale di 5000/1480=3,37, quindi **3** frammenti da 1480 byte più il frammento finale ampio 5000 % 1480 = **560** byte (il resto della divisione precedente).

Il router quindi assegna, su base locale, il valore **23** al campo *Identification*, un valore ancora non utilizzato per i propri pacchetti.

In definitiva i quattro frammenti, con i campi significativi, risultano:

Frammento	Total Lenght	Identification	DF	MF	Fragment Offset
0	20 + 1480	23	0	1	0
1	20 + 1480	23	0	1	185
2	20 + 1480	23	0	1	370
3	20 + 560	23	0	0	555

3 Protocollo ICMP

Il protocollo **ICMP** (*Internet Control Message Protocol*, RFC 792) è dichiarato di livello 3 Network di TCP/IP, anche se è incapsulato all'interno di datagrammi IP, a differenza di ARP ma come TCP, con un valore di IP *Protocol* = 1 (vedi tabella precedente).

IP usa ICMP in varie occasioni, spesso su iniziativa dei router, che segnalano agli host alcune situazioni anomale o condizioni d'errore durante l'espletamento delle proprie funzioni. Infatti IP è un protocollo disconnesso, ovvero senza controllo dell'errore e senza consegna garantita dei pacchetti (protocollo di tipo *best effort*, ovvero del tipo "faccio il meglio che posso").

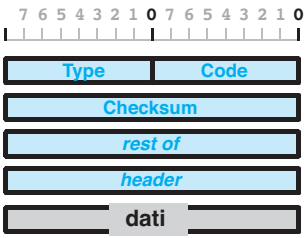
ICMP aiuta IP ad affrontare le situazioni d'errore (che non è progettato per risolvere) attraverso delle segnalazioni, tra cui si ricordano quelle direttamente utilizzate per indicare le condizioni anomale già analizzate descrivendo il protocollo IP:

- Quando un pacchetto IP non può essere frammentato ma deve «passare» attraverso una rete con MTU inferiore alla sua ampiezza, il pacchetto viene scartato e il router invia al mittente un messaggio ICMP *Destination Unreachable-Fragmentation Needed And DF Set*.
- Quando il timer per riassemblaggio scade senza aver ricevuto tutti i frammenti, il pacchetto originale viene scartato e il router invia al mittente un messaggio ICMP *Time Exceeded-Fragment reassembly time exceeded*.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

- Quando un router, abbassando il TTL di un pacchetto IP in transito, arriva a zero lo scarta, ma notifica al mittente un messaggio ICMP *Time Exceeded-Time To Live Exceeded in Transit*.

Inoltre ICMP segnala problemi abbastanza tipici, come per esempio quando un router non riesce a individuare un cammino per una rete o un host di destinazione: il mittente riceve un messaggio ICMP *Destination Unreachable-Destination network unreachable* o *Destination host unreachable*. I messaggi ICMP spesso non vengono esibiti dai programmi applicativi che usano IP, come per esempio i browser per la navigazione web: le applicazioni, infatti, mascherano questi messaggi d'errore con proprie versioni del messaggio o ponendo rimedio alla segnalazione con soluzioni alternative.



Le segnalazioni ICMP avvengono tramite l'invio di un pacchetto abbastanza semplice (tra parentesi la dimensione in bit):

- **Type** (8): codifica ICMP del tipo di messaggio da consegnare.
- **Code** (8): sottotipo per il tipo di messaggio; precisa l'indicazione.
- **Checksum** (16): somma di controllo del pacchetto.
- **rest of header** (32): parte dell'header disponibile per altre informazioni.
- **dati**: payload di ICMP; tipicamente contiene tutto l'header IP del pacchetto che è coinvolto nella segnalazione, più otto byte del campo dati di quel pacchetto. Il campo è di ampiezza variabile perché l'intestazione IP è variabile.

Le codifiche più importanti per *Type/Code* sono riportate in tabella:

Type	Descrizione	Code	Messaggio
0	Echo Reply	0	*Echo reply (usato dal programma **ping**)*
3	Destination Unreachable	0	*Destination network unreachable*
		1	*Destination host unreachable*
		2	*Destination protocol unreachable*
		3	*Destination port unreachable*
		4	*Fragmentation required, and DF flag set*
		5	*Source route failed*
		6	*Destination network unknown*
		7	*Destination host unknown*
		8	*Source host isolated*
		9	*Network administratively prohibited*
		10	*Host administratively prohibited*
		11	*Network unreachable for TOS*
		12	*Host unreachable for TOS*
		13	*Communication administratively prohibited*
		14	*Host Precedence Violation*
		15	*Precedence cutoff in effect*
4	Source Quench	0	*Source quench (congestion control)*
5	Redirect Message	0	*Redirect Datagram for the Network*
		1	*Redirect Datagram for the Host*
		2	*Redirect Datagram for the TOS & network*
		3	*Redirect Datagram for the TOS & host*
6	Alternate Host Address	0	
8	Echo Request	0	*Echo request (usato dal programma **ping**)*
9	Router Advertisement	0	*Router Advertisement*

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

10	Router Solicitation	0	*Router discovery/selection/solicitation*
11	Time Exceeded	0	*TTL expired in transit*
		1	*Fragment reassembly time exceeded*
12	Parameter Problem: Bad IP header	0	*Pointer indicates the error*
		1	*Missing a required option*
		2	*Bad length*
13	Timestamp	0	*Timestamp*
14	Timestamp Reply	0	*Timestamp reply*
15	Information Request	0	*Information Request*
16	Information Reply	0	*Information Reply*
17	Address Mask Request	0	*Address Mask Request*
18	Address Mask Reply	0	*Address Mask Reply*
30	Traceroute	0	*Information Request*

ESEMPIO

Datagramma ICMP

Data la sequenza di byte esadecimali seguente, decodificare il datagramma IP contenuto:

```
45 00 00 3c 8e 9b 00 00 80 01 2a 62 c0 a8
00 72 c0 a8 00 01 08 00 49 5c 03 00 01 00
61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e
6f 70 71 72 73 74 75 76 77 61 62 63 64 65
66 67 68 69
```

Il primo byte (byte 0 = 45h) indica la coppia *Version/HIL* dell'header di IP, cioè 45h significa 4=IPv4 e IHL=**5**, cioè nessuna *Options* e quindi header IP lungo 5 · 4= **20** byte.

Si nota che il campo *Protocol* vale **1** (byte 9 = 1), che la codifica IANA riserva a **ICMP**.

Il 21simo byte (byte 20 = 8) è perciò la parte dati del pacchetto IP, ovvero il primo byte del pacchetto ICMP.

Esso coincide con il campo *Type*=**8**, cioè si tratta di un ICMP *Echo Request con Code* = **0**.

Si nota che il mittente IP ha indirizzo 192.168.0.114 e il destinatario IP ha indirizzo 192.168.0.1, rispettivamente i byte 12÷15 e 16÷19.

3.1 ICMP routing

Oltre a segnalare condizioni d'errore ICMP consente di realizzare un sottile strato di routing per gli host che, normalmente, realizzano il livello 3 Network in modo abbastanza elementare rispetto allo strato di Rete implementato sui router.

Per esempio, un router può ricevere un pacchetto IP da un host e, consultando la propria TdI, può rendersi conto che l'host non ha deciso correttamente l'instradamento: un altro router è più indicato per ricevere quel pacchetto.

In questo caso il router notifica all'host «distratto» il nuovo router che deve considerare come ottimale con un messaggio ICMP *Redirect Message* contenente l'indirizzo IP del router su cui effettuare la redirezione nel campo *rest of header*. In questo modo la rudimentale TdI dell'host può aggiornarsi con una nuova regola temporanea.

Inoltre, tra gli ultimi messaggi a essere stati introdotti nel protocollo ICMP, si trovano *Address Mask Request* e *Address Mask Reply*, per permettere a un host di richiedere e quindi impostare automaticamente uno dei parametri base per il routing degli host, ovvero la *subnet mask* usata nella rete di appartenenza (cfr. ***Subnetting***).

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

PROGRAMMA

Pacchetto ICMP

Data una sequenza di byte rappresentante un pacchetto IP, scrivere un programma in linguaggio C che verifica se si tratta di un pacchetto ICMP.

(In Appendice la versione per Java)

OUTPUT

```
Ip frame:
45 00 00 3c 8e 9b 00 00 80 01 2a 62 c0 a8 00 72 c0 a8 00 01 08 00 49 5c 03 00 01 00 61
62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74 75 76 77 61 62 63 64 65 66 67 68
HIL: 5, perciò i dati IP iniziano dall'indice: 20
IP Protocol: 01h
       Pacchetto ICMP (01)
       Type: 08h Code: 00h (Echo Request)
```

Per verificare se il datagramma IP trasporta ICMP bisogna verificare se il campo *Protocol* dell'header IP contiene il valore **1** (vedi Tabella pag. 111 per il campo *Protocol* di IP).

Quindi bisogna «saltare» l'header IP calcolandone la dimensione (*HIL*4*) e decodificare i primi due byte del campo dati di IP, che coincidono con la coppia *Type/Code* di ICMP.

Un esempio di programma in linguaggio C potrebbe essere:

```
00 #include <stdio.h>
01 #include <string.h>
02 #include <stdlib.h>
03 #include "getBytesFromTextFile.c"
04
05 #define IPPROTOCOL_ICMP (0x01)
06 #define ICMP_TYPE_ECHOREQUEST (8)
07 #define ICMP_CODE_ECHOREQUEST (0)
08
09 int main (void)
10 {
11   unsigned char* pb;
12   int i,n;
13   unsigned char byte;
14   unsigned int idxIPData;
15
16   pb = getBytesFromTextFile("icmp.txt",&n);
17   printf("Ip frame:\n");
18   for (i=0; i<n; i++)
19   {
20     printf("%02x ",pb[i]);
21   }
22
23   byte = pb[0] & 0x0f;
24   idxIPData = byte*4;
25   printf("\nHIL: %d, perciò i dati IP iniziano dall'indice: %d",byte, idxIPData);
26
27   byte = pb[9];
28   printf("\nIP Protocol: %02xh",byte);
29   if (byte==IPPROTOCOL_ICMP)
30   {
31     unsigned char Type, Code;
32
33     printf("\n\tPacchetto ICMP (%02x)",byte);
34     Type = pb[idxIPData]; Code = pb[idxIPData+1];
35     printf("\n\tType: %02xh Code: %02xh",Type,Code);
36     if (Type == ICMP_TYPE_ECHOREQUEST && Code==ICMP_CODE_ECHOREQUEST)
37     {
38       printf(" (Echo Request)");
39     }
40   }
41   else printf("\n\tNon è un pkt ICMP (%02xh)",byte);
42
43   return 0;
44 }
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

4 Programma ping

Uno degli usi più proficui del protocollo ICMP viene fatto dal programma universalmente noto come **ping**, disponibile su piattaforme Windows, Linux e anche su sistemi operativi dedicati come IOS di Cisco Inc. (cfr. ***Sistemi e reti, vol. 1, Applicazioni, ping***).

Fornendo l'indirizzo IP di un host al programma *ping*, esso opera spedendo messaggi ICMP *Echo Request* al destinatario e aspettando i relativi messaggi di risposta ICMP *Echo Reply* che ogni host su una rete è costretto a inviare (come prescritto dalla RFC 1122).

Durante il processo, *ping* misura l'**RTT** (*Round Trip Time*) dei pacchetti ICMP e registra eventuali pacchetti persi. Al termine mostra l'output dei risultati fornendo un'indicazione relativamente veloce sullo stato di raggiungibilità dell'host. Per la sua semplicità ed estrema utilità il programma *ping* è una utility amministrativa di rete indispensabile per verificare se un determinato host è acceso e/o raggiungibile.

RTT

L'**RTT**, detto anche **RTD**, è il tempo impiegato da un pacchetto su una rete per viaggiare da un mittente a un destinatario e tornare al mittente.

Il termine si usa in svariati contesti, come per esempio per il programma *ping*.

Il nome «ping» è stato deciso dall'autore del programma Mike Muss (1958-2000) nel 1983, in analogia con il suono emesso dai sonar. In seguito **ping** fu definito come acronimo per *Packet InterNet Groper*.

ESEMPIO

Ping e MTU

Determinare l'MTU della rete Ethernet usando il comando ping, sapendo che su sistemi operativi di tipo Windows ping genera datagrammi IP di dimensioni fisse pari a 32 byte, mentre per sistemi operativi di tipo Linux pari a 64 byte.

Affrontiamo il problema su un sistema operativo di tipo Windows (per Linux la soluzione è identica, benché con una variante nell'uso di *ping*).

Siccome un pacchetto IP generato da *ping* per Windows (IP incapsula ICMP in datagrammi IP) è piccolo (32 byte) e sicuramente inferiore all'MTU di Ethernet (infatti i messaggi *ping* normalmente non generano errori), si impone a *ping* di generare un pacchetto di dimensione desiderata, per esempio 2000 byte.

Consultando l'help del comando (**ping /?**), l'opzione da specificare è **–l 2000**.

Inoltre si può imporre a *ping* di impostare i suoi pacchetti IP come non frammentabili, cioè con **DF=1**. L'opzione da specificare, in questo caso, è **–f**.

In questo modo un pacchetto che eccede l'MTU della rete e non è frammentabile genera un errore:

```
C:\Windows\system32>ping 192.168.0.1 -l 2000 -f

Esecuzione di Ping 192.168.0.1 con 2000 byte di dati:
È necessario frammentare il pacchetto ma DF è attivo.
È necessario frammentare il pacchetto ma DF è attivo.
È necessario frammentare il pacchetto ma DF è attivo.
È necessario frammentare il pacchetto ma DF è attivo.

Statistiche Ping per 192.168.0.1:
   Pacchetti: Trasmessi = 4, Ricevuti = 0,
   Persi = 4 (100% persi),

C:\Windows\system32>
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

ESEMPIO

Provando varie dimensioni inferiori con il comando proposto, si giunge a determinare che la prima dimensione accettata (non dà errori usando *ping*) vale **1472**:

```
C:\Windows\system32>ping 192.168.0.1 -l 1472 -f

Esecuzione di Ping 192.168.0.1 con 1472 byte di dati:
Risposta da 192.168.0.1: byte=1472 durata=1ms TTL=128
Risposta da 192.168.0.1: byte=1472 durata=1ms TTL=128
Risposta da 192.168.0.1: byte=1472 durata=1ms TTL=128
Risposta da 192.168.0.1: byte=1472 durata=1ms TTL=128

Statistiche Ping per 192.168.0.1:
   Pacchetti: Trasmessi = 4, Ricevuti = 4,
   Persi = 0 (0% persi),
Tempo approssimativo percorsi andata/ritorno in millisecondi:
   Minimo = 1ms, Massimo =  1ms, Medio =  1ms

C:\Windows\system32>
```

In effetti si deve considerare che *ping* interpreta il parametro **–l** come dimensione del pacchetto IP esclusa l'intestazione IP (20 byte) e il pacchetto ICMP incapsulato (8 byte).

Quindi l'MTU di Ethernet vale 1472 + 20 + 8 = **1500** byte.

Il comando *ping* corrispondente per Linux è: **ping –M do –s 2000 –c 4 192.168.0.1**.

```
linux:~# ping -M do -s 2000 -c 4 192.168.0.1

PING 192.168.0.1 (192.168.0.1) 2000(2028) bytes of data.
From 192.168.0.34 icmp_seq=1 Frag needed and DF set (mtu = 1500)
From 192.168.0.34 icmp_seq=1 Frag needed and DF set (mtu = 1500)
From 192.168.0.34 icmp_seq=1 Frag needed and DF set (mtu = 1500)
From 192.168.0.34 icmp_seq=1 Frag needed and DF set (mtu = 1500)
--- 192.168.0.1 ping statistics ---
0 packets transmitted, 0 received, +4 errors

linux:~# ping -M do -s 1472 -c 4 192.168.0.1
PING 192.168.0.1 (192.168.0.1) 1472(1500) bytes of data.
1480 bytes from 192.168.0.1: icmp_req=1 ttl=64 time=1.56 ms
1480 bytes from 192.168.0.1: icmp_req=2 ttl=64 time=1.43 ms
1480 bytes from 192.168.0.1: icmp_req=3 ttl=64 time=1.38 ms
1480 bytes from 192.168.0.1: icmp_req=4 ttl=64 time=1.40 ms

--- 192.168.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.388/1.446/1.565/0.075 ms
linux:~#
```

In Appendice si può trovare una lista di comandi *ping* di utilità generale.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

4.1 Programma traceroute

Una seconda proficua utilizzazione del protocollo ICMP riguarda il programma **traceroute** (*tracert* per i sistemi Windows, cfr. ***Sistemi e reti, vol. 1, Applicazioni, traceroute***).

Come nel caso di *ping*, traceroute/tracert rappresenta una utility amministrativa molto diffusa, offrendo risposta quasi immediata alla richiesta dell'elenco dei router attraversati da un pacchetto per raggiungere una destinazione data.

Il principio di funzionamento di *traceroute* è di una semplicità imbarazzante: se si invia un pacchetto IP qualsiasi a un host remoto (quello di cui si vuol tracciare la strada) con il campo *TTL*=1, il primo router sulla strada sarà costretto a rispondere al mittente con un pacchetto d'errore ICMP *Time Exceeded*.

Avrà, infatti, abbassato il *TTL* del pacchetto e, resosi conto di aver raggiunto 0, dovrà scartarlo e notificare lo scarto al mittente con il messaggio ICMP *Time Exceeded*.

Il messaggio d'errore giungerà al mittente che ora può annotarsi l'indirizzo IP del primo router attraversato.

Ora basta proseguire il procedimento aumentando il *TTL* di una unità a ogni tentativo, fino alla ricezione di un pacchetto ICMP *Echo Reply* (se come pacchetto di prova si era usato un ICMP *Echo Request*). Alla ricezione dell'ICMP *Echo Reply*, *traceroute* sa che la lista dei router attraversati è completata e può stampare il resoconto.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

ESERCIZI PER LA VERIFICA ORALE

Saper rispondere ai **requisiti fondamentali** dà una sufficiente garanzia per sentirsi pronti all'interrogazione. Saper anche rispondere ai **requisiti avanzati** dimostra una padronanza eccellente degli argomenti del capitolo.

Requisiti fondamentali

1. Rappresentare la pila dello stack TCP/IP mettendo in evidenza il protocollo IP in relazione con la pila OSI.
2. Discutere la natura disconnessa di TCP/IP.
3. Descrivere la funzione del protocollo ARP.
4. Spiegare per quale ragione un pacchetto ARP request ha sempre destinazione un indirizzo di broadcast.
5. Discutere la dimensione classica dei datagrammi IP, la massima e la minima.
6. Commentare i seguenti campi dell'header IP: TTL, Protocol, Source e Destination Address.
7. Spiegare come si fa a capire se l'header di IP di un datagramma vale 20 byte.
8. Illustrare le ragioni della frammentazione operata dal protocollo IP.
9. Descrivere la funzione del protocollo ICMP.
10. Ricordare i più importanti messaggi ICMP.
11. Descrivere il programma ping e mostrarne esempi d'uso.
12. Descrivere il programma traceroute e mostrarne esempi d'uso.
13. Illustrare il concetto di «best effort» applicato ad un protocollo.

Requisiti avanzati

1. Spiegare come e dove vengono realizzati i livelli OSI mancanti nello stack TCP/IP.
2. Descrivere la funzione e l'utilità di ARP cache.
3. Spiegare per quale motivo il protocollo ARP deve essere distribuito a tutti i nodi in modalità sia client che server.
4. Commentare i seguenti campi dell'header IP: IHL, Total Length, Identification e Flags.
5. Mostrare come si può desumere la dimensione della parte dati (payload) di un pacchetto IP.
6. Descrivere quale effetto collaterale provoca la presenza del campo Options nell'header IP.
7. Commentare la modalità adottata da IP per la frammentazione dei pacchetti.
8. Spiegare per quale motivo il protocollo ICMP non è implementato allo stesso modo su tutti i nodi di una rete.
9. Commentare il tipo di messaggio ICMP denominato Redirect Message.
10. Discutere il funzionamento del programma ping tramite messaggi ICMP.
11. Mostrare il comando ping per verificare se un host è connesso alla rete pubblica (Internet).
12. Illustrare il funzionamento del programma traceroute attraverso i messaggi ICMP.
13. Elencare le fasi di ping utili per determinare l'MTU di un link.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

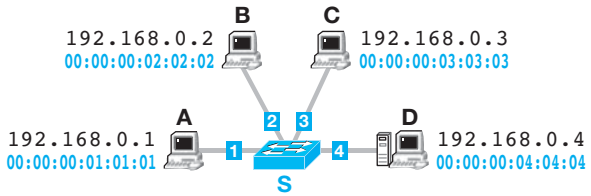
ESERCIZI PER LA VERIFICA SCRITTA

ARP, ICMP, IP

1 Sia dato il seguente schema di rete, con indicati gli indirizzi di livello 3 (IP address) e livello 2 (MAC address). Tutti i sistemi sono stati appena avviati, quindi le ARP cache sono vuote e anche la CAM dello switch. L'host **A** esegue il comando:

```
ping 192.168.0.4
```

a) Elencare tutti i frame generati dal comando fino al suo termine (completando la tabella).
b) Elencare le ARP cache di ogni host al termine del comando (completando le tabelle).
c) Riportare il filtering database dello switch al termine del comando (completando la tabella).



t	protocollo	Addr L2	Addr L3
1	ARP Request	Mitt: 00:00:00:01:01:01 Dest: ff:ff:ff:ff:ff:ff	---
2			
3			
4	ICMP Echo Reply	Mitt: 00:00:00:04:04:04 Dest: 00:00:00:01:01:01	192.168.0.4 192.168.0.1

Arp cache **A**

IP	MAC
192.168.0.4	00:00:00:04:04:04

Arp cache **B**

IP	MAC

Arp cache **C**

IP	MAC

Arp cache **D**

IP	MAC

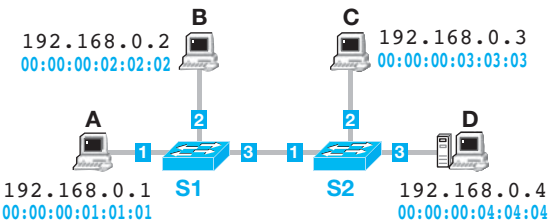
Filtering database **S**

Dest MAC Address	Ports
00:00:00:04:04:04	1

2 Sia dato il seguente schema di rete, con indicati gli indirizzi di livello 3 (IP address) e livello 2 (MAC address). Tutti i sistemi sono stati appena avviati, quindi le ARP cache sono vuote così come le CAM degli switch. L'host **A** esegue il comando:

```
ping 192.168.0.2
```

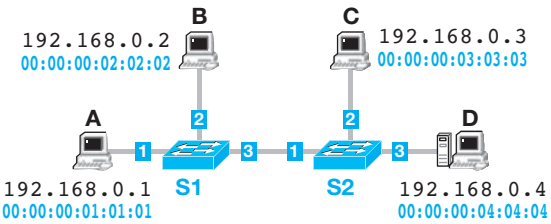
a) Elencare tutti i frame generati dal comando fino al suo termine.
b) Elencare le ARP cache di ogni host al termine del comando.
c) Riportare i filtering database degli switch al termine del comando.



3 Sia dato il seguente schema di rete, con indicati gli indirizzi di livello 3 (IP address) e livello 2 (MAC address). Tutti i sistemi sono stati appena avviati, quindi le ARP cache sono vuote così come le CAM degli switch. L'host **A** esegue il comando:

```
ping 192.168.0.4
```

a) Elencare tutti i frame generati dal comando fino al suo termine.
b) Elencare le ARP cache di ogni host al termine del comando.
c) Riportare i filtering database degli switch al termine del comando.



4 Calcolare quanto vale il campo Total Length di un pacchetto IP se l'intestazione è ampia 28 byte ed il campo dati ampio 500 byte.

5 Calcolare quanto vale il campo dati (payload) di un pacchetto IP se HIL vale 12 e Total Length 10000.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

6 Dato il seguente header IP (in esadecimale), calcolarne l'Header checksum e verificare se coincide con quello del pacchetto.

```
45 00 00 a8 52 87 00 00 80 01 af d1 0a 64
11 30 0a 64 12 05
```

7 Dato il seguente header IP (in esadecimale), calcolare la dimensione del campo dati (payload) trasportato.

```
45 00 00 a8 52 87 00 00 80 01 af d1 0a 64
11 30 0a 64 12 05
```

8 Dato il seguente header IP (in esadecimale), mostrare gli indirizzi IP mittente e destinazione, nonché il tipo di pacchetto trasportato.

```
45 00 00 a8 52 87 00 00 80 01 af d1 0a 64
11 30 0a 64 12 05
```

9 Calcolare quanto vale il campo Total Length di un pacchetto IP se l'intestazione è ampia 20 byte ed il campo dati ampio 400 byte.

10 Calcolare quanto vale il campo dati (payload) di un pacchetto IP se HIL vale 6 e Total Length 1000.

11 Dato il seguente header IP (in esadecimale), calcolarne l'Header checksum e verificare se coincide con quello del pacchetto.

```
45 00 00 3c 8e 9b 00 00 80 01 2a 62 c0 a8
00 72 c0 a8 00 01
```

12 Dato il seguente header IP (in esadecimale), calcolare la dimensione del campo dati (payload) trasportato.

```
45 00 00 3c 8e 9b 00 00 80 01 2a 62 c0 a8
00 72 c0 a8 00 01
```

13 Dato il seguente header IP (in esadecimale), mostrare gli indirizzi IP mittente e destinazione, nonché il tipo di pacchetto trasportato.

```
45 00 00 3c 8e 9b 00 00 80 01 2a 62 c0 a8
00 72 c0 a8 00 01
```

14 Calcolare quanto vale il campo Total Length di un pacchetto IP se l'intestazione è ampia 60 byte ed il campo dati ampio 400 byte.

15 Calcolare quanto vale il campo dati (payload) di un pacchetto IP se HIL vale 8 e Total Length 2000.

16 Dato il seguente header IP (in esadecimale), calcolarne l'Header checksum e verificare se coincide con quello del pacchetto.

```
45 00 05 dc 61 d1 20 00 80 01 30 cc c0 a8
00 72 c0 a8 00 c1
```

17 Dato il seguente header IP (in esadecimale), calcolare la dimensione del campo dati (payload) trasportato.

```
45 00 05 dc 61 d1 20 00 80 01 30 cc c0 a8
00 72 c0 a8 00 c1
```

18 Dato il seguente header IP (in esadecimale), mostrare gli indirizzi IP mittente e destinazione, nonché il tipo di pacchetto trasportato.

```
45 00 05 dc 61 d1 20 00 80 01 30 cc c0 a8
00 72 c0 a8 00 c1
```

19 Data la seguente trama Ethernet (in esadecimale), decodificarla.

```
00 15 c5 37 e1 c1 00 0b db 71 d7 39 08
00 45 00 00 a8 52 87 00 00 80 01 af d1
0a 64 11 30 0a 64 12 05 08 00 1c 2f e4
0e a3 0d
```

20 Data la seguente trama Ethernet (in esadecimale), decodificarla.

```
ff ff ff ff ff ff 00 16 ce 6e 8b 24 08
06 00 01 08 00 06 04 00 01 00 16 ce 6e
8b 24 c0 a8 00 72 00 00 00 00 00 00 c0
a8 00 01 00 15 c5 37 e1 c1 00 0b db 71
d7 39 08 00 45 00 00 a8 52 87 00 00 80
01 af d1 0a 64 11 30 0a 64 12 05 08 00
1c 2f e4 0e a3 0d
```

21 Data la seguente trama Ethernet (in esadecimale), decodificarla.

```
00 03 93 cd e6 be 00 0d 54 a5 c0 da 08 06
00 01 08 00 06 04 00 02 00 0d 54 a5 c0 da
0a 14 00 0b 00 03 93 cd e6 be 0a 14 14 0a
55 55 5 55 55 55 55 55 55 55 55 55 55 55
55 55 55 55 39 a7 d7 ec
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Indirizzamento IP

B3

L'indirizzamento IP è parte integrante del processo di instradamento dei messaggi sulla rete. Gli indirizzi IP, che devono essere univoci sulla rete di appartenenza, sono lunghi 32 bit (quattro byte) e sono spesso espressi scrivendo i valori decimali di ciascun byte separati dal carattere «punto», detta notazione **dotted** degli indirizzi IP (cfr. ***Sistemi e reti, vol. 1, Indirizzi IP e porte TCP***).

Esempi di indirizzi IP sono: 34.0.0.1, 129.130.7.4 e 197.67.12.3.

Agli indirizzi IP si associano, per comodità, una o più stringhe alfanumeriche di facile memorizzazione, dette **hostname**, che possono essere definite localmente al sistema operativo, in un file denominato **hosts**.

Questo approccio diviene impraticabile quando la rete IP cresce di dimensione e allora si preferisce utilizzare un sistema a base di dati distribuita per la gestione dei nomi (**DNS**, *Domain Name System*).

Occorre subito evidenziare che non sono i nodi della rete, cioè gli host e i gateway(NB) ad avere un indirizzo IP, bensì le interfacce fisiche a bordo dei nodi. Cosicché se un nodo ha tre interfacce (per esempio un gateway), esso ha tre indirizzi IP. Poiché la maggior parte dei nodi ha una sola interfaccia (per esempio gli host), è comune parlare dell'indirizzo IP di un nodo. Questo tuttavia è senza dubbio sbagliato nel caso dei router che hanno, per definizione, più di un'interfaccia fisica.

NB. *Si consideri che nel gergo TCP/IP gli IS (router) sono anche indicati con il termine **gateway**, mentre gli ES (stazioni) sono indicati con il termine **host**.*

L'assegnazione di indirizzi è quindi di fondamentale importanza per il buon funzionamento di una rete TCP/IP, sia per quelle locali, sia soprattutto per quella mondiale, cioè Internet.

L'autorità che gestisce l'assegnazione degli indirizzi IP su Internet è l'**ICANN** (*Internet Corporation for Assigned Names and Numbers*) tramite lo **IANA** (*Internet Assigned Numbers Authority*), che agisce sotto le direttive della RFC 2050.

Lo IANA opera tramite alcune agenzie continentali dette **RIR** (*Regional Internet Registry*), tra cui il **RIPE** (*Reseaux IP European*) per l'Europa. Ogni stato poi ha il proprio ente di riferimento che fa capo a questi. Infine i **LIR** (*Local Internet Registry*), dislocati sul territorio nazionale, gestiscono la distribuzione degli indirizzi IP agli utenti assieme agli **ISP** (*Internet Service Provider*, cfr. ***Internet Service Provider***).

I LIR italiani sono circa 600 (2012), tra cui i grandi operatori telefonici nazionali come Fastweb, Telecom, Tiscali, Wind e Vodafone.

Gli indirizzi IP comprendono due parti significative, che contengono informazioni implicite per il routing. La prima parte (**network prefix** o **netId**) indica la rete a cui appartiene un host; la seconda parte (**host number** o **hostId**) indica l'host all'interno della rete.

1 Classful

Il dio di Internet

«[...] Dio, almeno in Occidente, è spesso rappresentato come un uomo con una barba fluente e i sandali... se la Rete ha un dio, questo è probabilmente **Jon Postel**, un uomo che uguaglia questa descrizione a una T. Il diritto del signor Postel alla divinità informatica, al di là delle apparenze, deriva dal fatto che è il presidente e, in realtà, l'unico membro della *Internet Assigned Numbers Authority*, l'organismo che coordina quasi tutti gli indirizzi di Internet. [...]» (*The Economist, 8 febbraio 1997*).

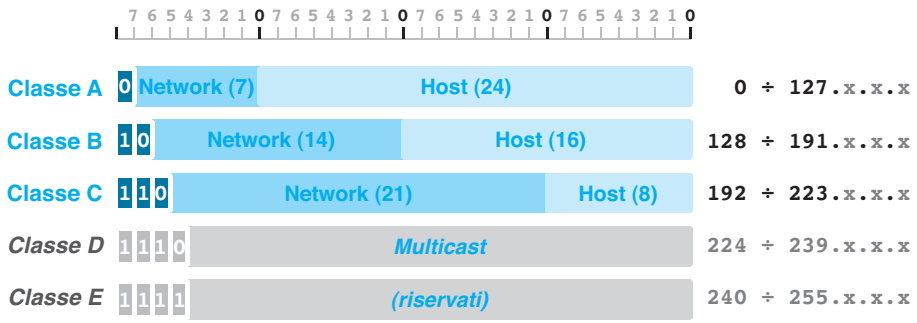
«Un nome indica quanto cerchiamo. Un indirizzo indica dov'è. Una via indica come arrivarci.»

(*Jon Postel, RFC 791, 1981*)

La specifica originale della classificazione degli indirizzi IPv4 è contenuta nelle RFC 760, 791 e 1166, ed è denominata **classful** network architecture. Essa è ufficialmente decaduta nel 1993 a favore della *Classless Inter-Domain Routing* (CIDR, RFC 1519, cfr.), pur contenendo elementi fondamentali per la comprensione di quest'ultima.

Gli indirizzi IPv4 sono ampi 32 bit, disposti su 4 byte in modo big endian.

All'interno dei byte i bit sono a loro volta disposti in big endian. Nello schema classful sono suddivisi in cinque classi, come schematizzato:



1. **Classe A**. Sono concepiti per poche reti di dimensioni molto grandi. I bit che indicano la rete (*netId*) sono 7 e quelli che indicano l'host (*hostId*) sono 24. Quindi si possono avere al massimo 2^7=128 reti di classe A, ciascuna con un numero di host pari a 2^{24}.

   Gli indirizzi di classe A sono riconoscibili dato che il primo byte dell'indirizzo è compreso tra **0** e **127** (ovvero il primo MSb sempre a 0).
2. **Classe B**. Sono concepiti per un numero medio reti di dimensioni medio-grandi. I bit che indicano la rete (*netId*) sono 14 e quelli che indicano l'host (*hostId*) sono 16. Quindi si possono avere 2^{14} reti di classe B, ciascuna con numero di host pari a 2^{16}. Gli indirizzi di classe B sono riconoscibili in quanto il primo campo dell'indirizzo è compreso tra **128** e **191** (ovvero i primi due MSb valgono sempre 01).
3. **Classe C**. Sono concepiti per molte reti di piccole dimensioni. I bit che indicano la rete (*netId*) sono 21 e quelli che indicano l'host (*hostId*) sono 8. Quindi si possono avere 2^{21} reti di classe C, ciascuna con un numero di host pari a 2^8. Gli indirizzi di classe C sono riconoscibili in quanto il primo campo dell'indirizzo è compreso tra **192** e **223** (ovvero i primi tre MSb valgono sempre 110).
4. **Classe D**. Sono riservati ad applicazioni di multicast secondo quanto descritto nella RFC 1112. Gli indirizzi di classe D sono riconoscibili in quanto il primo campo dell'indirizzo è compreso tra **224** e **239** (ovvero i primi quattro MSb valgono sempre 1110).
5. **Classe E**. Questi indirizzi sono riservati per usi futuri. Gli indirizzi di classe E sono riconoscibili in quanto il primo campo dell'indirizzo è compreso tra **240** e **255** (ovvero i primi cinque MSb valgono sempre 11111).

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Dato un indirizzo IP (cfr. ***Sistemi e reti, vol. 1, Indirizzi IP e porte TCP***), per individuare la sua classe di appartenenza è sufficiente discriminare la presenza dei **leading bit** della corrispondente classe, ovvero (vedi figura precedente[NB]):

Classe A: `b0 = 0;`
Classe B: `b0 b1 = 10;`
Classe C: `b0 b1 b2 = 110;`
Classe D: `b0 b1 b2 b3 = 1110;`
Classe E: `b0 b1 b2 b3 = 1111.`

Dato un indirizzo IP, le due componenti (netId, hostId), si estraggono con un'opportuna maschera di bit detta **netmask**.

Ogni classe ha la propria netmask, che è un valore a 32 bit con i MSb a 1 e i LSb a 0 in ragione delle lunghezze dei rispettivi netId e hostId:

Netmask classe A:
`11111111000000000000000000000000 = ff000000h = 255.0.0.0`
Netmask classe B:
`11111111111111110000000000000000 = ffff0000h = 255.255.0.0`
Netmask classe C:
`11111111111111111111111100000000 = ffffff00h = 255.255.255.0`

NB. *Ovviamente la classe di un indirizzo IP la si può dedurre immediatamente ricordando gli intervalli numerici del primo byte delle classi e confrontandoli con il primo byte dell'indirizzo: l'indirizzo IP 137.168.0.1 è di classe B perché 137 sta tra 128 e 191, e così via.*
Anche la netId si deduce facilmente dall'indirizzo IP, ricordandosi quanti byte sono a 255 nelle netmask delle classi: siccome l'indirizzo IP 137.168.0.1 è di classe B e la classe B ha netmask con i primi due byte a 255 (255.255.0.0), il netId dell'indirizzo coincide con i primi due byte del suo indirizzo, ovvero 137.168.0.0.

ESEMPIO

Indirizzi IP: classi e reti di appartenenza

Determinare la classe dei seguenti indirizzi IP e verificare l'appartenenza reciproca alla stessa rete: `10.1.0.3, 192.168.0.1, 137.168.0.1, 137.168.249.17.`

Le classi degli indirizzi si determinano con una maschera appropriata da confrontare con i rispettivi leading bit:

maschere	leading bit
IPMASK_A=10000000b=80h=128d	**IPCLASS_A**=**0**0000000b=00h=0d
IPMASK_B=11000000b=c0h=192d	**IPCLASS_B**=**10**000000b=80h=128d
IPMASK_C=11100000b=e0h=224d	**IPCLASS_C**=**110**00000b=c0h=192d

La classe dell'indirizzo si calcola dal primo byte dell'indirizzo, mascherato con la maschera delle tre classi e confrontato con le tre classi.

a) **10.1.0.3**
10 AND IPMASK_A = 0; siccome 0 coincide con IPCLASS_A, l'indirizzo è di **classe A**.

b) **192.168.0.1**
192 AND IPMASK_A = 128, non è di classe A; 192 AND IPMASK_B = 192, non è di classe B. 192 AND IPMASK_C = 192, coincide con IPCLASS_C, quindi l'indirizzo è di **classe C**.

c) **137.168.0.1**
137 AND IPMASK_A = 128, non è di classe A; 137 AND IPMASK_B = 192 = 128, coincide con IPCLASS_B, quindi l'indirizzo è di **classe B**.

d) **137.168.249.17**
137 AND IPMASK_A = 128, non è di classe A; 137 AND IPMASK_B = 192 = 128, coincide con IPCLASS_B, quindi l'indirizzo è di **classe B**.

Per verificare le appartenenze alla stessa rete servono le netmask.

netmask

IPNETMASK_A=11111111000000000000000000000000b=ff000000h=4278190080d
IPNETMASK_B=11111111111111110000000000000000b=ffff0000h =4294901760d
IPNETMASK_C=11111111111111111111111100000000b=ffffff00h =4294967040d

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

ESEMPIO

Per estrarre i netId bisogna trasfomare gli indirizzi dotted in decimale:

10.1.0.3 =0ah.01h.00h.03h=0a010003h=167837699d
192.168.0.1 =c0h.a8h.00h.01h=c0a80001h=3232235521d
137.168.0.1 =89h.a8h.00h.01h=89a80001h=2309488641d
137.168.249.17=89h.a8h.f9h.11h =89a8f911h =2309552401d

Ora si estraggono i netId tramite le netmask corrispondenti:

netId(10.1.0.3) =IPNETMASK_A AND 167837699 =167772160
netId(192.168.0.1) =IPNETMASK_C AND 3232235521=3232235520
netId(137.168.0.1) =IPNETMASK_B AND 2309488641=2309488640
netId(137.168.249.17)=IPNETMASK_B AND 2309552401=2309488640

Si nota che **137.168.0.1** e **137.168.249.17** sono due indirizzi della **stessa rete**.

PROGRAMA

Classful e reti di appartenenza

Dati i seguenti indirizzi IP (`10.1.0.3, 192.168.0.1, 137.168.0.1, 137.168.249.17`), scrivere un programma per verificare l'appartenenza alla stessa rete.

OUTPUT

```
10.   1.  0.  3 classe A (netmask: ff000000) e netId 0A000000
192.168.  0.  1 classe C (netmask: ffffff00) e netId C0A80000
137.168.  0.  1 classe B (netmask: ffff0000) e netId 89A80000
137.168.249. 17 classe B (netmask: ffff0000) e netId 89A80000

137.168.0.1 e 137.168.249.17 stessa rete
```

Seguendo le indicazioni dell'***ESEMPIO*** precedente, un esempio di programma in linguaggio C potrebbe essere:

```
00 #include <stdio.h>
01
02 typedef struct {
03   unsigned char mask;
04   unsigned char leading;
05   unsigned int netmask;
06 } CLASSFUL;
07 CLASSFUL classiIp[] =
   {{0x80,0x00,0xff000000},{0xc0,0x80,0xffff0000},{0xe0,0xc0,0xffffff00}};
08
09 typedef struct {
10   unsigned char a,b,c,d;  // IP dotted
11   unsigned int ip;        // IP decimale
12   unsigned int netid;     // netId corrispondente alla classe di appartenenza
13 } IPINFO;
14 IPINFO Ip[4];
15
16 int main (void)
17 {
18   int i,j;
19   // I quattro indirizzi dati
20   Ip[0].a= 10; Ip[0].b=  1; Ip[0].c=  0; Ip[0].d=3;
21   Ip[1].a=192; Ip[1].b=168; Ip[1].c=  0; Ip[1].d=1;
22   Ip[2].a=137; Ip[2].b=168; Ip[2].c=  0; Ip[2].d=1;
23   Ip[3].a=137; Ip[3].b=168; Ip[3].c=249; Ip[3].d=17;
24
25   i = 0;
26   for (i=0; i<4; i++)
```

PROGRAMMA

```
27   {
28     for (j=0;j<3;j++)
29     {
30       Ip[i].ip = Ip[i].a*0x1000000 + Ip[i].b*0x10000 + Ip[i].c*0x100 + Ip[i].d;
31       if ((Ip[i].a & classiIp[j].mask) == classiIp[j].leading)
32       {
33         Ip[i].netid = Ip[i].ip & classiIp[j].netmask;
34         printf("\n%3d.%3d.%3d.%3d",Ip[i].a,Ip[i].b,Ip[i].c,Ip[i].d);
35         printf("classe %c (mask: %x) e netId %X",'A'+j, classiIp[j].netmask, Ip[i].netid);
36       }
37     }
38   }
39
40   for (i=0; i<4-1; i++)
41   {
42     for (j=i+1;j<4;j++)
43     {
44       if (Ip[i].netid==Ip[j].netid)
45       {
46         printf("\n%d.%d.%d.%d",Ip[i].a,Ip[i].b,Ip[i].c,Ip[i].d);
47         printf(" e %d.%d.%d.%d stessa rete",Ip[j].a,Ip[j].b,Ip[j].c,Ip[j].d);
48       }
49     }
50   }
51
52   return 0;
53 }
```

Dopo aver memorizzato in una struttura dati ad hoc i valori caratteristici di classful (per le tre classi, maschere e leading bit corrispondenti, **righe 2÷8**), per analizzare gli indirizzi IP è sufficiente possederne una versione in notazione decimale. La conversione è in evidenza (**riga 30**).

Il confronto per determinare la classe di appartenenza è il classico confronto «a maschera di bit» tra il primo byte dell'indirizzo, la maschera e i leading bit della classe (**riga 31**).

2 Instradamento classful

Lo schema di indirizzamento classful facilita il routing dei pacchetti, dato che dall'indirizzo IP destinazione si può estrarre velocemente la parte netId. (netId): se confrontato con il netId dell'indirizzo di una propria interfaccia, si può desumere immediatamente se l'indirizzo IP destinazione appartiene alla stessa rete di quell'interfaccia o meno.

La regola di instradamento implicita a classful si sintetizza in:

1. *Un pacchetto IP con indirizzo destinazione avente netId uguale al netId di un proprio indirizzo IP può essere instradato* **direttamente**, *semplicemente inoltrandolo sulla stessa interfaccia alla quale il proprio indirizzo IP è associato.*
2. *Un pacchetto IP con indirizzo destinazione avente netId diverso dal netId di una qualsiasi propria interfaccia, deve essere inoltrato a una stazione intermedia detta* **gateway di default** *(a volte indicato come* **next hop***).*

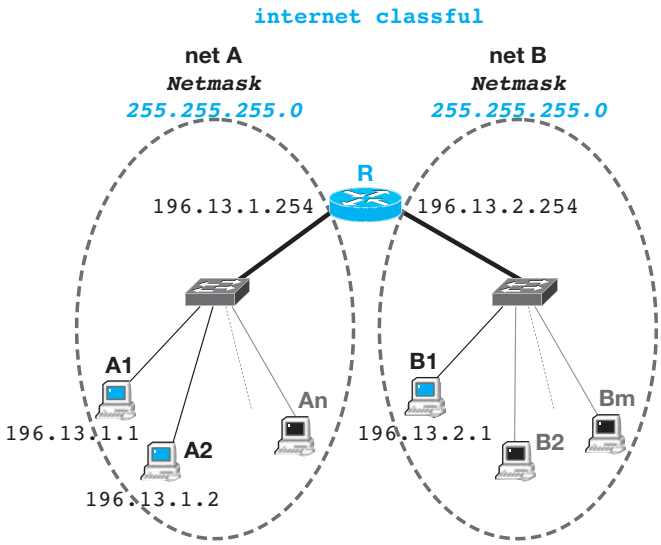
Vediamo ora come si mette in pratica l'instradamento implicito classful in un caso reale ma generalizzabile.

Per esempio, siano date due reti di classe C (196.13.1.x e 196.13.2.x) da interconnettere con un router. Chiamiamo le due reti **net A** e **net B**, e **A1,**

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

..., An e B1, ..., Bm gli host connessi. I numeri **n** e **m** non possono superare 256, essendo reti di classe C. Il router **R** deve avere due interfacce, e quindi due indirizzi IP, uno appartenente a net A e l'altro appartenente a net B.

La netmask di entrambe le reti è quella di classe C, ovvero 255.255.255.0:



Vediamo l'instradamento di un pacchetto IP spedito da **A1** verso **A2**.

Il routing attuato da A1 è il seguente:

1. Estrazione del netId dal proprio indirizzo e dall'indirizzo di A2:
   netId(A1) = (196.13.1.1 AND 255.255.255.0) = 196.13.1.0;
   netId(A2) = (196.13.1.2 AND 255.255.255.0) = 196.13.1.0.
2. Confronto dei due netId: sono uguali, cioè consegna **diretta**. Basta inoltrare il pacchetto sulla stessa interfaccia, con il MAC destinatario di A2 eventualmente recuperato con ARP, e il pacchetto giungerà a destinazione.

Vediamo ora l'instradamento di un pacchetto IP spedito da **A1** verso **B1**.

Il routing attuato da A1 è il seguente:

1. Estrazione del netId dal proprio indirizzo e dall'indirizzo di B1:
   netId(A1) = (196.13.1.1 AND 255.255.255.0) = 196.13.1.0
   netId(B1) = (196.13.2.1 AND 255.255.255.0) = 196.13.2.0
2. Confronto dei due netId: sono diversi, quindi consegna indiretta tramite **gateway di default**. Il pacchetto va inviato al gateway di default (R) inbustandolo con l'indirizzo MAC di R (eventualmente recuperato previo ARP).

Si deduce che i parametri fondamentali di un host per il routing classful sono la **netmask** (della rete di appartenenza) e l'**indirizzo IP di un gateway di default**.

Benché rudimentale rispetto ai router, gli host utilizzano una tabella di routing che sintetizza il ragionamento dell'instradamento appena esposto.

Per esempio, per l'host **A1** la TdI viene rappresenta nel seguente modo:

Indirizzo	Mask	Gateway	Interfaccia
0.0.0.0	0.0.0.0	196.13.1.254	196.13.1.1
196.13.1.0	255.255.255.0	196.13.1.1	196.13.1.1

Le righe della tabella sono due, e sono rispettivamente le due regole per la consegna tramite gateway di default (o consegna indiretta) e la regola per la consegna diretta.

Il processo di routing per la destinazione **A2** descritto nel paragrafo precedente viene attuato tramite la TdI nel seguente modo, ricordando che l'indirizzo destinazione di A2 vale 196.13.1.2:

a. Si prende in considerazione la prima riga della TdI e si effettua l'AND tra l'indirizzo destinazione e il valore della colonna *Mask* (0.0.0.0): 196.13.1.2 AND 0.0.0.0 = 0.0.0.0[(NB)].

Siccome il risultato coincide con il valore della colonna *Indirizzo* la regola è vera (consegna indiretta).

b. Si ripete il procedimento per la seconda riga: si effettua l'AND tra l'indirizzo destinazione e il valore della colonna *Mask* (255.255.255.0): 196.13.1.2 AND 255.255.255.0 = 196.13.1.0.

Anche in questo caso la regola è vera, ma siccome nel valore *Mask* i bit a uno sono più numerosi (rispetto al valore *Mask* della regola precedente), tra le due si sceglie questa (consegna diretta).

NB. *Si nota che, quando presente, la regola che contiene come valore di Mask 0.0.0.0, è una regola sempre vera per ogni indirizzo (da instradare). Questa regola è anche detta «regola per pacchetti stranieri» o «dell'ultima opportunità».*

PROGRAMMA

Instradamento classful

Data la TdI dell'host A del caso appena descritto, realizzare l'instradamento di un pacchetto diretto a B5 (196.13.2.5) con un programma in linguaggio C.

OUTPUT

```
Indirizzo       Mask            Gateway         Use
0.0.0.0         0.0.0.0         196.13.1.254    196.13.1.1
196.13.1.0      255.255.255.0   196.13.1.1      196.13.1.1
Regola n.1 per 196.13.2.5:  vera.
Regola n.2 per 196.13.2.5:  falsa.
```

Dopo aver convenientemente descritto la TdI dell'host A in una struttura dati (**righe 2÷11**), si utilizza una funzione che converte un indirizzo dotted in forma decimale (**righe 43÷49**).

Seguendo le indicazioni del testo, un esempio di programma in linguaggio C potrebbe essere:

```
00 #include <stdio.h>
01
02 typedef struct {
03   unsigned char Ia,Ib,Ic,Id;  // colonna Indirizzo
04   unsigned char Ma,Mb,Mc,Md;  // colonna Mask
05   unsigned char Ga,Gb,Gc,Gd;  // colonna Gateway (next hop)
06   unsigned char Ua,Ub,Uc,Ud;  // colonna Use (interfaccia d'uscita)
07 } TDI;
08 TDI tdiHost[] = {
09   {   0,0,0,0,        0,0,0,0,  196,13,1,254,  196,13,1,1},
10   {196,13,1,0,  255,255,255,0,    196,13,1,1,  196,13,1,1}
11 };
12
13 unsigned int DottedToDecimal(unsigned char a,unsigned char b,unsigned char c,unsigned char d);
14
15 int main (void)
16 {
17   int i;
18   unsigned char a,b,c,d;
19
20   printf("Indirizzo\tMask\t\tGateway\t\tUse\n");  // stampa della TdI
21   for (i=0; i<sizeof(tdiHost)/sizeof(TDI); i++)
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

PROGRAMMA

```
22      printf("%d.%d.%d.%d\t%d.%d.%d.%d\t%d.%d.%d.%d\t%d.%d.%d.%d\n",
                              tdiHost[i].Ia,tdiHost[i].Ib,tdiHost[i].Ic,tdiHost[i].Id,
                              tdiHost[i].Ma,tdiHost[i].Mb,tdiHost[i].Mc,tdiHost[i].Md,
                              tdiHost[i].Ga,tdiHost[i].Gb,tdiHost[i].Gc,tdiHost[i].Gd,
                              tdiHost[i].Ua,tdiHost[i].Ub,tdiHost[i].Uc,tdiHost[i].Ud);
23
24    a=196;  b=13;  c=2;  d=5;  // Indirizzo da instradare
25    for (i=0; i<sizeof(tdiHost)/sizeof(TDI); i++)
26    {
27      unsigned int u;
28
29      printf("Regola n.%d per %d.%d.%d.%d: ",i+1,a,b,c,d);
30      u = DottedToDecimal(tdiHost[i].Ma,tdiHost[i].Mb,tdiHost[i].Mc,tdiHost[i].Md);
31      u = u & DottedToDecimal(a,b,c,d);
32
33      if (u == DottedToDecimal(tdiHost[i].Ia,tdiHost[i].Ib,tdiHost[i].Ic,tdiHost[i].Id))
34        printf(" vera.\n");
35      Else
36        printf(" falsa.\n");
37    }
38
39    return 0;
40  }
41
42
43  unsigned int DottedToDecimal(unsigned char a,unsigned char b,unsigned char c,unsigned char d)
44  {
45    unsigned int ul;
46
47    ul = a*0x1000000 + b*0x10000 + c*0x100 + d;
48    return ul;
49  }
```

Le righe (regole) della TdI vengono analizzate con il ciclo a **righe 25÷37**.

Per ogni regola, per verificare se è vera sull'indirizzo specificato (**a.b.c.d**):

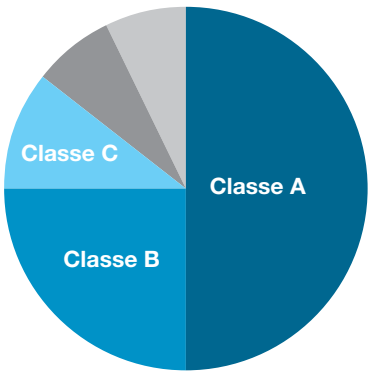
a) si trasforma in decimale il valore *Mask* della regola (**riga 30**);

b) si effettua la maschera AND con l'indirizzo dato (**riga 31**);

c) confronta il risultato con il valore *Indirizzo* della regola, a sua volta trasformato in decimale (**riga 33**).

2.1 Problemi dell'indirizzamento classful

L'intero spazio degli indirizzi potenziale di IPv4, 2^{32} = 4.294.967.296, cioè più di 4 miliardi di indirizzi disponibili, se suddiviso con lo schema classful è rappresentabile graficamente tra le classi:



La gerarchia degli indirizzi descritta dal modo classful, a causa dell'enorme e anche imprevista diffusione di TCP/IP, è diventata troppo rigida, infatti:

- Internamente alle organizzazioni che fanno richiesta di indirizzi IP si sprecano molti indirizzi: ogni sottorete interna, fatalmente, non sfrutta tutti i numeri di rete acquistati.
- Gli ISP, a loro volta, sono costretti a distribuire blocchi di indirizzi sovradimensionati rispetto alle esigenze del richiedente. Tipico il caso delle reti di classe B, le prime a soffrire del problema: molte organizzazioni posseggono più di 256 macchine (rete di classe C), ma molte meno di 65536 (rete di classe B).
- Infine, le reti di classe A sono troppo grandi (2^{24}= 16777216 indirizzi ognuna) e poche organizzazioni, in realtà, sono all'altezza di richiederne un lotto (a oggi una sessantina, l'ultima assegnazione è del 2010).

Tutto ciò produce anche l'effetto di costringere i router degli ISP a usare tabelle di instradamento inutilmente ampie (per esempio, nel caso di instradamento di numerose reti di classe C assegnate alla medesima organizzazione).

Per rimediare a questi problemi l'IETF ha introdotto, a partire dal 1993, un nuovo schema di indirizzamento per IPv4, allarmato anche dalla realistica prospettiva di un esaurimento degli indirizzi IP, effettivamente accaduta il 3 febbraio 2011.

Il nuovo schema di indirizzamento per IPv4 si chiama **CIDR** (*Classless Inter-Domain Routing*, RFC 1519) ed è quello attualmente in uso.

Atraverso l'adozione progressiva della tecnica di subnetting su reti private e supernetting (CIDR) i problemi di classful sono stati arginati, in attesa dell'adozione di uno schema di indirizzamento radicalmente differente (cfr. ***Appendice. IPv6***).

Con le reti private e il subnetting si affronta il problema dello spreco degli indirizzi all'interno delle organizzazioni: ogni host non necessita più di un indirizzo IP classful.

Con il supernetting/CIDR si affronta il problema dello spreco degli indirizzi dovuto alla rigidità delle classi classful, consentendone l'aggregazione in base alle esigenze dell'organizzazione che ne fa richiesta.

Reti fisiche e logiche

Il subnetting prevede normalmente che ogni subnet coincida con una rete fisica, per esempio una switched LAN.

In realtà due host appartenenti a due subnet differenti possono essere anche direttamente connessi sulla stessa rete fisica, per esempio collegati allo stesso switch. Tuttavia una comunicazione tra i due host dovrà passare necessariamente dal gateway che connette le due subnet.

Alla prima trasmissione di un host verso l'altro, il gateway invierà al mittente un messaggio ICMP *Redirect* per informare l'host di spedire direttamente i pacchetti sull'interfaccia di rete.

3 Subnetting

Il problema della suddivisione di reti classful in più sottoreti fisiche all'interno di una stessa organizzazione è stato affrontato con il **subnetting** (RFC 950).

Il subnetting diventa molto efficace se applicato alle cosiddette **reti private** (*Private Internets*, RFC 1918), ovvero a lotti di indirizzi IP destinati esclusivamente a indirizzare host che *non* partecipano direttamente alla rete Internet mondiale.

Un'organizzazione quindi non deve più richiedere lotti di indirizzi classful per ogni sottorete che intende realizzare ed è libera di utilizzare uno (o più) di questi lotti speciali per assegnare indirizzi IP alle proprie macchine in modo del tutto autonomo. Gli host locali potranno partecipare, sebbene «indirettamente», alla rete Internet mondiale attraverso alcune stazioni router con interfacce sia sulla rete privata, sia sulla rete pubblica (cfr. ***Sistemi e reti, vol. 3, Internetworking. NAT, Proxy***).

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Così facendo si configura un apparente paradosso: milioni di macchine che operano in Internet hanno lo stesso indirizzo IP (per esempio 192.168.0.1).

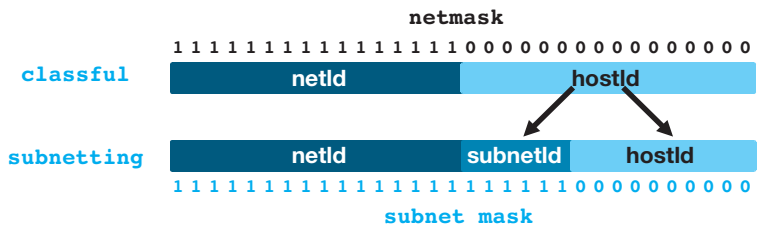
Le reti private hanno anche introdotto il concetto di *intranet*, sostanzialmente LAN organizzate sul modello TCP/IP.

I lotti di indirizzi IP riservati per le reti private e sottratti alla distribuzione (e alla circolazione) su Internet sono:

range IPv4	netmask	n. indirizzi	classful
10.0.0.0 ÷ 10.255.255.255	255.0.0.0	2^{24} = 16.777.216	*1 rete di classe A*
172.16.0.0 ÷ 172.31.255.255	255.255.0.0	2^{20} = 1.048.576	*16 reti di classe B contigue*
192.168.0.0 ÷ 192.168.255.255	255.255.255.0	2^{16} = 65.536	*256 reti di classe C contigue*

Ora è possibile scegliere liberamente il lotto (o *pool*) di indirizzi privati più idoneo alle caratteristiche della rete da organizzare, decidendo per ogni sottorete distinta un valore detto **subnet mask**, del tutto analogo alla netmask, ma con una quantità consecutiva di «uni» tale da «ricoprire» la parte netId degli indirizzi e parte della porzione hostId. I rimanenti bit finali rimarranno come al solito, tutti a zero.

In questo modo, con i valori netmask e subnet mask un indirizzo IP si suddivide in tre parti: netId, **subnetId** e hostId:



Il valore subnetId ora ha il compito di numerare le sottoreti operanti all'interno della rete privata; per ogni sottorete saranno disponibili tanti host quanti ne prevede la parte hostId.

Con il subnetting il meccanismo di routing implicito di classful rimane invariato. L'unica differenza riguarda il fatto che ogni host deve essere configurato specificando la subnet mask di appartenenza (invece della sola netmask della classe come in classful).

ESEMPIO

Subnetting

Si deve organizzare un piano degli indirizzi in un'organizzazione che prevede tre gruppi di macchine operanti su settori differenziati, l'uno da 50 host, il secondo da 30, il terzo da 20.

Dai dati forniti dal testo si determina che il numero totale degli host da organizzare è **100**, mentre le sottoreti previste sono **3**. La sottorete più ampia dovrà indirizzare **50** host.

La scelta del lotto di indirizzi privati ricade all'interno delle reti di classe C, che possono indirizzare fino a 256 host, numero sufficiente per soddisfare la richiesta del testo (100 host).

Tra le 256 reti private di classe C si sceglie la prima, 192.168.0.0 (netmask 255.255.255.0).

In questo modo rimangono a disposizione per il subnetting gli ultimi **8** bit (*hostId*).

Dovendo organizzare 3 sottoreti, sono sufficienti **2** bit di subnetId.

Infatti $2^1 < \mathbf{3} < 2^2$, cioè **1** bit è troppo poco, consente solo 2 sottoreti, mentre con **2** bit si possono ottenere 4 sottoreti: 00, 01, 10, 11.

Di contro, alla parte *hostId* rimangono 8 – 2 = **6** bit, per un totale teorico di 2^6 = **64** host per sottorete. Questo soddisfa la richiesta della dimensione massima della sottorete riportata dal testo (50 macchine). ▶

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

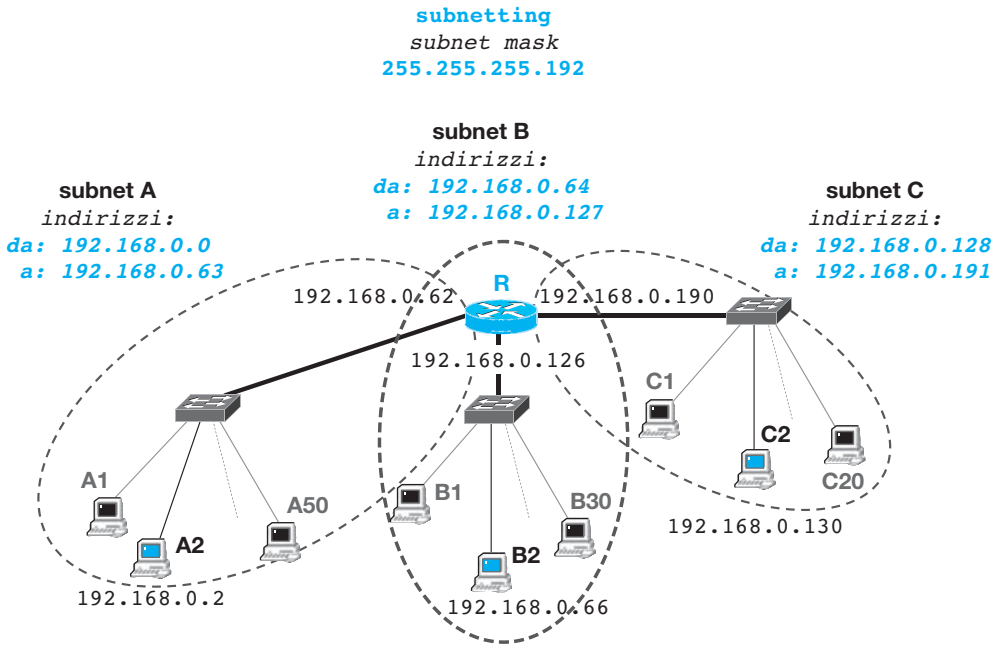
ESEMPIO

In definitiva il piano degli indirizzi prevede una subnet mask di lunghezza pari a **26 bit a 1**, e la rete suddivisa in subnet può essere espressa come **192.168.0.0** con subnet mask **255.255.255.192**.

Si nota, infatti, che 26 bit a 1 (subnet mask) e 6 bit a 0 (hostId) si rappresentano, a quartetti, come:

```
11111111 . 11111111 . 11111111 . 11000000
    ↓          ↓          ↓          ↓
   255    .   255    .   255    .   192
```

Naturalmente le due sottoreti devono essere interconnesse con un router **R** che deve avere almeno tre interfacce, una per ogni subnet:



La rete ha quindi subnet mask = **255.255.255.192**, lunghezza subnetId=**2** e lunghezza hostId=**6**.

Le quattro subnet avranno i seguenti valori (l'ultima non sarà utilizzata):

subnet A

subnetId = **00**

da: 11000000.1010100.00000000.**00**000000, (192.168.0.0)
a: 11000000.1010100.00000000.**00**111111, (192.168.0.63)

subnet B

subnetId = **01**

da: 11000000.1010100.00000000.**01**000000, (192.168.0.64)
a: 11000000.1010100.00000000.**01**111111, (192.168.0.127)

subnet C

subnetId = **10**

da: 11000000.1010100.00000000.**10**000000, (192.168.0.128)
a: 11000000.1010100.00000000.**10**111111, (192.168.0.191)

subnet D

subnetId = **11**

da: 11000000.1010100.00000000.**11**000000, (192.168.0.192)
a: 11000000.1010100.00000000.**11**111111, (192.168.0.255)

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

PROGRAMMA

Calcolo delle subnet

Dato un indirizzo IP privato e una subnet mask in formato CIDR (direttamente il numero dei bit a 1), calcolare e stampare tutte le subnet disponibili.

OUTPUT

```
Rete privata del gruppo 192.168.0.0 (netmask /24)
Subnet Mask /26; subnetid: 11
Subnet disponibili:
192.168.168.0
192.168.168.64
192.168.168.128
192.168.168.192
```

Per ridurre il numero di righe del programma, si forniscono i due dati (*indirizzo IP privato* e *subnet mask*) direttamente nel codice (hard coded), **righe 17÷18**.

Per visualizzare la subnetId in binario, si usa la funzione *DectoszBin()*, qui solo inclusa, ma già riportata nel ***PROGRAMMA. Autonegoziazione***.

Seguendo le indicazioni del testo, un esempio di programma in linguaggio C potrebbe essere:

```
00 #include <stdio.h>
01 #include "dectoszbin.c"
02
03 typedef struct {
04   unsigned char id1,id2;   // identificativi classi private
05   unsigned int netmasklen; // lunghezza netmask
06 } PRIVATEIP;
07 PRIVATEIP Ipp[] = {{10,0,8},{172,16,16},{192,168,24}};
08
09
10 int main (void)
11 {
12   int i,idx;
13   unsigned char a,b,c,d,subnetlen,hostlen;
14   unsigned int subnetmask, subnetid, u;
15   char szBin[32];
16
17   a=192;  b=168;  c=0;  d=0;  // indirizzo IP privato da usare
18   Subnetmask = 26;            // subnet mask da applicare
19
20   idx = -1;
21   for (i=0;i<3;i++)
22     if (a==Ipp[i].id1) idx = i;  // gruppo privato individuato
23   if (idx==-1) return 1;
24
25   subnetlen = subnetmask - Ipp[idx].netmasklen;
26   if (subnetmask<Ipp[idx].netmasklen) return 2;
27
28   hostlen = 32 - subnetmask;
29   subnetid = 1<<subnetlen;
30   DectoszBin(subnetid-1,subnetlen,szBin);
31
32   printf("Rete privata del gruppo %d.%d.%d.%d (netmask /%d)\n",
                                  Ipp[idx].id1,Ipp[idx].id2,0,0,Ipp[idx].netmasklen);
33   printf("Subnet Mask /%d; subnetid: %s\n",subnetmask,szBin);
34   printf("Subnet disponibili:\n");
35   for (i=0;i<subnetid;i++)
36   {
37     u = i << hostlen;
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

PROGRAMMA

```
38      switch (idx)
39      {
40        case 0:
41          printf("%d.%d.%d.%d\n",
                    Ipp[idx].id1,(u & 0x00ff0000)>>16,(u & 0x0000ff00)>>8,(u & 0x000000ff));
42          break;
43        case 1:
44          printf("%d.%d.%d.%d\n",
                    Ipp[idx].id1,Ipp[idx].id2,(u & 0x0000ff00)>>8,(u & 0x000000ff));
45          break;
46        case 2:
47          printf("%d.%d.%d.%d\n",
                    Ipp[idx].id1,Ipp[idx].id2,Ipp[idx].id2,(u & 0x000000ff));
48          break;
49      }
50    }
51
52    return 0;
53  }
```

Con le **righe 21÷22** si individua il lotto della rete privata impostata.

Con la **riga 25** si calcola il n. di bit riservati alla *subnetId*.

Con la **riga 28** si calcola il n. di bit riservati all'*hostId*.

Con la **riga 29** si calcola il valore della *subnetId*, che coincide con il numero di subnet disponibili, come $2^{\text{subnetlen}}$, e usando l'operatore di *shift a sinistra* (<<), che equivale all'elevamento a potenze di due.

Dopodiché si calcolano le singole subnet semplicemente enumerandole con un ciclo, ma spostando i bit nella posizione corretta, cioè a sinistra dell'*hostId* (**riga 37**).

Infine, la stampa delle singole subnet in formato dotted è ottenuta isolando i singoli byte che contengono la stringa *subnetId+hostId* con opportuni shift a destra (>>) alle **righe 41**, **44** e **47**.

4 Instradamento con subnetting

Con il subnetting vengono introdotte alcune notazioni speciali (**Special addresses**, RFC 1009) che, in modo conciso, significano altrettante condizioni relativamente agli indirizzi IP.

Scrivendo l'indirizzo IP nella forma **`{netId, hostId}`** nel modo classful oppure **`{netId, subnetId, hostId}`** nel modo subnetting, e assumendo che **`0`** significhi «bit tutti a zero» e **`-1`** significhi «bit tutti a 1» quando presenti nei campi netId, subnetId o hostId, si hanno i seguenti casi:

(a) **`{0, 0}`** *This host on this network.*
Utilizzabile come indirizzo IP sorgente, indica genericamente un host che non ha (ancora) un indirizzo IP. Si scrive anche 0.0.0.0.

(b) **`{0, hostId}`** *Specified host on this network.*
Utilizzabile come indirizzo IP sorgente, indica un preciso host sulla propria rete (indeterminata) di appartenenza.

(c) **`{-1, -1}`** *Limited broadcast.*
Utilizzabile come indirizzo IP destinazione, il pacchetto deve essere in-

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

viato a tutti gli host di una rete o subnet, ma mai ad altre. Si scrive anche 255.255.255.255.

(d) **`{netId, -1}`** *Directed broadcast to specified network.*
Utilizzabile come indirizzo IP destinazione, il pacchetto deve essere inviato a tutti gli host della rete classful indicata in netId.

(e) **`{netId, subnetId, -1}`** *Directed broadcast to specified subnet.*
Utilizzabile come indirizzo IP destinazione, il pacchetto deve essere inviato a tutti gli host della subnet indicata in subnetId, a sua volta appartenente alla rete indicata in netId.

(f) **`{netId, -1, -1}`** *Directed broadcast to all subnets of network.*
Utilizzabile come indirizzo IP destinazione, il pacchetto deve essere inviato a tutti gli host di tutte le subnet della rete specificata in netId.

(g) **`{127, any}`** *Internal host loopback address.*
Indirizzo di loopback (cfr. ***Sistemi e reti, vol. 1, Applicazioni***). Utilizzabile sia come indirizzo sorgente che destinazione di un pacchetto, non viene passato sulla rete fisica. Classico indirizzo di loopback è 127.0.0.1, indicato anche come **localhost**.

(h) **`{netId, 0}`** *Specified network.*
Non utilizzabile come indirizzo IP. Indica la rete specificata in netId.

(i) **`{netId, subnetId, 0}`** *Specified subnet.*
Non utilizzabile come indirizzo IP. Indica la subnet specificata in subnetId della rete specificata in netId.

Queste notazioni sono estremamente utili, come si vedrà, soprattutto per il routing, ma introducono una limitazione: negli indirizzamenti ottenuti con subnet non si possono più usare gli indirizzi che indicano la rete, come prescritto nelle notazioni **(h)** e **(i)**, né gli indirizzi di broadcast diretto, come indicato nelle notazioni **(d)**, **(e)** e **(f)**.

In sostanza, una volta definita una subnet si ha a disposizione il suo nome (nella forma **`{netId, subnetId, 0}`**) e il suo intero gruppo di host (nella forma **`{netId, subnetId, -1}`**), ma questi due indirizzi non possono essere assegnati a delle stazioni della subnet.

Per esempio, nella prima subnet della rete di classe C 192.168.0.0 con netmask 255.255.255.192 (vedi ***ESEMPIO*** precedente), i due indirizzi 192.168.0.0 e 192.168.0.63 non possono essere assegnati a delle stazioni, perché indicano rispettivamente il nome della subnet e tutti i 62 host della subnet.

Ora per un host instradare i pacchetti IP in una rete su cui è stato effettuato il subnetting diventa abbastanza sistematico, tenendo presente le regole di consegna diretta (per esempio se l'indirizzo IP di destinazione sta nella stessa sottorete del mittente) e indiretta (se l'indirizzo IP di destinazione non sta nella stessa sottorete del mittente).

Per un gateway che connette due o più sottoreti le regole di instradamento sono altrettanto immediate.

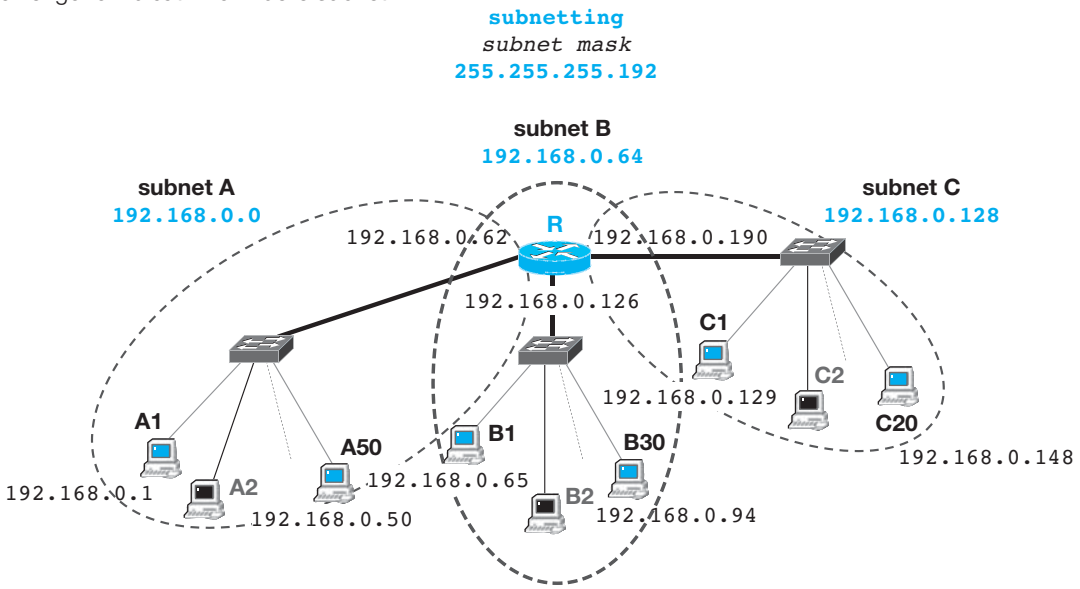
Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

ESEMPIO

Instradamento con subnetting

Si deve organizzare un piano degli indirizzi in un'organizzazione che prevede tre gruppi di macchine operanti su settori differenziati, l'uno da 50 host, il secondo da 30, il terzo da 20. Indicare le tabelle di instradamento delle macchine significative.

Il piano degli indirizzi per la rete proposta potrebbe di nuovo essere quello indicato nell'Esempio precedente. Ora però vengono indicati i nomi delle subnet.



Le tre subnet avranno i seguenti valori caratteristici:

subnet A

subnet mask: **255.255.255.192**
nome: 192.168.0.0
broadcast: 192.168.0.63
indirizzi disponibili: 192.168.0.1 ÷ 192.168.0.64

subnet B

subnet mask: **255.255.255.192**
nome: 192.168.0.64
broadcast: 192.168.0.127
indirizzi disponibili: 192.168.0.65 ÷ 192.168.0.126

subnet C

subnet mask: **255.255.255.192**
nome: 192.168.0.128
broadcast: 192.168.0.191
indirizzi disponibili: 192.168.0.129 ÷ 192.168.0.190

Le macchine significative per quanto riguarda l'instradamento sono, per esempio, un host di una subnet (per esempio **A1**) e, naturalmente, il router **R**.

Per quanto riguarda la TdI dell'host **A1 (192.168.0.1)** sulla prima subnet:

Indirizzo	Mask	Gateway	Interfaccia	Metrica	
0.0.0.0	0.0.0.0	192.168.0.62	192.168.0.1	1	*(pacchetti stranieri)*
127.0.0.0	255.0.0.0	127.0.0.1	127.0.0.1	1	***(loopback)***
192.168.0.0	255.255.255.192	192.168.0.1	192.168.0.1	1	*(sulla rete di provenienza)*
192.168.0.1	255.255.255.255	127.0.0.1	127.0.0.1	1	***(host)***
192.168.0.63	255.255.255.255	192.168.0.1	192.168.0.1	1	*(broadcast sulla rete di provenienza)*
224.0.0.0	224.0.0.0	192.168.0.1	192.168.0.1	1	
255.255.255.255	255.255.255.255	192.168.0.1	192.168.0.1	1	***(broadcast limitato)***

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

ESEMPIO

Per quanto riguarda la TdI del router **R (192.168.0.62, 192.168.0.126, 192.168.0.190)**:

Indirizzo	Mask	Gateway	Interfaccia	Metrica	
0.0.0.0	0.0.0.0	(nulla)	(nulla)		*(pacchetti stranieri)*
127.0.0.0	255.0.0.0	127.0.0.1	127.0.0.1	1	***(loopback)***
192.168.0.0	255.255.255.192	192.168.0.62	192.168.0.62	1	*(sulle reti di provenienza)*
192.168.0.64	255.255.255.192	192.168.0.126	192.168.0.126	1	
192.168.0.128	255.255.255.192	192.168.0.190	192.168.0.190	1	
192.168.0.62	255.255.255.255	127.0.0.1	127.0.0.1	1	***(host)***
192.168.0.126	255.255.255.255	127.0.0.1	127.0.0.1	1	
192.168.0.190	255.255.255.255	127.0.0.1	127.0.0.1	1	
192.168.0.63	255.255.255.255	192.168.0.62	192.168.0.62	20	*(broadcast sulle reti di provenienza)*
192.168.0.127	255.255.255.255	192.168.0.126	192.168.0.126	20	
192.168.0.191	255.255.255.255	192.168.0.190	192.168.0.190	20	
224.0.0.0	224.0.0.0	192.168.0.62	192.168.0.62	1	
224.0.0.0	224.0.0.0	192.168.0.126	192.168.0.126	1	
224.0.0.0	224.0.0.0	192.168.0.190	192.168.0.190	1	
255.255.255.255	255.255.255.255	192.168.0.62	192.168.0.62	1	***(broadcast limitato)***
255.255.255.255	255.255.255.255	192.168.0.126	192.168.0.126	1	
255.255.255.255	255.255.255.255	192.168.0.190	192.168.0.190	1	

Le TdI riportate sono state ottenute da un sistema operativo di tipo Windows, tramite il comando **route** digitato da una shell di MSDOS (**route print**) e redirezionate su file (per esempio *route print > tdiA1.txt* e *route print > tdiR.txt*).

Per il sistema operativo Linux le stesse tabelle sono riportate con un formato leggermente diverso e in modo più compatto (non sono indicate le regole standard per i broadcast e l'host). Per esempio, la TdI del router **R** sotto Linux viene riportata nel seguente modo:

Destination	Genmask	Gateway	Flags	Metric	Use	
0.0.0.0	0.0.0.0	(nulla)			(nulla)	*(pacchetti stranieri)*
127.0.0.0	255.0.0.0	0.0.0.0			lo	***(loopback)***
192.168.0.0	255.255.255.192	192.168.0.62	U	0	eth0	*(sulle reti di provenienza)*
192.168.0.64	255.255.255.192	192.168.0.126	U	0	eth1	
192.168.0.128	255.255.255.192	192.168.0.190	U	0	eth2	

Linux indica le interfacce di inoltro con il nome convenzionale **ethx** per linee LAN (o **pppx** per linee WAN e **lo** per l'interfaccia di loopback), senza indicarne l'indirizzo IP come invece prevede lo stile di Windows. Naturalmente le interfacce eth0, eth1 e eth2 possiedono i corrispondenti indirizzi IP 192.168.0.62, 192.168.0.126, 192.168.0.190.

È necessario ricordare che il processo di instradamento attuato dai nodi di una rete TCP/IP si limita ad analizzare tutte le singole righe della TdI e a verificarne la «verità» (cfr. ***Instradamento classful***), e potrebbe succedere che alla fine del computo più di una «riga» risulti verificata.

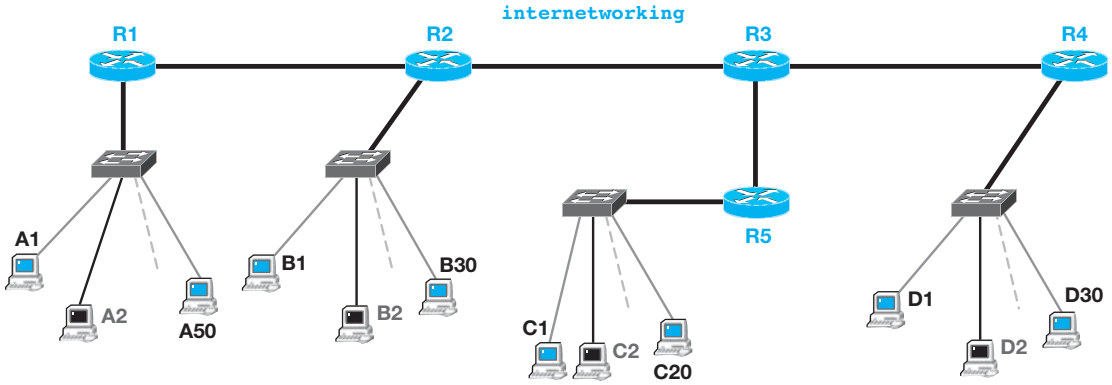
In questo caso si decide la regola di inoltro scegliendo quella che nel campo *Mask/Genmask* contiene più «uni» e, a parità di criterio, quella con il valore di *Metrica/Metric* più alto.

5 Internetwork

Un esempio significativo di subnetting avviene quando è necessario organizzare una rete con diversi router interconnessi. In questo caso l'organizzazione, quantunque utilizzi indirizzi IP privati, diventa un ottimo caso di topologia di internetwork generale.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Per esempio, si debba configurare una rete descritta dal seguente schema:



Si nota che le LAN da organizzare sono 4 (LANA, LANB, LANC, LAND), e la più numerosa contiene 50 host.

Bisogna però considerare che le connessioni punto-punto tra i router (**link**), spesso realizzata con linee Ethernet veloci, rappresentano anch'esse altrettante LAN, seppur composte, ognuna, da soli due host.

Ciò si rende necessario dato che ogni interfaccia deve disporre di un indirizzo IP, come prescrive il protocollo IP.

Naturalmente gli indirizzi IP sulle interfacce dei link non devono appartenere ai gruppi di indirizzi destinati alle 4 LAN della rete; nello stesso tempo gli indirizzi IP sullo stesso link devono appartenere alla stessa subnet per sfruttare la consegna diretta allorquando un pacchetto deve attraversare il link. In conclusione, le subnet da organizzare sono 8: 4 LAN e 4 link (R1-R2, R2-R3, R3-R4 e R3-R5).

Ora, usando i criteri noti, sono necessari 6 bit per l'hostId (2^6=64 > 50) e ne rimarrebbero solo 8–6=2 a disposizione per la subnetId, utilizzando il gruppo 192.168.0.0 con 24 bit di subnetmask. Con 2 bit per la subnetId rimangono a disposizione solo 4 subnet (2^2=4). Ciò è insufficiente, per le 8 subnet richieste.

Le scelte ora possono essere almeno due: o usare la classe di indirizzi privati 192.168.0.0 con 16 bit di subnetmask oppure la classe 172.16.0.0 con 24 bit di subnetmask. Si opta per quest'ultima.

Ora si hanno 16 bit a disposizione per subnetId e hostId, pertanto si decide una subnetId lunga 4 bit (2^4=16 subnet) per evitare una scelta troppo stringente. Si useranno le prime quattro subnet per le LAN e le ultime 4 subnet per i link.

La rete ha quindi nome **172.16.0.0** (subnetmask 255.255.240.0), con subnetId = **4** (bit) e hostId = 32–20 = **12** (bit).

Le quattro subnet per le LAN:

subnet LANA

subnetId = **0000**
da: 172.16.**0000**0000.00000000, (172.16.0.0)
a: 172.16.**0000**1111.11111111, (172.16.15.255)

subnet LANB

subnetId = **0001**
da: 172.16.**0001**0000.00000000, (172.16.16.0)
a: 172.16.**0001**1111.11111111, (172.16.31.255)

subnet LANC

subnetId = **0010**
da: 172.16.**0010**0000.00000000, (172.16.32.0)
a: 172.16.**0010**1111.11111111, (172.16.47.255)

subnet LAND

subnetId = **0011**
da: 172.16.**0011**0000.00000000, (172.16.48.0)
a: 172.16.**0011**1111.11111111, (172.16.63.255)

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Le quattro subnet per i link:

subnet R1-R2

subnetId = **0100**
da: 172.16.**0100**0000.00000000, (172.16.192.0)
a: 172.16.**0100**1111.11111111, (172.16.207.255)

subnet R2-R3

subnetId = **1101**
da: 172.16.**1101**0000.00000000, (172.16.208.0)
a: 172.16.**1101**1111.11111111, (172.16.223.255)

subnet R3-R4

subnetId = **1110**
da: 172.16.**1110**0000.00000000, (172.16.224.0)
a: 172.16.**1110**1111.11111111, (172.16.239.255)

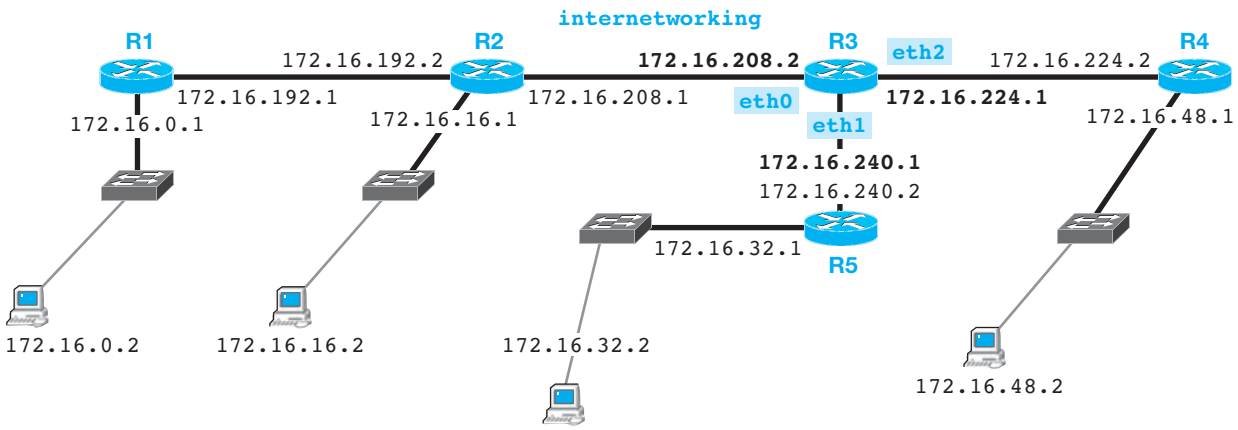
subnet R3-R5

subnetId = **1111**
da: 172.16.**1111**0000.00000000, (172.16.240.0)
a: 172.16.**1111**1111.11111111, (172.16.255.255)

Si nota che gli estremi di assegnazione delle subnet specificano il *nome* della subnet e l'indirizzo di *broadcast* della subnet.

Rimangono pertanto $2^{12} - 2 = 4096 - 2 =$ **4094** indirizzi disponibili per ogni subnet.

Ora la rete può essere rappresentata:



Si può riportare la TdI di un router, per esempio il router R3.

Bisogna scrivere una regola per ogni subnet, magari tralasciando le regole di loopback e di broadcast e trascurando la colonna della metrica per semplicità.

Per quanto riguarda la TdI del router **R3** (con interfacce 172.16.208.2, 172.16.240.1, 172.16.224.1):

Indirizzo	Mask	Gateway	Interfaccia
172.16.0.0	255.255.240.0	172.16.208.1	172.16.208.2
172.16.16.0	255.255.240.0	172.16.208.1	172.16.208.2
172.16.32.0	255.255.240.0	172.16.240.2	172.16.240.1
172.16.48.0	255.255.240.0	172.16.224.2	172.16.224.1
172.16.192.0	255.255.240.0	172.16.208.1	172.16.208.2
172.16.208.0	255.255.240.0	–	172.16.208.2
172.16.224.0	255.255.240.0	–	172.16.224.1
172.16.240.0	255.255.240.0	–	172.16.240.1

Nella colonna *Indirizzo* si riportano i nomi di tutte le subnet (8 subnet).

Nella colonna *Mask* la netmask relativa alle subnet (stessa per tutta la rete).

Le prime quattro regole riguardano le 4 LAN, le successive 4 i link.

Si nota che le ultime tre subnet (i link R2-R3, R3-R4 e R3-R5) sono direttamente connesse con R3, e infatti la consegna è *diretta*.

VLSM

In origine, e prima dell'avvento di CIDR, il subnetting prevedeva tassativamente che in una rete «subnettata» potesse definirsi una sola subnet mask, comune a tutte le sottoreti.

Con **VLSM** (*Variable Length Subnet Masks*, RFC 1009) tale limitazione è stata abolita. Un esempio di rete a cui viene applicato VLSM è riportato in *Appendice*.

VLSM consente una distribuzione più razionale degli indirizzi IP, evitando sprechi e ottimizzando il blocco degli indirizzi a disposizione (vedi **Esercizio 25**).

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

La stessa TdI del router **R3** può essere scritta in maniera più compatta:

Rete	Next-hop	Interfaccia
172.16.0.0/20	172.16.208.1	eth0
172.16.16.0/20	172.16.208.1	eth0
172.16.32.0/20	172.16.240.2	eth1
172.16.48.0/20	172.16.224.2	eth2
172.16.192.0/20	172.16.208.1	eth0
172.16.208.0/20	-	eth0
172.16.224.0/20	-	eth2
172.16.240.0/20	-	eth1

dove: **eth0:** 172.16.208.2
eth1: 172.16.240.1
eth2: 172.16.224.1

Ora si può indicare la configurazione di un **host** della sottorete LANA, per esempio quello con indirizzo IP 172.16.0.2, subnet mask 255.255.240.0 e gateway di default 172.16.0.1.

Analogamente per un **host** della sottorete LAND, per esempio quello con indirizzo IP 172.16.48.2, subnet mask 255.255.240.0 e gateway di default 172.16.48.1.

6 Il programma route

Sia Windows che Linux mettono a disposizione un programma denominato **route** che consente di gestire la TdI del nodo su cui viene eseguito.

L'uso più proficuo è quello che consente di visualizzare la TdI:

```
C:\Windows>route print
===========================================================================
Elenco interfacce
 14...c0 cb 38 a9 fd a8 ....Bluetooth Device (Personal Area Network)
 12...68 a3 c4 16 d8 ac ....DW1501 Wireless-N WLAN Half-Mini Card
 11...5c 26 0a 29 be 34 ....Intel(R) 82577LM Gigabit Network
  1...........................Software Loopback Interface 1
IPv4 Tabella route
===========================================================================
Route attive:
     Indirizzo rete           Mask          Gateway     Interfaccia   Metrica
          0.0.0.0          0.0.0.0    192.168.0.1     192.168.0.17     21
        127.0.0.0        255.0.0.0         On-link        127.0.0.1    306
        127.0.0.1  255.255.255.255         On-link        127.0.0.1    306
  127.255.255.255  255.255.255.255         On-link        127.0.0.1    306
      192.168.0.0    255.255.255.0         On-link      192.168.0.17    276
     192.168.0.17  255.255.255.255         On-link      192.168.0.17    276
    192.168.0.255  255.255.255.255         On-link      192.168.0.17    276
        224.0.0.0        240.0.0.0         On-link        127.0.0.1    306
        224.0.0.0        240.0.0.0         On-link      192.168.0.17    276
  255.255.255.255  255.255.255.255         On-link        127.0.0.1    306
  255.255.255.255  255.255.255.255         On-link      192.168.0.17    276
===========================================================================
Route permanenti:
   Indirizzo rete            Mask   Indir. gateway Metrica
          0.0.0.0          0.0.0.0      192.168.0.1       1
===========================================================================
C:\Windows>
```


Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Il comando *route print* è molto efficace se associato alla redirezione su file, in modo tale da conservare e analizzare la TdI (*route print > tdi.txt*).

```
linux:~# route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.0.0     0.0.0.0         255.255.255.0   U     1      0        0 eth0
0.0.0.0         192.168.0.211   0.0.0.0         UG    0      0        0 eth0
linux:~#
```

Con *route* si possono anche aggiungere ed eliminare regole (*route statiche*):

```
C:\Windows>route add 10.0.0.0 mask 255.0.0.0 192.168.0.1
 OK
C:\Windows>
```

```
linux:~# route add -net 10.0.0.0 netmask 255.0.0.0 gw 192.168.0.1
linux:~#
```

Si nota che l'ultimo parametro è il gateway.

In questo caso la regola viene persa al riavvio del sistema (per rendere persistente una regola si usa il parametro –p).

Per eliminare regole, invece:

```
C:\Windows>route delete 10.0.0.0
 OK
C:\Windows>
```

```
linux:~# route del -net 10.0.0.0 netmask 255.0.0.0 gw 192.168.0.1
linux:~#
```

7 Supernetting

Calcolatrici IP

Decidere il piano di indirizzamento di una rete IP dipende da vari criteri (esempio, il numero di subnet necessarie e il numero di host da indirizzare) e comporta numerosi calcoli (esempio, per individuare il nome della subnet o il suo indirizzo di broadcast).

Per facilitare questi compiti sono disponibili numerose versioni di **calcolatrici IP** (*IP Calculator*) che effettuano gran parte dei calcoli richiesti da un piano di indirizzamento, inserendo i criteri desiderati.

→

Il problema dello spreco degli indirizzi IP distribuiti dagli ISP alle organizzazioni richiedenti, spreco causato dalla rigidità dimensionale delle classi definite dallo schema classful, è stato affrontato con il **supernetting**. Il supernetting fa parte di un insieme di disposizioni emanate nel 1993 e denominato **CIDR** (*Classless Inter-Domain Routing*, RFC 1519).

Il CIDR spiega che:

1. la metà superiore della classe A (da 64 a 127) è riservata;
2. gli indirizzi di classe B sono assegnati solo se la rete ha almeno 32 sottoreti e oltre 4096 host complessivi;
3. gli indirizzi della metà inferiore della classe C sono assegnati a blocchi contigui in base alla localizzazione geografica;
4. gli indirizzi della metà superiore della classe C (da 208.0.0 a 223.255.255) non sono assegnati.

Come conseguenza del punto 3, il supernetting riesce ad aggregare più reti di classe C in una stessa rete fisica, superando il limite imposto dallo sche-

ma classful. In questo modo un ISP, invece di vendere un blocco di indirizzi di classe B a un'organizzazione richiedente che necessita di poche migliaia di indirizzi (sprecando altrettante migliaia di indirizzi), può aggregare un numero idoneo di reti di classe C e cederle come se fossero un unico blocco compatto rappresentante un'unica rete fisica.

Per ottenere ciò si usa una sequenza di 32 bit con la quale si ridefinisce la parte netId dell'indirizzo IP (con la solita operazione di AND), consentendo di nuovo la regola di routing implicito su una stessa rete fisica, questa volta aggregata. Questa sequenza, detta anche **maschera di supernetting**, deve sempre essere specificata per le reti che adottano CIDR.

CIDR introduce anche la notazione per la maschera di supernetting attraverso il carattere "/" (slash), indicata in gergo CIDR con **a.b.c.d/n**.

Per esempio, la notazione 192.192.0.0/22 significa che i primi 22 bit degli indirizzi devono essere mascherati in AND con altrettanti «uni» per ottenere il netId, ovvero per aggregare 4 reti di classe C (la cui netmask classful vale 24 bit).

Infatti 24–22=2, e questi 2 bit consentono di numerare 2^2=4 reti: 192.192.**0**.x, 192.192.**1**.x, 192.192.**2**.x, 192.192.**3**.x.

→ Ottime versioni di calcolatrici IP sono disponibili sul Web, come per esempio:

http://www.subnet-calculator.com

In questo sito è disponibile anche il calcolo con notazione CIDR.

ESEMPIO

Supernetting

Un Service Provider che ha ottenuto da un LIR un blocco di indirizzi IP descritto da 210.1.8.0/21 riceve la domanda di indirizzi da 4 aziende: A deve indirizzare 1000 indirizzi macchine, B 500, C 200 e D 100.

Mostrare le maschere di supernetting utilizzate dai router dell'ISP.

Per prima cosa si analizza la notazione 210.1.8.0/21.

Si nota che si tratta di un gruppo di indirizzi di classe C, come si desume dal primo byte (191<**210**<224).

Ricordiamo che in classful una rete di classe C ha la seguente netmask, netId e hostId:

```
11111111.11111111.11111111.00000000
  255   .   255   .   255   .    0
        netId (24 bit)          hostId (8 bit)
```

In notazione binaria, il blocco CIDR si può scrivere mettendo in evidenza inuovi netId e l'hostId:

```
   210   .    1    .    8    .    0      /   21
11010010.00000001.00001000.00000000
         netId (21 bit)       hostId (11 bit)
```

da cui si deduce che i bit a 1 di hostId del blocco CIDR posizionati a sinistra della netmask di rete classe C sono 3 (11–8=3).

Il gruppo assegnato quindi concatena 2^3=8 reti di classe C: 000, 001, 010, 011, 100, 101, 110, 111.

Per individuarle, basta sostituire le triplette di bit nella posizione indicata dai MSb dell'hostId della rete assegnata:

```
11010010.00000001.00001000.00000000, ovvero 210.1.8.0
11010010.00000001.00001001.00000000, ovvero 210.1.9.0
11010010.00000001.00001010.00000000, ovvero 210.1.10.0
11010010.00000001.00001011.00000000, ovvero 210.1.11.0
11010010.00000001.00001100.00000000, ovvero 210.1.12.0
11010010.00000001.00001101.00000000, ovvero 210.1.13.0
11010010.00000001.00001110.00000000, ovvero 210.1.14.0
11010010.00000001.00001111.00000000, ovvero 210.1.15.0
```

In totale il blocco assegnato dal LIR vale 2^{11}=2048 indirizzi. ▶

ESEMPIO

Per individuare le maschere di supernetting valide per le 4 aziende, si procede in questo modo:

1. Siccome la richiesta di maggior portata vale 1000 (Azienda A), i 2048 indirizzi devono essere divisi in due, e una delle due metà va assegnata all'azienda A.

Per far ciò si riserva il primo bit della parte hostId: per il valore 0 esso designerà i 1024 indirizzi per l'azienda A, per il valore 1, avremo i rimanenti indirizzi (da distribuire a B, C e D):

```
11001001.00000001.00001000.00000000 /22
  210   .   1   .    8    .    0     /22 per l'azienda A.

11001001.00000001.00001100.00000000 /22
  210   .   1   .   12    .    0     /22 per i rimanenti B,C e D.
```

2. Il rimanente gruppo di 1024 indirizzi deve essere ulteriormente suddiviso. Si nota che la richiesta maggiore vale 500 (Azienda B). Pertanto si effettua un nuovo dimezzamento riservando un ulteriore bit (a zero) del nuovo hostId:

```
11001001.00000001.000011[0]0.00000000 /23
  210   .   1   .   12    .    0     /23 per l'azienda B.

11001001.00000001.000011[1]0.00000000 /23
  210   .   1   .   14    .    0     /23 per i rimanenti C e D.
```

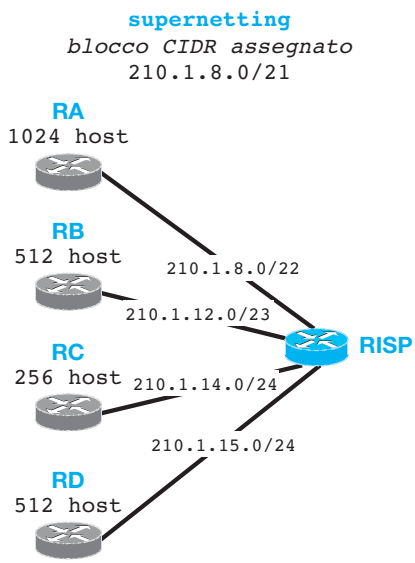
3. L'ultimo gruppo di 512 indirizzi deve essere ripartito tra le aziende C e D. Nessuna delle due esige più della metà a disposizione (256), quindi il gruppo può essere suddiviso con un ulteriore bit della parte hostId:

```
11001001.00000001.0000111[0].00000000 /24
  210   .   1   .   14    .    0     /24 per l'azienda C.

11001001.00000001.0000111[1].00000000 /24
  210   .   1   .   15    .    0     /24 per l'azienda D.
```

In definitiva le sole quattro maschere di supernetting che l'ISP dovrà prevedere nelle sue tabelle di instradamento sono:

210.1. 8.0/22 per l'azienda A;
210.1.12.0/23 per l'azienda B;
210.1.14.0/24 per l'azienda C;
210.1.15.0/24 per l'azienda D.



NB. *La notazione CIDR a.b.c.d/n, per la sua compattezza, è stata adottata anche come notazione per il subnetting, cosicché una rete di classe C 192.168.0.0 con maschera di subnet 255.255.255.192 può essere indicata come 192.168.0.0/26.*

8 Autonomous System

Lo schema di internetworking al di fuori delle aree private mostrato nel paragrafo precedente funziona solo perché tra gli ISP, nel loro complesso, viene organizzata una gerarchia che li consolida in gruppi di reti specifiche detti **Autonomous System** (**AS**, RFC 1771).

Un AS è quindi un insieme di router gestito da una singola amministrazione tecnica, dotato di un identificativo numerico univoco su scala mondiale (**ASN**, *Autonomous System Number*) rilasciato da un RIR in accordo con IANA.

Si può dire che il routing complessivo su Internet tra due host qualsiasi avviene su quattro livelli gerarchici.

Infatti, a partire dal basso e in ambito locale, troviamo il routing all'interno di una subnet, risolto dal singolo host con la consegna diretta.

Quindi abbiamo un routing di secondo livello tra le subnet di una rete, risolto da un router locale che instrada con consegna indiretta.

Una volta uscito dall'ambito locale attraverso un ISP, il pacchetto entra nella rete TCP/IP «pubblica» generando il cosiddetto **transito IP**.

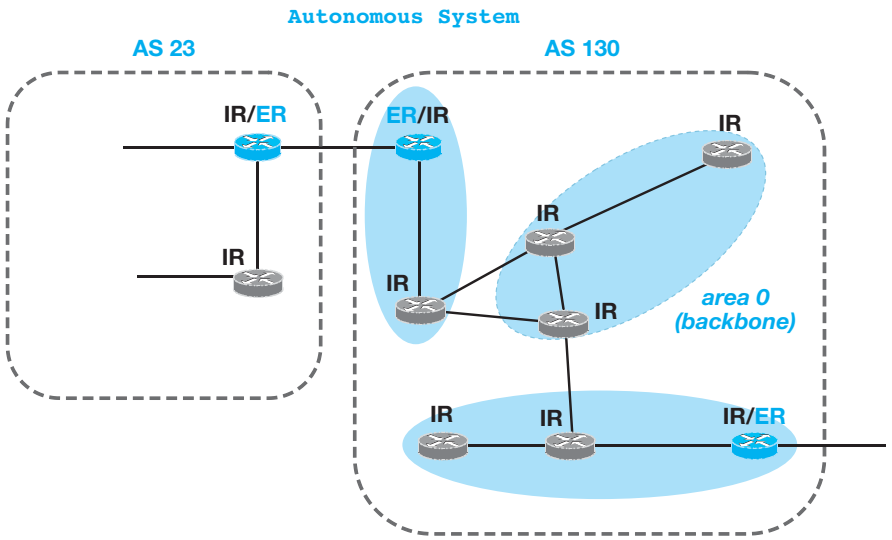
Qui avvengono gli altri due livelli di routing:

- all'interno di un Autonomous System (routing **intra-AS**);
- tra gli Autonomous System (routing **inter-AS**).

All'interno di un AS operano due tipi di router, in relazione al routing intra o extra AS: quelli che instradano i pacchetti solo con altri router dell'AS detti **IR** (*Interior Router*) e quelli che hanno il compito di collegare fra di loro AS differenti, detti **ER** (*Exterior Router*).

Un AS è governato da due famiglie distinte di protocolli di routing: la prima, **IGP** (*Interior Gateway Protocol*), responsabile del routing intra-AS; la seconda, **EGP** (*Exterior Gateway Protocol*), utilizzata nel contesto del routing inter-AS.

I router IR normalmente utilizzano lo stesso IGP in tutto l'AS; i router ER devono implementare sia un IGP (per comunicare con gli IR del proprio AS) sia un EGP (per comunicare con ER di altri AS).



Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

Per facilitare il routing intra-AS i router vengono partizionati in aree numerate in modo che l'area 0 (o **backbone**) rappresenti il centro stella di una topologia logica in cui i nodi sono le aree stesse.

Gli AS che hanno più di una connessione verso altri AS sono detti **multihoming**. In questo caso il transito IP è bilanciato, ovvero se una linea che accede a Internet cade, l'AS può sopperire con le rimanenti e normalmente può decidere regole di distribuzione del transito IP in base a vari criteri. In questi casi è necessario utilizzare un routing EGP.

Negli altri casi l'AS può impostare il routing inter-AS con routing statico (cfr. ***FDR***), dato che tutto il traffico in uscita dall'AS deve essere instradato verso l'unico AS collegato.

Siccome ogni AS è gestito da una singola autorità amministrativa, che spesso coincide con un'azienda privata (cfr. ***Internet Service Provider***), il routing intra-AS è motivato da efficienza e velocità, mentre il routing inter-AS è regolato anche da criteri politico-economici che spesso contrastano i requisiti di ottimalità nella scelta dei percorsi. Non è un caso che gli algoritmi intra-IS siano spesso basati su *Link State Packet*, algoritmo estremamente efficiente e deterministico, mentre il routing inter-AS sia praticamente basato solo su algoritmi di tipo *Distance Vector*.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

ESERCIZI PER LA VERIFICA ORALE

Saper rispondere ai **requisiti fondamentali** dà una sufficiente garanzia per sentirsi pronti all'interrogazione. Saper anche rispondere ai **requisiti avanzati** dimostra una padronanza eccellente degli argomenti del capitolo.

Requisiti fondamentali

1. Discutere il rapporto tra interfacce fisiche e indirizzi IP.
2. Elencare le fasi della distribuzione degli indirizzi IP pubblici attraverso gli enti preposti.
3. Illustrare le classi di indirizzi IP classful, ricordandone gli intervalli.
4. Descrivere le nozioni di netId, HostId e netmask.
5. Ricordare la regola di instradamento implicita attraverso i termini consegna diretta e gateway di default.
6. Descrivere le colonne tipiche di una TdI per Windows/Linux.
7. Elencare i problemi intrinseci alla gerarchia classful.
8. Discutere la tecnica di subnetting attraverso gli indirizzi privati e la nozione di subnet mask.
9. Illustrare la tecnica CIDR e il suo obiettivo.
10. Definire Autonomous System e ricordare che tipi di routing prevede.
11. Riuscire a calcolare quante subnet si possono definire con la notazione `192.168.0.0/25`.
12. Spiegare quali valori sono indispensabili ad un host per essere operativo su una rete TCP/IP.
13. Disegnare su un foglio uno schema di rete con un router e tre LAN, quindi ipotizzare una organizzazione degli indirizzi su `192.168.0.0/26` assegnando indirizzi alle interfacce del router.

Requisiti avanzati

1. Spiegare per quale motivo un ES (o router o gateway) deve avere più di una interfaccia di rete.
2. Ricordare cosa è l'hostname.
3. Collocare l'ISP nella catena degli enti che distribuiscono gli indirizzi IP pubblici.
4. Elencare i leading bit per le tre prime classi di indirizzi IP dello schema classful.
5. Indicare le lunghezze in bit del netId e hostId per le classi di indirizzi IP dello schema classful.
6. Descrivere la regola di instradamento classful attraverso le regole nella TdI.
7. Ricordare i tre blocchi di indirizzi riservati per le reti TCP/IP private.
8. Indicare almeno tre indirizzi speciali di IP e spiegare quali limitazioni introduce la loro adozione.
9. Spiegare l'uso del comando route.
10. Elencare i nomi simbolici per le interfacce usati da Linux nelle TdI.
11. Illustrare il concetto di link e subnet di link.
12. Ricordare almeno due prescrizioni CIDR.
13. Spiegare le diverse politiche di routing adottate all'interno degli Autonomous System.

ESERCIZI PER LA VERIFICA SCRITTA

Indirizzi IP e instradamento classful

1 Determinare la classe di appartenenza dei seguenti indirizzi IP e individuarne uno non valido: `208.34.54.12`, `238.34.2.1`, `114.34.2.8`, `129.14.6.8`, `241.34.2.8`, `10.1.1.1`, `129.0.1.5`, `234.1.0.9`, `1.1.1.1`, `193.2.1.89`, `134.5.67.3`, `100.200.300.1`.

2 Determinare la classe di appartenenza dei seguenti indirizzi IP e individuarne uno non valido: `255.255.1.0`, `2.2.2.2`, `1.2.3.4`, `127.1.4.1`, `127.0.0.0`, `191.191.9.911`, `16.2.3.4`, `150.0.0.1`, `4.3.2.1`, `194.2.3.0`, `100.200.100.200`.

3 Dati gli indirizzi IP dell'Esercizio 1, estrarne netId e hostId, quindi riportare un secondo indirizzo appartenente alla stessa rete.

4 Dati gli indirizzi IP dell'Esercizio 2, estrarne netId e hostId, quindi riportare un secondo indirizzo appartenente alla stessa rete.

5 Data la seguente TdI di un host:

Indirizzo	Mask	Gateway	Interfaccia
0.0.0.0	0.0.0.0	196.13.1.254	196.13.1.1
196.13.1.0	255.255.255.0	196.13.1.1	196.13.1.1

desumere l'indirizzo IP dell'host e riportare due indirizzi IP di reti differenti che vengono instradati in modo indiretto e due indirizzi IP che sono instradati in modo diretto.

6 Data la seguente TdI di un host:

Indirizzo	Mask	Gateway	Interfaccia
196.13.1.0	255.255.255.0	0.0.0.0	eth0
0.0.0.0	0.0.0.0	192.168.0.1	eth0

ipotizzare un indirizzo IP dell'host e riportare due indirizzi IP di reti differenti che vengono instradati in modo indiretto e due indirizzi IP che sono instradati in modo diretto.

7 Data la seguente TdI di un nodo:

Indirizzo	Mask	Gateway	Interfaccia
0.0.0.0	0.0.0.0	10.1.1.2	10.1.1.1
172.16.0.0	255.255.0.0	172.16.0.1	172.16.0.1
172.17.0.0	255.255.0.0	172.17.0.1	172.17.0.1

desumere gli indirizzi IP del nodo quindi riportare due indirizzi IP di reti differenti che vengono instradati in modo indiretto e due indirizzi IP che sono instradati in modo diretto.

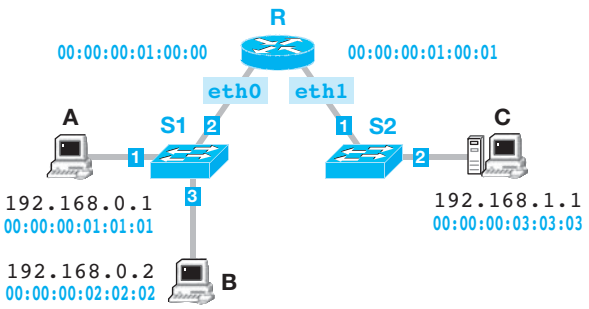
8 Sia dato il seguente schema di rete, con indicati gli indirizzi di livello 3 (IP address) e livello 2 (MAC address). Tutti i sistemi sono stati appena avviati, quindi le ARP cache sono vuote così come le CAM degli switch.

a) Decidere i due indirizzi IP per le interfacce eth0 e eth1 del router R.
b) Riportare la configurazione degli host.

L'host A quindi esegue il comando:

```
ping 192.168.1.1
```

c) Elencare tutti i frame generati dal comando fino al suo termine.
d) Elencare le ARP cache di ogni host al termine del comando.
e) Riportare i filtering database degli switch al termine del comando.



9 Rispondere ai quesiti c), d) e e) dell'esercizio precedente, se il comando eseguito dall'host A è `ping 192.168.0.2`.

10 Rispondere ai quesiti c), d) e e) dell'esercizio precedente, se il comando eseguito dall'host A è `ping 10.1.1.1`.

11 Data la seguente TdI di un nodo:

Indirizzo	Mask	Gateway	Interfaccia
0.0.0.0	0.0.0.0	10.1.1.2	10.1.1.1
120.0.0.0	255.0.0.0	120.16.0.1	120.16.0.1
121.0.0.0	255.0.0.0	172.17.0.1	120.17.0.1

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

desumere gli indirizzi IP del nodo quindi riportare due indirizzi IP di reti differenti che vengono instradati in modo indiretto e due indirizzi IP che sono instradati in modo diretto.

Subnetting e VLSM

12 Data la seguente notazione `192.168.1.0` con subnetmask `255.255.255.192`, individuare la notazione CIDR equivalente quindi riportare tutte le sottoreti e indicare quanti host per sottorete.

13 Data la seguente notazione `200.1.1.0` con subnetmask `255.255.255.192`, individuare la notazione CIDR equivalente quindi riportare tutte le sottoreti e indicare quanti host per sottorete.

14 Data la seguente notazione `192.168.0.0` con subnetmask `255.255.192.0`, individuare la notazione CIDR equivalente quindi riportare tutte le sottoreti e indicare quanti host per sottorete.

15 Data la seguente notazione `190.1.0.0` con subnetmask `255.255.192.0`, individuare la notazione CIDR equivalente quindi riportare tutte le sottoreti e indicare quanti host per sottorete.

16 Data la seguente notazione `10.0.0.0` con subnetmask `255.192.0.0`, individuare la notazione CIDR equivalente quindi riportare tutte le sottoreti e indicare quanti host per sottorete.

17 Dato il blocco di indirizzi privato `192.168.0.0/24`, definire una subnetmask tale da poter organizzare una serie di LAN il cui gruppo più numeroso vale 72 host.

18 Dato il blocco di indirizzi privato `192.168.0.0/16`, definire una subnetmask tale da poter organizzare una serie di LAN il cui gruppo più numeroso vale 12000 host.

19 Dato il blocco di indirizzi privato `10.0.0.0/8`, definire una subnetmask tale da poter organizzare una serie di LAN il cui gruppo più numeroso vale 120000 host.

20 Va organizzata una LAN con circa 2500 host e si dispone del gruppo di indirizzi `172.16.0.0` con netmask `255.255.0.0`. Decidere la subnetmask e una subnet, quindi riportare il nome della subnet e il suo indirizzo di broadcast. Quante subnet rimangono disponibili? Quanti indirizzi rimangono disponibili?

21 [VLSM] Circa l'esercizio precedente, individuare una seconda subnetmask tale che possa indirizzare ulteriori LAN con un massimo di 50 host l'una. Quante subnet di questo tipo risultano?

22 Va organizzata una LAN con 50 host e si dispone del gruppo di indirizzi `192.168.0.0/24`.
Decidere la subnetmask e una subnet, quindi riportare il nome della subnet e il suo indirizzo di broadcast. Quante subnet rimangono disponibili? Quanti indirizzi rimangono disponibili?

23 [VLSM] Circa l'esercizio precedente, individuare una seconda subnetmask tale che possa indirizzare ulteriori LAN con un massimo di 10 host l'una. Quante subnet di questo tipo risultano?

24 Un host ha indirizzo IP `140.252.1.1` e subnetmask `255.255.248.0`. Stabilire se gli host con i seguenti indirizzi IP appartengono alla stessa sottorete: `140.252.4.5`, `140.252.128.1`, `140.252.1.22`, `192.43.235.6`.

25 [VLSM] Un'organizzazione dispone di una rete di classe C `199.1.1.0/24` e deve organizzare quattro LAN da 70, 30, 20 e 17 host. Mostrare che è necessario usare VLSM. Riportare le subnet mask delle 4 reti.

26 Data la rete `194.0.13.0/26`, riportare i nomi delle sottoreti, il range di indirizzi per ogni sottorete e l'indirizzo di broadcast per ogni sottorete.

27 Data la rete `170.13.0.0/18`, riportare i nomi delle sottoreti, il range di indirizzi per ogni sottorete e l'indirizzo di broadcast per ogni sottorete.

28 Un'organizzazione dispone di una rete di classe C 199.1.1.0 e deve organizzare quattro LAN da 10, 30, 20 e 17 host. Definire una subnetmask appropriata e individuare per ogni sottorete il nome, il range di indirizzi e l'indirizzo di broadcast.

29 Un'organizzazione dispone di una rete di classe C `200.1.2.0` e deve organizzare sette LAN da 10, 10, 12, 28, 20, 2 e 17 host. Definire una sub-

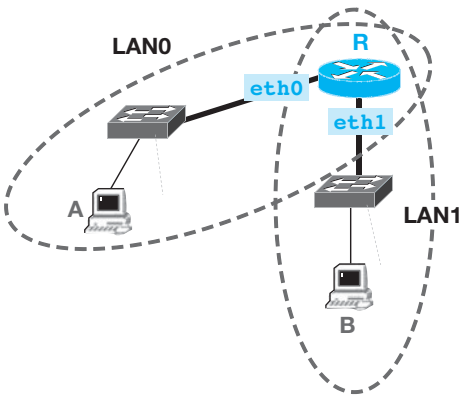
Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

netmask appropriata e individuare per ogni sottorete il nome, il range di indirizzi e l'indirizzo di broadcast.

Reti e inoltro

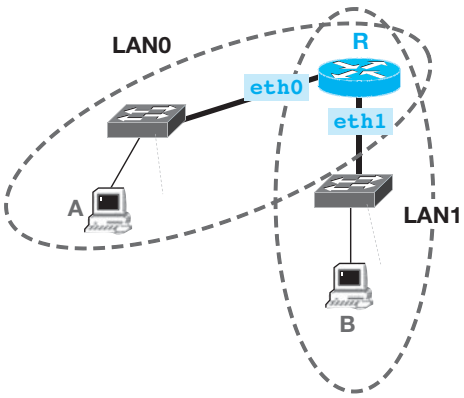
30 Dato il seguente schema di rete con due LAN (32 e 77 host) e il blocco di indirizzi `192.168.3.0/24`:

a) decidere una subnetmask idonea per pianificare gli indirizzi dei nodi;
b) assegnare indirizzi appropriati alle interfacce del router;
c) riportare la configurazione di un host A (LAN0) e di un host B (LAN1);
d) compilare la TdI del router semplificata (*senza metrica né regole di loopback e broadcast*).



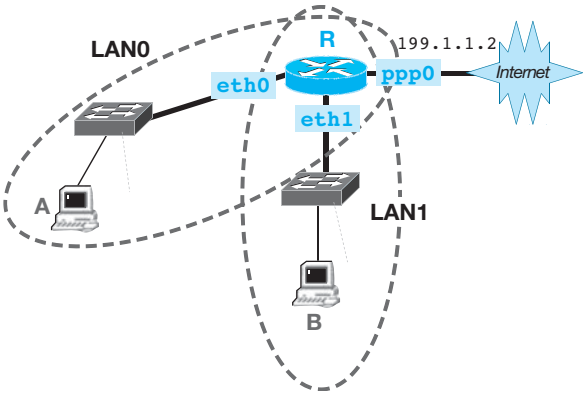
31 Dato il seguente schema di rete con due LAN (320 e 770 host) e il blocco di indirizzi `192.168.0.0/16`:

a) decidere una subnetmask idonea per pianificare gli indirizzi dei nodi;
b) assegnare indirizzi appropriati alle interfacce del router;
c) riportare la configurazione di un host A (LAN0) e di un host B (LAN1);
d) compilare la TdI del router semplificata.



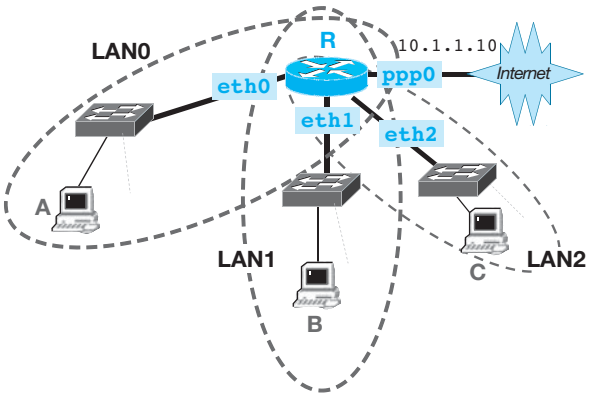
32 Dato il seguente schema di rete con due LAN, un collegamento a Internet e la rete `172.16.0.0/16`:

a) definire le due subnet e assegnare indirizzi appropriati alle interfacce del router;
b) riportare la configurazione di un host A (LAN0) e di un host B (LAN1);
c) compilare la TdI del router semplificata.



33 Dato il seguente schema di rete con tre LAN, un collegamento a Internet e la rete `192.168.0.0/24`:

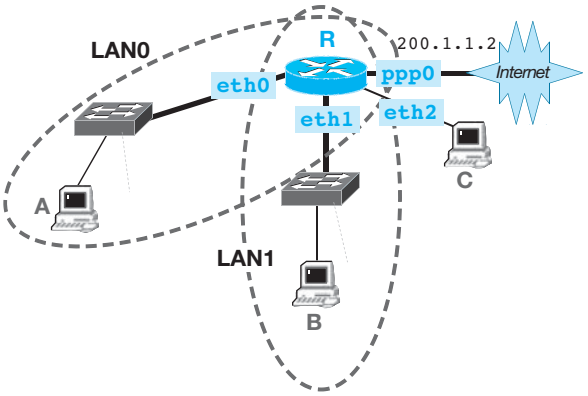
a) definire le subnet e assegnare indirizzi appropriati alle interfacce del router;
b) riportare la configurazione di un host A (LAN0) e di un host C (LAN2);
c) compilare la TdI del router semplificata.



34 Dato il seguente schema di rete con due LAN, un collegamento a Internet e la rete `192.168.0.0/16`:

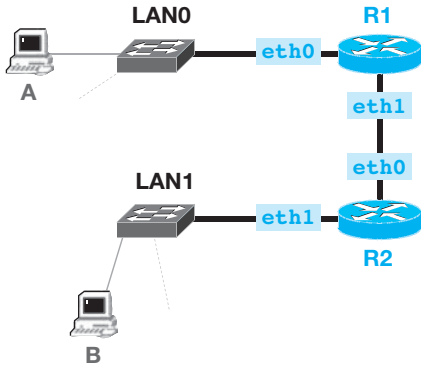
a) definire le subnet e assegnare indirizzi appropriati alle interfacce del router;
b) riportare la configurazione di un host A (LAN0) e dell'host H;
c) compilare la TdI del router semplificata.

Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*



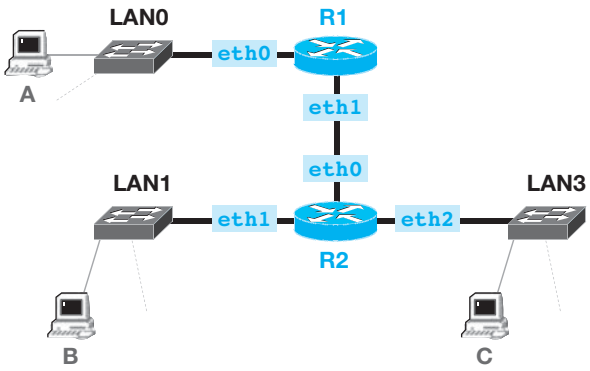
35 Dato il seguente schema di rete:

a) definire le subnet e assegnare indirizzi appropriati alle interfacce del router;
b) riportare la configurazione dell'host A (LAN0) e dell'host B (LAN2);
c) compilare le TdI dei router semplificate.



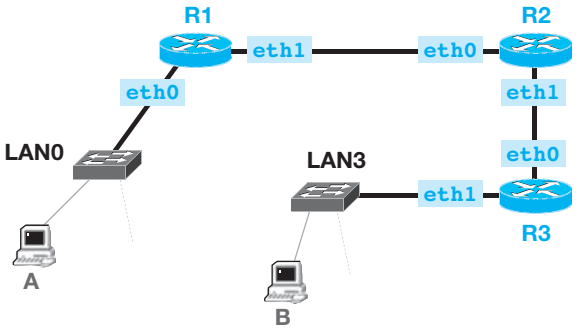
36 Dato il seguente schema di rete:

a) definire le subnet e assegnare indirizzi appropriati alle interfacce dei router;
b) riportare la configurazione dell'host A (LAN0) e dell'host B (LAN2);
c) compilare le TdI dei router semplificate.

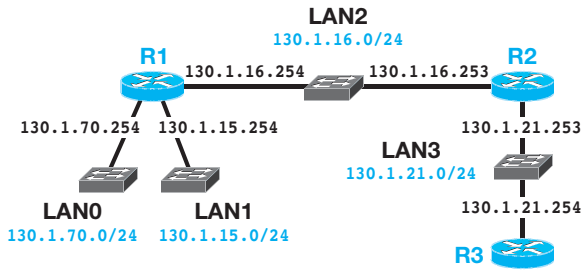


37 Dato il seguente schema di rete:

a) definire le subnet e assegnare indirizzi appropriati alle interfacce del router;
b) riportare la configurazione dell'host A (LAN0) e dell'host B (LAN3);
c) compilare le TdI dei router semplificate.

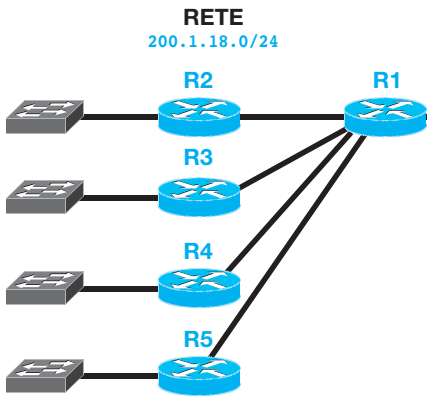


38 Data la seguente rete, compilare le TdI dei router.



39 [VLSM] Data la seguente rete, definita come `200.1.18.0/24`, con LAN0 100 host, LAN1 50 host, LAN2 30 host, LAN3 15 host e LAN4 10 host:

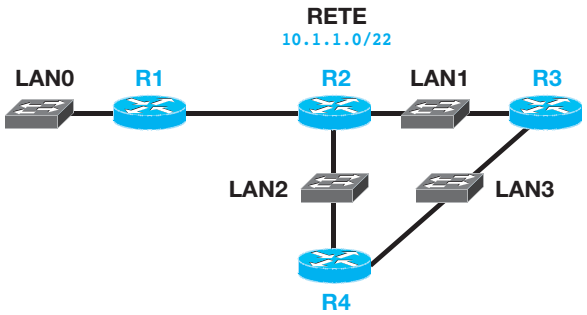
a) individuare le sottoreti opportune;
b) assegnare gli indirizzi alle interfacce dei router;
c) compilare le TdI dei router.



Paolo Ollari **CORSO DI SISTEMI E RETI - Vol. 2** © Zanichelli 2013 *per Informatica*

40 [VLSM] Data la seguente rete, definita come `10.1.1.0/22`, con LAN0 500 host, LAN1 250 host, LAN2 120 host, LAN3 100 host:

a) individuare le sottoreti opportune;
b) assegnare gli indirizzi alle interfacce dei router;
c) compilare le TdI dei router.



41 Ad un ISP è assegnato lo spazio di indirizzi CIDR `200.1.20.0/22`.
Mostrare come è possibile suddividere gli indirizzi tra i seguenti clienti:

A: 400 host; B: 200 host; C,D: 50 host; E: 30 host; F : 20 host; G : 10 host.

42 Ad un ISP è assegnato lo spazio di indirizzi CIDR `140.175.0.0/21`.
Mostrare come è possibile suddividere gli indirizzi tra i seguenti clienti:

A: 900 host; B,C,D: 200 host; E,F,G: 50 host e H,I,L,M con soli 2 host.